

# Appunti di Ingegneria del Software

Appunti basati sulle lezioni di Davide Falessi e Gulyx

A.A. 2025/2026

# Indice

|    |                                                                   |     |
|----|-------------------------------------------------------------------|-----|
| 1  | Teoria della Misurazione e Scale di Misura                        | 4   |
| 2  | Software Metrics and Their Use                                    | 8   |
| 3  | GQM+S, Stima dei Costi e Planning Poker                           | 16  |
| 4  | Introduzione all'Architettura Software                            | 20  |
| 5  | Debito Tecnico e Code Smells                                      | 24  |
| 6  | Machine Learning e Software Analytics per la Software Engineering | 27  |
| 7  | SZZ: Identifying Buggy Entities                                   | 39  |
| 8  | Leveraging Defects Lifecycle for Labeling Defective Classes       | 46  |
| 9  | Defect Labeling, Evaluation and Metrics                           | 54  |
| 10 | Snoring: rumore nei dataset di defect prediction                  | 68  |
| 11 | Effort-Aware Accuracy Metrics e valutazione dei classificatori    | 76  |
| 12 | Milestone 1: Dataset Creation                                     | 85  |
| 13 | Balancing                                                         | 92  |
| 14 | Weka GUI e Feature Selection                                      | 98  |
| 15 | Milestone 2: Classifiers Accuracy                                 | 106 |
| 16 | What If I Had No Smells?                                          | 113 |
| 17 | Characterizing Automated Refactoring                              | 122 |
| 18 | Introduzione e concetti generali di Software Testing              | 128 |
| 19 | Test Automation e Continuous Testing                              | 135 |
| 20 | Unit Testing, Integration Testing, JUnit, Mockito e Maven         | 144 |
| 21 | Unit and Integration Testing: Frameworks                          | 154 |

|           |                                                   |            |
|-----------|---------------------------------------------------|------------|
| <b>22</b> | <b>Approcci alla generazione dei test</b>         | <b>165</b> |
| <b>23</b> | <b>Adeguatezza dei test e criteri di coverage</b> | <b>187</b> |
| <b>24</b> | <b>Mutation Testing</b>                           | <b>196</b> |
| <b>25</b> | <b>Coverage-Driven Test Generation</b>            | <b>203</b> |
| <b>26</b> | <b>Evosuite Test Generation</b>                   | <b>209</b> |

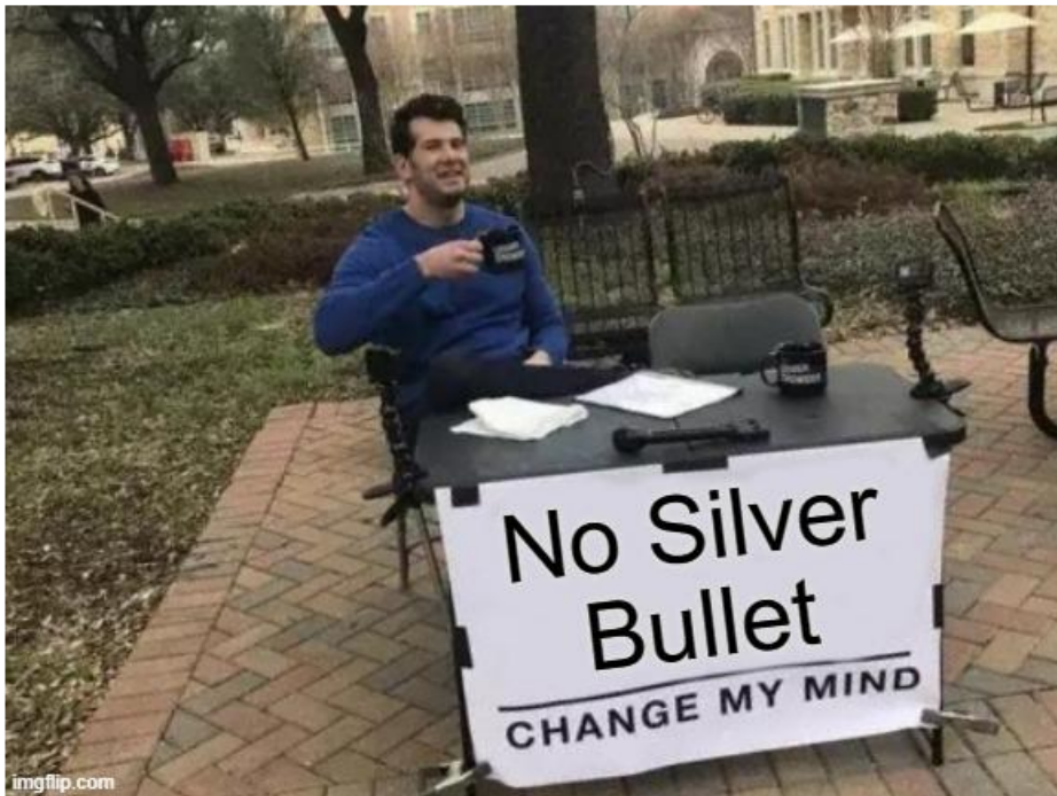


Figura 1:

# Capitolo 1

## Teoria della Misurazione e Scale di Misura

### 1.1 Introduzione alla Misurazione del Software

La misurazione nel software engineering non è un fine, ma uno strumento decisionale di supporto. Come ricordano Lord Kelvin e Tom DeMarco, l'assenza di misurazione impedisce il controllo e, di conseguenza, il miglioramento dei processi e dei prodotti.

#### 1.1.1 Definizioni Formali

Secondo i modelli standard, è necessario distinguere tre concetti fondamentali:

- **Misura (Measure):** Rappresentazione numerica di una caratteristica o di un attributo di un prodotto o processo software (es. 5000 LOC, 30 mesi-uomo).
- **Metrica (Metric):** La scala o il sistema di misurazione utilizzato per quantificare una misura. Definisce le regole di assegnazione dei valori (es. Complessità Ciclomantica, Densità dei Difetti).
- **Misurazione (Measurement):** Il processo pratico di assegnazione dei numeri alle misure utilizzando una metrica.

#### 1.1.2 Proprietà di una Metrica di Qualità

Per essere considerata valida, una metrica deve soddisfare i seguenti criteri:

**Rilevanza:** Deve misurare un attributo significativo e importante del prodotto o processo software.

**Oggettività:** Il risultato deve basarsi su dati verificabili e riproducibili, non su opinioni o stime soggettive.

**Affidabilità e Consistenza:** La metrica deve produrre risultati costanti in condizioni diverse e deve essere coerente con gli obiettivi del progetto.

**Usabilità:** Il calcolo e l'interpretazione devono essere facili, intuitivi e non richiedere sforzi o risorse eccessive.

### Nota del Prof

La pertinenza di una metrica è strettamente legata al contesto di business. Un errore critico in un sistema assicurativo potrebbe riguardare l'accuratezza dei calcoli (oracolo) con conseguenze disastrose, mentre per un motore di ricerca come Google la priorità assoluta risiede nella disponibilità del servizio (uptime), dove anche poche ore di downtime globale risultano inaccettabili.

## 1.2 Tassonomia delle Scale di Misura

La scelta della scala utilizzata dipende dalla natura dei dati misurati e determina quali operazioni statistiche siano matematicamente applicabili.

### 1.2.1 Scala Nominale e Ordinale

- **Nominale:** Utilizzata per assegnare valori a etichette o categorie senza un ordine intrinseco o magnitudine (es. tipi di difetti: logica, UI, performance, sicurezza). Operazioni ammesse: Moda, frequenza.
- **Ordinale:** Prevede un ordine di rango per le categorie, ma la distanza o magnitudine tra i valori non è definita né significativa (es. priorità dei difetti: Critical, High, Medium, Low). Operazioni ammesse: Mediana.

### Nota del Prof

Le scale ordinali, pur limitando le analisi statistiche alla mediana, offrono spesso una maggiore oggettività rispetto a scale numeriche forzate. Valutare la severità di un bug su una scala 0-100 introduce un'alta soggettività (es. la differenza tra 65 e 70 è arbitraria), mentre una distinzione qualitativa (Blocker vs Non-Blocker) risulta più robusta, netta e oggettiva.

### 1.2.2 Scale Numeriche (Intervallo e Rapporto)

In queste scale i valori sono assegnati in ordine di rango e la magnitudine della differenza tra i punti ha un significato reale e misurabile.

- **Intervallo:** La magnitudine delle differenze è significativa, ma non possiede uno zero assoluto reale.
- **Rapporto (Ratio):** È il tipo di scala più sofisticato, in cui la magnitudine delle differenze è calcolabile e possiede uno zero assoluto reale (es. LOC, sforzo, tempo).

**Statistiche ammesse per entrambe:** Media, Deviazione Standard, Varianza, Range. Per la scala di Rapporto sono ammessi anche coefficiente di variazione e media geometrica.

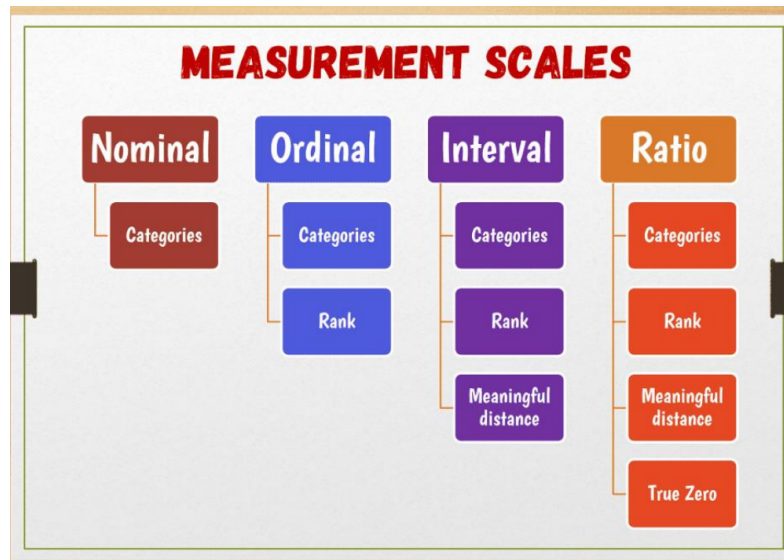


Figura 1.1: Scale di Misura: Nominale, Ordinale, Intervallo e Rapporto.

### 1.3 Modelli di Qualità: Lo Standard ISO/IEC 25010

Lo standard ISO 25010 definisce una tassonomia dettagliata degli attributi di qualità, distinguendo tra qualità del prodotto e qualità in uso. Le caratteristiche principali individuate dallo standard sono:

1. **Functional Suitability (Adeguatezza funzionale):** Completezza, correttezza e adeguatezza funzionale.
2. **Performance Efficiency (Efficienza delle prestazioni):** Comportamento temporale, utilizzo delle risorse e capacità strutturale.
3. **Security (Sicurezza):** Riservatezza, integrità, non ripudio, responsabilità e autenticità.
4. **Maintainability (Manutenibilità):** Modularità, riusabilità, analizzabilità, modificabilità e testabilità.

#### Nota del Prof

È cruciale distinguere sempre il modello di qualità dalle metriche: il modello, come l'ISO 25010, definisce *cosa* significa la qualità fornendone una tassonomia, mentre le metriche definiscono *come* quantificarla operativamente. L'adozione di questi standard fallisce sistematicamente se manca la "buona fede" aziendale; l'uso delle metriche al solo scopo di ottenere certificazioni formali di facciata, senza mirare a un miglioramento sostanziale del prodotto, sviscerisce l'utilità dell'intero sistema di valutazione ingegneristico.

### 1.4 Introduzione alle Metriche Object-Oriented

Le metriche object-oriented vengono impiegate per misurare la qualità dei sistemi software ad oggetti e identificare tempestivamente difetti di progettazione o futuri problemi di

manutenibilità. Vanno sempre interpretate come indicatori di rischio e valutate in modo comparativo, rispetto a una baseline o a un trend, e non in senso assoluto.

#### **Nota del Prof**

In ambito manageriale, è molto più efficace adottare un approccio pratico basato sulle distribuzioni piuttosto che sulle medie globali. Risulta strategicamente più utile concentrare le attività di analisi, refactoring e test mirati esclusivamente sul 10% delle classi che presentano i valori peggiori di complessità (*outliers*), piuttosto che tentare di ottimizzare e riscrivere l'intero sistema in modo uniforme.

# Capitolo 2

## Software Metrics and Their Use

### 2.1 Metriche software e decisioni

Le **metriche software** sono strumenti di supporto alle decisioni ingegneristiche. Non rappresentano una verità assoluta sul sistema e non devono essere interpretate come giudizi automatici sulla qualità.

#### IMPORTANTE PER L'ESAME

Una metrica ha senso solo se è collegata a un obiettivo, a un contesto e a una decisione concreta. Un valore alto o basso non è automaticamente positivo o negativo.

Il processo di misurazione può essere visto come una **measurement-to-decision pipeline**:

1. **Goal / Decision**: quale decisione si vuole supportare?
2. **Operational definition**: cosa si misura, su quale artefatto e con quali regole?
3. **Collection**: raccolta dei dati tramite strumenti e procedure definite.
4. **Analysis**: analisi tramite baseline, trend, distribuzioni o test statistici.
5. **Interpretation rule**: regole per interpretare i valori osservati.
6. **Action**: azione conseguente, come refactoring, redesign o modifica della strategia di test.

#### Nota del Prof

Raccogliere dati senza sapere quale decisione prendere è inutile. Una metrica non ha valore perché produce un numero, ma perché aiuta a decidere cosa fare.

Anche la scala di misura è importante. Una scala ordinale, come *low*, *medium*, *high*, può essere più robusta per descrivere concetti qualitativi, ma limita le operazioni statistiche. Una scala numerica permette calcoli come media e varianza, ma può introdurre soggettività se i numeri non hanno un significato operativo chiaro.

## 2.2 Quality model vs metrics

È fondamentale distinguere tra **modello di qualità** e **metrica**.

- Il **modello di qualità** definisce *cosa* significa qualità in un certo contesto.
- La **metrica** definisce *come* quantificare operativamente uno specifico attributo.

Per esempio, un modello di qualità può stabilire che la manutenibilità è rilevante; una metrica può poi provare a quantificarla tramite complessità, accoppiamento, coesione o code smells.

### Nota del Prof

Applicare metriche e standard senza un reale obiettivo di business rende la misurazione sterile. Se un'organizzazione usa le metriche solo per rispettare formalmente un contratto, senza voler migliorare davvero il prodotto, il processo perde significato ingegneristico.

### IMPORTANTE PER L'ESAME

Non bisogna scegliere una metrica prima di sapere quale decisione deve supportare. Una metrica senza contesto decisionale è solo un numero.

## 2.3 ISO/IEC 25010

Lo standard **ISO/IEC 25010** fornisce una tassonomia degli attributi di qualità del software. Serve a costruire un vocabolario condiviso per descrivere qualità del prodotto e qualità in uso.

### Solo nelle slide (non spiegato a voce)

ISO/IEC 25010 può essere usato per specificare requisiti, pianificare attività di Quality Assurance, definire criteri di accettazione e supportare attività di benchmarking.

Lo standard indica quali dimensioni della qualità considerare, ma non definisce automaticamente quali metriche usare, quali soglie applicare o quali decisioni prendere.

Le caratteristiche principali richiamate sono:

- **Functional suitability**: completezza, correttezza e appropriatezza funzionale;
- **Performance efficiency**: comportamento temporale, uso delle risorse e capacità;
- **Security**: confidenzialità, integrità e non ripudio;
- **Maintainability**: modularità, riusabilità, analizzabilità e testabilità.

### IMPORTANTE PER L'ESAME

ISO/IEC 25010 aiuta a definire cosa osservare nella qualità del software, ma non sostituisce la scelta delle metriche né l'interpretazione ingegneristica dei risultati.

## 2.4 Metriche Object-Oriented

Le **metriche Object-Oriented** servono a individuare possibili problemi di progettazione, manutenibilità, testing e comprensione del codice nei sistemi a oggetti.

### IMPORTANTE PER L'ESAME

Le metriche OO devono essere lette come **indicatori di rischio**. Un valore critico non dimostra automaticamente che una classe sia sbagliata, ma segnala che quella classe merita attenzione.

L'interpretazione deve essere comparativa e contestuale. È utile osservare baseline di progetto, trend nel tempo, distribuzioni, outlier e confronto tra classi simili.

### Nota del Prof

Non ha senso cercare una soglia universale valida per tutte le classi. È più utile osservare il peggior 10% delle classi del progetto e concentrarsi sugli outlier.

## 2.5 CK Metrics

### Solo nelle slide (non spiegato a voce)

La suite CK, proposta da Chidamber e Kemerer, valuta aspetti come complessità, ereditarietà, accoppiamento e coesione nei sistemi object-oriented.

**WMC – Weighted Methods per Class:** misura la complessità complessiva dei metodi di una classe, spesso approssimata tramite il numero di metodi. Un valore alto può indicare troppe responsabilità, maggiore difficoltà di testing e possibile necessità di dividere la classe.

**DIT – Depth of Inheritance Tree:** misura la profondità della classe nella gerarchia di ereditarietà. Un valore alto può indicare riuso, ma anche maggiore difficoltà di comprensione e rischio di interazioni inattese tra classi base e derivate.

**NOC – Number of Children:** misura il numero di sottoclassi immediate. Un valore alto indica che la classe base è molto riutilizzata, ma anche che una sua modifica può generare regressioni in molte sottoclassi.

**CBO – Coupling Between Object Classes:** misura quante classi distinte sono accoppiate a una classe. Un valore alto indica rischio di bassa modularità, *ripple effects* e difficoltà di testing isolato.

**RFC – Response For a Class:** misura il numero di metodi che possono essere eseguiti in risposta a un messaggio ricevuto dalla classe. Un valore alto indica comportamento complesso e ampia superficie di test.

**LCOM – Lack of Cohesion in Methods:** misura la mancanza di coesione tra i metodi. Un valore alto può indicare che la classe contiene responsabilità diverse fuse insieme, suggerendo un possibile refactoring.

### IMPORTANTE PER L'ESAME

Le CK metrics non dicono automaticamente che una classe è errata. Servono a individuare classi potenzialmente rischiose da analizzare con attenzione.

## 2.6 Altre metriche

La **Cyclomatic Complexity** misura la complessità del flusso di controllo di un metodo o programma. Cresce in presenza di strutture decisionali come **if**, **while**, **for** e **switch**.

### Solo nelle slide (non spiegato a voce)

Nel testing, la Cyclomatic Complexity può essere usata per stimare il numero minimo di casi necessari a coprire i percorsi di base.

Un valore alto indica spesso codice più difficile da comprendere, mantenere e testare. Tuttavia, non bisogna interpretare il valore in modo meccanico: alcuni problemi sono intrinsecamente complessi.

Il **Fan-in** misura quante classi o componenti dipendono da una certa classe. Se è alto, la classe è critica perché molto usata. Il **Fan-out** misura invece da quante classi o componenti dipende una classe; se è alto, può indicare forte dipendenza dal resto del sistema.

### Nota del Prof

Non bisogna rifattorizzare tutto automaticamente. Una classe può essere complessa o molto connessa perché il problema che risolve è intrinsecamente complesso. Separare artificialmente responsabilità non divisibili può peggiorare la manutenibilità.

Le metriche possono essere osservate a diversi livelli:

- **method-level:** Cyclomatic Complexity, nesting depth;
- **class-level:** LOC, WMC, CBO, RFC, LCOM;
- **package/module-level:** accoppiamento afferente, accoppiamento efferente, instability;
- **process-level:** code churn, frequenza di modifica, ownership concentration.

Le metriche di processo sono spesso molto predittive perché descrivono come il codice evolve nel tempo, non solo come appare in uno snapshot.

## 2.7 Function Points

I **Function Points** sono una metrica usata per stimare la grandezza funzionale di un sistema software. Sono indipendenti dal linguaggio di programmazione, dalla tecnologia, dal framework e dall'implementazione concreta.

### IMPORTANTE PER L'ESAME

I Function Points sono un argomento frequente nelle domande d'esame.

L'idea centrale è misurare il software dal punto di vista dell'utente o del business. Per questo motivo sono utili nelle fasi iniziali del progetto, quando il codice non esiste ancora ma sono già disponibili requisiti o descrizioni funzionali.

Un concetto fondamentale è l'**application boundary**, cioè il confine tra ciò che appartiene all'applicazione e ciò che è esterno.

### Solo nelle slide (non spiegato a voce)

L'application boundary stabilisce cosa è interno al sistema e cosa deve essere considerato esterno.

Il calcolo dei Function Points si basa su cinque categorie:

- **ILF (Internal Logical File)**: dati mantenuti internamente dall'applicazione;
- **EIF (External Interface File)**: dati usati dall'applicazione ma mantenuti esternamente;
- **EI (External Input)**: transazioni in ingresso che modificano dati interni;
- **EO (External Output)**: transazioni in uscita che producono informazioni derivate;
- **EQ (External Inquiry)**: interrogazioni che leggono dati senza modificarli.

I Function Points possono essere usati per stimare dimensione funzionale, effort, costi, schedule e produttività attesa. Per ottenere stime realistiche, è necessario calibrare i valori usando dati storici dell'organizzazione.

### 2.7.1 Vantaggi e svantaggi dei Function Points

I **Function Points** hanno diversi vantaggi. Il principale è che permettono di stimare la dimensione funzionale del sistema prima che il codice venga scritto. Questo li rende utili nelle fasi iniziali del progetto, quando sono disponibili requisiti e descrizioni funzionali, ma non ancora l'implementazione.

Un altro vantaggio è la loro indipendenza dalla tecnologia. Poiché non dipendono dal linguaggio di programmazione, dal framework o dall'architettura scelta, consentono di confrontare sistemi sviluppati con tecnologie diverse. Inoltre, essendo basati sul punto di vista dell'utente o del business, aiutano a collegare la stima tecnica alle funzionalità realmente percepite dagli stakeholder.

### Solo nelle slide (non spiegato a voce)

I Function Points sono particolarmente utili per stime preliminari di size, effort, costi e schedule, soprattutto quando vengono calibrati con dati storici di produttività dell'organizzazione.

Tuttavia, i Function Points presentano anche alcuni limiti. Il calcolo richiede interpretazione e può quindi introdurre soggettività, soprattutto nella definizione dell'application boundary e nella classificazione di ILF, EIF, EI, EO ed EQ. Due analisti diversi potrebbero produrre stime differenti se non condividono criteri operativi chiari.

Inoltre, i Function Points misurano la dimensione funzionale, ma non catturano direttamente aspetti non funzionali come performance, sicurezza, affidabilità, usabilità o complessità tecnica. Per questo motivo, due sistemi con gli stessi Function Points possono richiedere effort molto diversi se uno dei due ha requisiti extra-funzionali più stringenti.

### IMPORTANTE PER L'ESAME

Il vantaggio dei Function Points è che permettono una stima precoce e indipendente dalla tecnologia. Il limite principale è che non misurano direttamente qualità, complessità tecnica o requisiti non funzionali.

### Nota del Prof

I Function Points ignorano molti requisiti non funzionali. Due sistemi con le stesse funzionalità possono avere lo stesso numero di Function Points, anche se uno deve essere molto più sicuro, performante o robusto.

### IMPORTANTE PER L'ESAME

I Function Points non misurano direttamente la qualità. Misurano la dimensione funzionale del sistema dal punto di vista utente/business.

## 2.8 GQM+Strategies

**GQM+Strategies** estende l'approccio Goal-Question-Metric collegando obiettivi di business, obiettivi software, domande, misure, strategie e target.

Business Goals → Software Goals → Questions → Measures → Strategies / Targets

L'idea è che ogni metrica debba rispondere a una domanda, ogni domanda debba essere collegata a un obiettivo e ogni obiettivo debba avere senso rispetto alla strategia dell'organizzazione.

### Nota del Prof

Tutto nasce dalla modellazione del business. Se non si definisce come l'organizzazione ottiene successo, qualsiasi metrica o attività di refactoring può diventare inutile o persino dannosa.

Per esempio, l'obiettivo generico "migliorare la qualità" può essere trasformato in una domanda come "la copertura dei test è sufficiente?", associato a una misura come la code coverage e collegato a una strategia come aumentare la copertura introducendo nuovi unit test.

**Solo nelle slide (non spiegato a voce)**

Il reference process di GQM+Strategies include le fasi Initialize, Characterize, Set Goals, Choose Process, Execute Model, Analyze Results, Package and Improve.

**Initialize:** avvio dell'attività di misurazione.

**Characterize:** analisi del contesto organizzativo e progettuale.

**Set Goals:** definizione degli obiettivi.

**Choose Process:** scelta del processo di misurazione.

**Execute Model:** esecuzione del modello.

**Analyze Results:** analisi dei risultati.

**Package and Improve:** consolidamento della conoscenza e miglioramento futuro.

**IMPORTANTE PER L'ESAME**

GQM+Strategies serve ad allineare le metriche alle decisioni organizzative. Una metrica è utile solo se risponde a una domanda collegata a un obiettivo.

## 2.9 Collegamento con il progetto

Nel progetto, le metriche vengono usate per analizzare il software, costruire dataset, stimare la difettosità e valutare il refactoring. Non basta calcolare valori numerici: bisogna interpretarli e collegarli a decisioni concrete.

**IMPORTANTE PER L'ESAME**

Nel progetto le metriche devono supportare decisioni come individuare classi rischiose, valutare bugginess, scegliere dove intervenire, analizzare l'effetto del refactoring e discutere i risultati.

Le metriche possono essere estratte a livello di classe, per esempio tramite SonarCloud, e possono contribuire alla costruzione del dataset di defect prediction. Possono inoltre aiutare a identificare classi più rischiose, confrontare versioni prima e dopo un refactoring e discutere il rapporto tra maintainability, testing e code smells.

Nel caso del refactoring automatico tramite LLM, l'analisi deve verificare se la nuova versione della classe:

- compila correttamente;
- preserva la funzionalità originale;

- supera i test;
- riduce i code smells;
- non peggiora metriche correlate positivamente con la bugginess.

### **Nota del Prof**

Il valore del progetto non dipende dal fatto che l'LLM produca sempre un refactoring perfetto, ma dalla capacità di interpretare i risultati e prendere decisioni motivate sulla base dei dati raccolti.

Gli errori comuni da evitare sono:

- usare medie globali senza analizzare distribuzioni e outlier;
- applicare soglie arbitrarie come se fossero verità assolute;
- presentare metriche senza spiegare quale decisione supportano;
- ignorare la complessità intrinseca del problema;
- interpretare ogni valore alto come necessariamente negativo;
- non spiegare il razionale dietro le scelte di misurazione.

### **IMPORTANTE PER L'ESAME**

Una buona discussione progettuale non si limita a mostrare numeri. Deve spiegare cosa significano quei numeri, quali decisioni supportano e quali limiti hanno.

# Capitolo 3

## GQM+S, Stima dei Costi e Planning Poker

### 3.1 GQM+Strategies (Goal-Question-Metric + Strategies)

L'approccio GQM+Strategies (GQM+S) è una metodologia che combina il metodo classico GQM con l'identificazione delle migliori strategie per raggiungere gli obiettivi. Questo approccio collega a cascata gli obiettivi aziendali alla misurazione:

Business Goals → Software Goals → Questions → Measures → Strategies / Targets.

#### Nota del Prof

Il CQM originale è stato co-creato dal Professor Cantone. L'estensione "+ Strategies" serve ad associare l'azione tecnica al risultato voluto. Tutto parte dal Business: ogni feature o refactoring viene fatto in primis per raggiungere obiettivi aziendali.

#### 3.1.1 Il Reference Process (I 6 Step del GQM+S)

Il processo si sviluppa in sei step principali per permettere il miglioramento continuo:

0. **Initialize:** Impegno (Commitment), Staffing e Responsabilità.
1. **Characterize:** Definire lo scope dell'applicazione e caratterizzare il contesto ambientale.
2. **Set Goals:** Analisi dello status quo, prioritizzazione degli obiettivi e costruzione della griglia goals-strategies.
3. **Choose Process:** Pianificare l'implementazione delle strategie e organizzare la raccolta/analisi dei dati.
4. **Execute Model:** Esecuzione delle strategie, raccolta dati e feedback.
5. **Analyze Results:** Analisi dei dati, revisione delle strategie e analisi costi/benefici.
6. **Package and Improve:** Adattamento dei goal e registrazione delle "lessons learned".

### Nota del Prof

L'ultimo step di "packaging" si basa sul concetto di *Experience Factory*: lo scopo è impacchettare l'esperienza in una knowledge base per riutilizzarla in futuro.

### Solo nelle slide (non spiegato a voce)

#### Esempi di utilizzo tipici:

- **Miglioramento del processo:** Ridurre lead time o rework (Metriche: cycle time, difetti sfuggiti).
- **Project management:** Migliorare prevedibilità (Metriche: milestone burndown, throughput).
- **Supporto decisionale:** Valutare l'adozione di nuove tecnologie (Metriche: esiti progetto pilota).

## 3.2 Introduzione alla Stima dei Costi

La stima si basa su tre domande fondamentali: quanto sforzo (*effort*) è richiesto, quanto tempo di calendario è necessario e qual è il costo totale dell'attività.

### Nota del Prof

Bisogna distinguere l'effort (lavoro puramente impiegato) dal tempo di calendario: un'attività può richiedere poco sforzo ma molto tempo se le risorse non sono subito disponibili.

### 3.2.1 Componenti del Costo ed Overheads

Il costo include Hardware/Software, viaggi, formazione e l'Effort, che è il fattore dominante (salari e costi sociali). Gli **Overheads** includono:

- Costi degli edifici (riscaldamento, luce).
- Costi di rete e comunicazione.
- Strutture condivise come mensa o biblioteca.

### Nota del Prof

Il salario dello sviluppatore a volte non copre neanche il 50% del costo totale per un'azienda a causa degli overhead. Inoltre, il software ha un costo marginale di replicazione quasi nullo: vendere a un secondo cliente non costa nulla di più in termini di produzione.

### 3.2.2 Il Ruolo del Linguaggio di Programmazione

Solo nelle slide (non spiegato a voce)

Il linguaggio impatta drasticamente su tempi e costi. Ad esempio, l'Assembly richiede molto più tempo per coding (8 settimane per 5000 linee) e testing (10 settimane) rispetto a un linguaggio ad alto livello per lo stesso sistema.

### 3.3 Tecniche di Stima

#### IMPORTANTE PER L'ESAME

All'esame non vengono chieste le tecniche a memoria, ma i vantaggi, gli svantaggi e quando usare l'una rispetto all'altra.

- **Algorithmic cost modelling:** Approccio basato su formule e dati storici, generalmente basato sulla dimensione del software (es. COCOMO).
- **Expert judgement:** Uno o più esperti usano la loro esperienza per predire i costi.
  - *Vantaggi:* Metodo relativamente economico; accurato se gli esperti hanno esperienza diretta in sistemi simili.
  - *Svantaggi:* Molto inaccurato se mancano veri esperti nel dominio.
- **Estimation by analogy:** Il costo è calcolato comparando il progetto con uno simile nello stesso dominio.
  - *Vantaggi:* Accurato se sono disponibili dati di progetti passati.
  - *Svantaggi:* Impossibile se non si è mai affrontato un progetto comparabile; richiede un database di costi sistematicamente aggiornato.
- **Parkinson's Law:** Il progetto costa qualunque risorsa sia disponibile. Il lavoro si espande per riempire il tempo disponibile.
  - *Vantaggi:* Non si va mai oltre il budget (*no overspend*).
  - *Svantaggi:* Il sistema rimane solitamente incompleto (*unfinished*).
- **Pricing to win:** Il costo è pari a quanto il cliente può spendere.
  - *Vantaggi:* Permette di vincere il contratto.
  - *Svantaggi:* Bassa probabilità che il cliente ottenga ciò che vuole; i costi non riflettono il lavoro reale richiesto.

### 3.4 Planning Poker

Il Planning Poker è una tecnica Agile per la stima del consenso, basata sulle carte di James Grenning.

### 3.4.1 Dinamiche e Regole del Gioco

Ogni partecipante riceve un mazzo di carte con una sequenza numerica, solitamente di **Fibonacci** (1, 2, 3, 5, 8, 13, 21, ecc.).

- **La Sfida:** Il moderatore presenta una *user story* e il team può fare domande al Product Owner.
- **Voto Segreto:** Ogni membro seleziona privatamente una carta che rappresenta la "grandezza" (*size*) del task, considerando tempo, rischio e complessità.
- **Svelamento e Dibattito:** Le carte vengono mostrate insieme. Se non c'è consenso, chi ha dato i valori estremi spiega le proprie ragioni. Il gruppo discute e si vota di nuovo finché non si raggiunge un accordo.

#### IMPORTANTE PER L'ESAME

**Perché i "buchi" di Fibonacci?** Servono a forzare la granularità corretta. All'aumentare della grandezza aumenta l'incertezza: la sequenza ci spinge a scegliere tra valori distanti (es. 13 o 21), riflettendo la reale imprecisione della stima per task grandi.

#### Nota del Prof

In caso di mancato consenso, si sceglie la stima superiore per tutelarsi dalla naturale indole ottimista dell'uomo, che tende a sottostimare i tempi ignorando gli imprevisti.

### 3.4.2 Vantaggi e Consigli Pratici (Tips)

Il Planning Poker favorisce la collaborazione e fa emergere i problemi tecnici in anticipo attraverso il dibattito.

- **Chi vota:** Solo chi farà materialmente il lavoro. I manager non votano per non influenzare le stime al ribasso.
- **Pareggi:** Se il team è diviso tra due taglie consecutive (es. 5 e 8), si sceglie la maggiore e si procede.

#### Solo nelle slide (non spiegato a voce)

##### Suggerimenti per l'efficienza:

- Usare un timer per limitare le discussioni a circa un minuto.
- Includere una carta "I need a break" per segnalare la stanchezza.
- Mantenere una **Baseline** consistente tra le varie iterazioni.

# Capitolo 4

## Introduzione all'Architettura Software

### 4.1 Cos'è un'Architettura Software

#### IMPORTANTE PER L'ESAME

L'architettura software è uno dei pilastri dell'ingegneria del software e rappresenta una delle domande più frequenti in sede d'esame.

Secondo lo standard **IEEE 1471**, l'architettura è definita come:

“L'organizzazione fondamentale di un sistema, incarnata nei suoi componenti, nelle loro relazioni reciproche e con l'ambiente, e i principi che ne governano la progettazione e l'evoluzione.”

L'intento principale, come sostenuto da Kruchten, è fornire un **controllo intellettuale** su sistemi di enorme complessità, permettendo agli stakeholder di comprendere la struttura senza annegare nei dettagli implementativi.

#### 4.1.1 Architetture Intended vs Unintended

Ogni sistema possiede intrinsecamente un'architettura, che può essere classificata in due modi:

- **Intended (Pianificata):** Esplicita e documentata prima della scrittura del codice. Segue l'approccio del *Forward Engineering*.
- **Unintended (Accidentale):** Implicita e spontanea. Emerge quando si scrive codice senza un piano; in questo caso, l'architettura deve essere ricostruita a posteriori (*Reverse Engineering*).

#### Nota del Prof

Immaginate di costruire una casa: nell'approccio *intended* disegnate prima la piantina. Nell'approccio *unintended* costruite le stanze a casaccio e solo alla fine vi accorgete di che forma ha la casa. Ricordate però il Manifesto Agile: l'architettura ha valore solo se serve a produrre codice migliore, non deve essere burocrazia inutile.

## 4.2 Il Modello 4+1 di Kruchten

Per gestire la complessità, Kruchten propone di decomporre l'architettura in diverse **viste** basate sul principio della *separation of concerns*.

- **Logical View:** Supporta i requisiti funzionali, ovvero ciò che il sistema deve fare per l'utente finale.
- **Development View:** Si concentra sull'organizzazione dei moduli e dei package nel codice sorgente.

### Nota del Prof

Un esempio tipico è lo **stack di Android**, che mostra come il sistema sia stratificato dal Kernel Linux fino alle applicazioni.

- **Dynamic View:** Analizza il comportamento a runtime, i processi e le interazioni tra oggetti. Viene solitamente modellata con i *Sequence Diagram*.

Solo nelle slide (non spiegato a voce)

### Completamento del modello:

- **Deployment View:** Si occupa della mappatura fisica del software sui nodi hardware e della distribuzione geografica dei server.
- **Scenarios (il “+1”):** Casi d'uso o sequenze di azioni che fanno da collante tra le altre quattro viste, verificandone la coerenza.

### Nota del Prof

Pensate a una casa: potete guardare la pianta elettrica, quella idraulica o quella strutturale. Sono tutte viste diverse dello stesso edificio, necessarie a stakeholder diversi (elettricista, idraulico, ingegnere).

## 4.3 Progettazione e Decision-Making

Il design architeturale si basa su tre fattori: **Riuso** (di pattern o componenti), **Metodo** (tecniche sistematiche) e **Intuizione** (l'esperienza dell'architetto).

### 4.3.1 L'importanza dei Requisiti Non Funzionali

#### Nota del Prof

L'architettura è dettata principalmente dai **Requisiti Non Funzionali** (attributi di qualità come sicurezza, velocità, scalabilità). Questi requisiti sono strutturali: non si può “aggiungere la sicurezza” alla fine del progetto. È come una casa: se volete che sia antisismica, deve nascere così. Non potete renderla antisismica una volta costruita senza abbatterla.

### 4.3.2 Trade-offs e il ruolo dell'Architetto

L'architetto deve bilanciare forze contrastanti provenienti da stakeholder diversi. Le decisioni sono spesso sub-ottimali e prese “al buio” (quando i requisiti sono ancora vaghi).

Solo nelle slide (non spiegato a voce)

Questo dilemma è rappresentato graficamente dalla scelta tra carne “Cotta” o “Cruda”: ogni scelta comporta un vantaggio per uno stakeholder e uno svantaggio per un altro.

## 4.4 Documentazione: Viste e Punti di Vista

### IMPORTANTE PER L'ESAME

Questa è una distinzione fondamentale. Molti studenti confondono i due termini: ricordateli bene per l'esame.

- **Architectural View (Vista):** È la rappresentazione del sistema da una certa prospettiva. È composta da uno o più *modelli* (o diagrammi).
- **Architectural Viewpoint (Punto di Vista):** È l'insieme di regole, convenzioni e linguaggi usati per costruire la vista.

### Nota del Prof

Esempio dello stadio: il *Viewpoint* è il concetto teorico di “guardare dalla tribuna stampa” (regole e angolazione). La *View* è la foto specifica che scattate in quello stadio particolare applicando quel punto di vista. In questo corso, Modello e Diagramma sono considerati sinonimi.

## 4.5 Software Product Lines (SPL)

Una SPL è una famiglia di prodotti che condividono un set comune di funzionalità (**Core Assets**). Si basa sul *Systematic Reuse*.

- **Scoping:** Decidere cosa includere nella piattaforma comune. Se lo scope è troppo largo si spreca soldi; se è troppo stretto si perde l'opportunità di riuso.
- **Vantaggi:** Riduzione drastica dei tempi e dei costi di sviluppo per i prodotti successivi al primo.

### Nota del Prof

Pensate alle auto: la Mercedes usa lo stesso tasto per il finestrino su tutti i modelli. La BMW usa lo stesso motore fisico per vari modelli, limitandone la potenza via software. Nel software il riuso è vitale, ma genera conflitti di budget: il Team A spesso non vuole pagare il costo extra per rendere un modulo riutilizzabile anche dal Team B.

## 4.6 Stili Architetture

Lo **Stile Architetture** è un pattern astratto; l'**Architettura** è la sua applicazione pratica.

1. **Client/Server:** Il server gestisce dati e sicurezza, i client richiedono servizi. Dissaccoppiamento parziale (il server non conosce i client).
2. **Peer-to-Peer:** Ogni nodo è sia client che server.

### Nota del Prof

Come BitTorrent: chi scarica deve anche caricare, evitando colli di bottiglia.

3. **Pipes and Filters:** Stream di dati trasformati in sequenza.

### Nota del Prof

Usato in sistemi critici come l'avionica. Performance altissime, riuso quasi nullo.

4. **Shared Data:** Comunicazione tramite un database centrale.

### Solo nelle slide (non spiegato a voce)

Si divide in **Repository** (l'agente chiede i dati) e **Blackboard** (il sistema notifica gli agenti quando i dati cambiano).

# Capitolo 5

## Debito Tecnico e Code Smells

### 5.1 Il Debito Tecnico (Technical Debt)

Il Debito Tecnico (TD) rappresenta l'applicazione di concetti finanziari al dominio dell'ingegneria del software, aggiungendo criteri di tempo ed economia al classico concetto di manutenibilità.

#### IMPORTANTE PER L'ESAME

Il debito tecnico si scompone in due concetti finanziari fondamentali che lo distinguono dalla semplice manutenibilità:

- **Capitale (Principal):** Il costo (effort) necessario per eliminare il debito, ovvero il tempo speso per il refactoring.
- **Interessi (Interest):** La penalità futura pagata se il debito non viene eliminato, che si traduce in rallentamento dello sviluppo o maggiore probabilità di bug.

#### 5.1.1 Cause e Conseguenze

Il debito emerge tipicamente per ottimizzazioni a breve termine che creano handicap a lungo termine. Le cause includono la **crescita organica** (aumento naturale della complessità) e **scelte opportunistiche** (trade-off per rispettare le scadenze).

#### Nota del Prof

Spesso si creano trade-off tra qualità interna e "Time to Market". Seguendo il principio Agile "Working code over comprehensive documentation", gli sviluppatori possono trascurare la modularizzazione o i commenti pur di consegnare software funzionante e farsi pagare .

#### IMPORTANTE PER L'ESAME

Il debito tecnico impatta sempre la release successiva: se il codice non è manutenibile oggi, lo sviluppo futuro sarà più lento e pronò ai difetti.

Solo nelle slide (non spiegato a voce)

Gli analizzatori statici (come SonarQube) faticano a calcolare gli "Interessi" (il costo futuro), rendendoli attualmente una metrica poco chiara.

### 5.1.2 Strategie di Gestione

#### Nota del Prof

Il debito tecnico "scade": se un modulo di scarso valore non verrà mai più modificato, non produrrà interessi. È un errore fare refactoring su codice che non verrà più toccato.

Le strategie di gestione includono:

- **Monitoraggio e Refactoring:** Uso di strumenti come SonarCloud per tracciare il trend delle violazioni.
- **Quality Gate:** Una regola che blocca automaticamente i commit contenenti nuove violazioni.

#### Nota del Prof

Metafora della barca bucata: la prima cosa da fare non è svuotare l'acqua (vecchio debito), ma tappare il buco (usare il quality gate per non farne entrare di nuovo).

## 5.2 Introduzione ai Code Smells

Occorre distinguere tra tre concetti:

- **Quality Rule:** Principio ingegneristico validato (es. commenti, bassa complessità).
- **Violation o Code Smell:** Porzione di codice non conforme a una regola di qualità.
- **Defect (Bug):** Problema che impatta la qualità esterna (es. crash).

#### IMPORTANTE PER L'ESAME

I Code Smells **non sono bug**. Non causano malfunzionamenti immediati ma rendono la manutenzione difficile. Agiscono come un "campo minato", aumentando la probabilità che uno sviluppatore introduca difetti reali.

## 5.3 Catalogo dei "Worst Smells"

I "Worst Smells" sono violazioni che non portano alcun vantaggio in nessuna circostanza, a differenza di alcuni smell "classici" (come God Class o Code Clones) che possono nascere da trade-off temporali.

### 5.3.1 Esempi di Worst Smells

- **Modificatori non ordinati:** Crea confusione visiva inutile.
- **Object.finalize() sovrascritto:** In Java, forzare la distruzione manuale è spesso deleterio.
- **Codice commentato:** Sporca la codebase; spesso sono rimasugli di prototipi.
- **Mancanza del "break" in uno Switch:** Causa comportamenti imprevisti proseguendo nei case successivi.
- **Concatenazione di Stringhe con '+' in un ciclo:** Degrada gravemente le performance; il refactoring corretto prevede l'uso di **StringBuilder**.
- **Pessimo Naming:** Esempi includono attributi chiamati A, B o C.

#### Nota del Prof

Chiamare un metodo `ThisIsAGreatMethod1` non fa risparmiare tempo significativo: non siamo in Formula 1 dove si risparmiano millisecondi.

#### Solo nelle slide (non spiegato a voce)

L'analisi su 27 progetti Apache ha rivelato che i Worst Smells non sono meno frequenti né più severi degli altri, ma non avendo ragioni di esistere, vanno bloccati sempre tramite Quality Gates.

# Capitolo 6

## Machine Learning e Software Analytics per la Software Engineering

### 6.1 Definizione di Machine Learning, ruolo di statistica e informatica

Il **Machine Learning (ML)** è lo studio degli algoritmi che migliorano le proprie performance in un determinato **task** attraverso l'**esperienza**. L'obiettivo è ottimizzare un criterio di performance utilizzando dati di esempio o esperienze passate.

Questa disciplina unisce due campi fondamentali:

- **Statistica**: necessaria per trarre inferenze a partire da un campione di dati.
- **Informatica**: necessaria per fornire algoritmi efficienti in grado di risolvere il problema di ottimizzazione, oltre a rappresentare e valutare il modello per l'inferenza.

#### Nota del Prof

È fondamentale comprendere che ogni misurazione e ogni modello di ML, sebbene si basi su dati raccolti nel passato, ha come unico scopo quello di **prendere decisioni sul futuro**. L'intero sistema si fonda su un assioma filosofico implicito ma cruciale: assumiamo che il futuro sia simile al passato. Senza questa assunzione di base, l'uso dei dati storici per addestrare un modello sarebbe completamente inutile.

### 6.2 Esempi d'uso: associazioni, classificazione, regressione

Il Machine Learning viene applicato a diverse tipologie di problemi.

#### 6.2.1 Apprendimento di associazioni

L'**apprendimento di associazioni** calcola la probabilità che un evento si verifichi in concomitanza di un altro. Un esempio tipico è la **basket analysis**, in cui si stima la probabilità che chi compra un certo prodotto acquisti anche un altro prodotto.

### 6.2.2 Classificazione

La **classificazione** serve a dividere i dati in categorie. Tra gli esempi più importanti troviamo:

- **Credit scoring**: differenziare clienti a basso o alto rischio in base a reddito e risparmi.
- **Face recognition**.
- **Character recognition**.
- **Speech recognition**.
- **Diagnosi medica**.
- **Predizione del click** su pubblicità web.

### 6.2.3 Regressione

La **regressione** viene utilizzata quando l'output non è una categoria, ma un valore numerico continuo. Un esempio classico è la predizione del **prezzo di un'auto usata** in base ai suoi attributi.

#### Nota del Prof

Sistemi come Google o Netflix basano il loro intero modello di business sulla predizione. Google non vende solo spazi pubblicitari, ma predice quale utente cliccherà su quale banner per massimizzare i profitti. Netflix usa algoritmi di classificazione e raccomandazione per permettere all'utente di trovare rapidamente il contenuto desiderato. Inoltre, i sistemi di autenticazione biometrica non cercano una corrispondenza pixel per pixel, ma utilizzano il ML per capire quali variazioni sono significative e quali no.

Le principali applicazioni del ML si articolano quindi in tre grandi famiglie: **associazioni**, **classificazione** e **regressione**. Esse coprono problemi di co-occorrenza, categorizzazione e stima di valori numerici.

## 6.3 Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement

I modelli di ML possono apprendere in modi diversi a seconda dei dati disponibili.

### 6.3.1 Supervised learning

Nel **supervised learning**, il modello riceve sia i dati di addestramento sia gli output desiderati, cioè le **label**.

### Nota del Prof

Un esempio è fornire al modello la dimensione di un tumore dicendogli esplicitamente se in passato si è rivelato benigno o maligno, affinché impari a classificarlo in futuro.

## 6.3.2 Unsupervised learning

Nell'**unsupervised learning**, vengono forniti dati di addestramento senza output desiderati. Lo scopo è trovare struttura, similarità o gruppi latenti nei dati.

### Nota del Prof

In questo caso l'obiettivo non è predire un valore esatto, ma organizzare i dati in cluster simili tra loro, ad esempio per raggruppare geni simili o fare network analysis.

## 6.3.3 Semi-supervised learning

### Solo nelle slide (non spiegato a voce)

Il **semi-supervised learning** utilizza dati di addestramento in cui solo una parte limitata delle istanze possiede le label corrette.

## 6.3.4 Reinforcement learning

### Solo nelle slide (non spiegato a voce)

Il **reinforcement learning** è una tipologia di apprendimento basata su un sistema di ricompense e punizioni. Nelle slide è citato, ma non viene approfondito a voce.

Nel complesso, la distinzione tra questi paradigmi dipende dal tipo di supervisione disponibile e dal modo in cui il sistema riceve informazione utile per apprendere.

## 6.4 Etica del data mining e del ML

L'utilizzo dei dati solleva questioni etiche rilevanti. L'anonimizzazione dei dati è difficile, e l'uso di certe informazioni può condurre a discriminazioni. Per esempio, utilizzare razza, sesso o religione per l'approvazione di un prestito è eticamente problematico, mentre in ambito medico gli stessi dati possono essere legittimi.

Le domande fondamentali sono:

- Chi ha accesso ai dati?
- Per quale scopo sono stati raccolti?
- Quali conclusioni possono essere tratte legittimamente?

### Nota del Prof

Quando un sistema ML prende decisioni, la responsabilità è un tema complesso. Se una Tesla a guida autonoma causa un incidente, oggi la responsabilità legale ricade ancora su chi è al volante, anche in virtù degli accordi di licenza accettati dagli utenti. Inoltre, il ML tende a replicare i bias umani presenti nei dati storici: se nel passato una certa etnia o un certo genere ha subito discriminazioni, il modello può apprendere e riprodurre quel comportamento in futuro.

L'etica del ML impone quindi attenzione a **privacy**, **bias**, **discriminazione** e **responsabilità**. I dati storici non sono neutrali, e i modelli possono ereditare le distorsioni del contesto da cui provengono.

## 6.5 Terminologia: istanze, feature, attributi nominali/-numerici, output, modelli

Per utilizzare correttamente gli algoritmi di ML è necessario padroneggiare una terminologia standard.

### 6.5.1 Instances

Le **instances** sono i singoli esempi indipendenti del concetto da apprendere.

### Nota del Prof

In un dataset strutturato a tabella, l'istanza rappresenta la singola riga, ad esempio la combinazione di una specifica classe software in una specifica release.

### 6.5.2 Features o attributi

Le **feature** sono gli aspetti misurabili di un'istanza. Possono essere:

- **Nominali**: valori simbolici o categorici.
- **Numerici**: valori quantitativi su cui si possono eseguire operazioni matematiche.

### Nota del Prof

Le feature rappresentano le colonne della tabella. In genere, l'ultima colonna è l'output o variabile da predire, mentre le altre sono metriche che aiutano nella predizione.

### 6.5.3 Output e modelli

La conoscenza prodotta da un sistema di ML può essere rappresentata tramite:

- **tabelle decisionali**;
- **modelli lineari**;

- alberi decisionali;
- regole;
- cluster.

### IMPORTANTE PER L'ESAME

Molti ambienti di ML, in particolare **Weka**, non supportano correttamente gli attributi con scala ordinale, ad esempio *Low*, *Medium*, *High*. Weka tende a trattarli come semplici categorie nominali, perdendo l'informazione dell'ordine. Per questo motivo, spesso occorre eliminare la colonna oppure trasformarla in una variabile binaria, accettando il relativo trade-off nella perdita di informazione.

Un dataset è dunque una tabella in cui le **righe** rappresentano le istanze e le **colonne** rappresentano le feature. La corretta interpretazione del tipo di attributo è essenziale, soprattutto quando si usano strumenti come Weka.

## 6.6 Software quality e costo dei bug

La **qualità del software** ha un impatto economico enorme. I bug costano a livello globale migliaia di miliardi di dollari, e le attività di **Quality Assurance (QA)** risultano estremamente costose.

Nei sistemi industriali di grandi dimensioni:

- vengono effettuate migliaia di **code review** al giorno;
- vengono eseguiti milioni di **test**;
- ogni modifica deve essere controllata con attenzione.

### Nota del Prof

Poiché il testing è molto oneroso, diventa fondamentale stimare in anticipo quali entità del codice, come classi o metodi, abbiano maggiore probabilità di essere difettose, così da concentrare lì l'effort di QA.

Ne segue che la defect prediction nasce anche come risposta economica e organizzativa al problema della qualità: non è sostenibile controllare tutto con la stessa intensità, quindi occorre **prioritizzare**.

## 6.7 Software analytics: definizione, obiettivi e fonti dati

La **Software Analytics** è la combinazione di **Software Data** e **Data Analytics**. Il software produce una grande quantità di dati distribuiti tra repository, sistemi di build, log di CI/CD, issue tracker, code review e file di configurazione.

Gli obiettivi principali della Software Analytics sono:

1. **Quality improvement**: prevenire crash e bug.

2. **Process improvement:** capire l'impatto di pratiche come code review e rapid release.
3. **Productivity improvement:** valutare l'effetto della CI e di altre pratiche sul team.
4. **Empirical theory building:** costruire teorie empiriche sull'ingegneria del software.

La Software Analytics sfrutta quindi i dati prodotti dagli strumenti di sviluppo per migliorare **qualità**, **processi**, **produttività** e comprensione empirica del software.

## 6.8 AI/ML applicati alla Software Engineering

L'Intelligenza Artificiale sta trasformando l'Ingegneria del Software.

### 6.8.1 Applicazioni alla qualità

Tra le applicazioni principali ci sono:

- **defect prediction;**
- **vulnerability prediction;**
- **malware prediction;**
- **test case generation.**

### 6.8.2 Applicazioni alla produttività

L'AI può supportare anche la produttività del team, ad esempio tramite:

- generazione di **UI**, requisiti, codice e commenti;
- previsione della **produttività** di sviluppatori o team;
- **recommendation** di developer o reviewer;
- identificazione del **turnover** degli sviluppatori.

#### Solo nelle slide (non spiegato a voce)

Nelle slide vengono citati esempi industriali concreti, come un **AI coding assistant** sviluppato in collaborazione con Mozilla, i tool **SapFix** e **Sapienz** di Facebook per trovare e correggere bug automaticamente, e **Sketch2Code** di Microsoft per trasformare schizzi su lavagna in codice HTML.

Nel contesto della Software Engineering, l'AI non serve quindi solo a prevedere bug, ma anche a supportare gli sviluppatori nella produzione di codice, nel testing e nella gestione del lavoro di squadra.

## 6.9 Defect prediction e ML for software defects

Nel contesto dei difetti software, il ML ha tre grandi obiettivi:

1. **Predire** i difetti futuri, per ottimizzare le risorse di QA.
2. **Spiegare** perché un software fallisce.
3. **Costruire** teorie empiriche sulla defectiveness.

### Nota del Prof

Prevenire è meglio che curare. Eliminare code smells e individuare le classi più rischiose consente di combattere i bug in modo proattivo e di stabilire priorità di testing più efficaci.

La defect prediction non si limita quindi a indicare *dove* è probabile trovare un difetto, ma cerca anche di chiarire *perché* quel rischio esiste e come ridurlo a livello di processo o di codice.

## 6.10 Pipeline: measure, clean data, analyze, model, explain

L'applicazione della Software Analytics alla defect prediction segue una pipeline precisa:

1. **Measure**: estrazione dei dati grezzi e raccolta delle metriche.
2. **Clean Data**: pulizia dei dati.
3. **Analyze**: ricerca di correlazioni e relazioni tra feature.
4. **Model**: costruzione e addestramento di modelli analitici o black-box.
5. **Explain**: estrazione di conoscenza utile per spiegare le predizioni.

### Nota del Prof

Nel corso verrà seguita esattamente questa pipeline: si partirà da una tabella grezza, la si analizzerà, si sceglieranno e si normalizzeranno le colonne, si costruirà il modello e infine si cercherà di estrarre conoscenza utile a evitare in futuro gli stessi errori.

### In sintesi

La pipeline consente di trasformare dati grezzi provenienti dai repository in **modelli predittivi** e poi in **conoscenza spiegabile e azionabile**. È uno schema concettuale fondamentale perché collega raccolta dati, modellazione e interpretazione.

## 6.11 Estrazione dati da ITS e VCS e raccolta metriche

Il primo passo della pipeline consiste nell'estrazione dei dati da due sorgenti principali:

- **ITS (Issue Tracking System)**: ad esempio Jira, che contiene ticket, bug report e informazioni di progetto.
- **VCS (Version Control System)**: ad esempio Git, che contiene codice sorgente, snapshot e log dei commit.

Da queste fonti si raccolgono diverse famiglie di metriche:

- **Code Metrics**: dimensione, complessità, design object-oriented, accoppiamento, coesione.
- **Process Metrics**: numero di commit, churn, difetti pre-release, pratiche di sviluppo.
- **Human Factors**: code ownership, numero di sviluppatori che hanno toccato il file, esperienza dello sviluppatore.

### Nota del Prof

Più persone modificano una stessa classe, maggiore è il rischio di introdurre difetti. Inoltre, se un commit è effettuato da uno sviluppatore che non ha esperienza su quella classe, la probabilità di errore cresce sensibilmente.

Combinando dati da **Jira** e **Git**, si ottengono quindi metriche sul codice, sul processo e sui fattori umani, tutte utili per la defect prediction.

## 6.12 Identificazione difetti, labeling e defect dataset

Per addestrare un modello di defect prediction è necessario costruire un dataset etichettato, in cui ogni classe venga marcata come **difettosa** o **non difettosa**.

Questo processo dipende dal **linkage** tra:

- ticket nell'Issue Tracking System;
- commit nel Version Control System;
- versioni del software coinvolte nel ciclo di vita del difetto.

### Nota del Prof

Quando uno sviluppatore effettua un commit per risolvere un problema, inserisce spesso nel messaggio del commit l'ID del ticket Jira associato. Questo ID rappresenta il razionale del cambiamento. Grazie a questa connessione tra commit e ticket, e tramite tecniche come **SZZ** e **Proportion**, è possibile non solo sapere quale commit ha corretto il bug, ma anche risalire a ritroso fino al commit che lo aveva introdotto.

## In sintesi

Il **labeling** dipende dalla capacità di collegare correttamente ticket, commit e release. Questo collegamento è alla base della costruzione del **defect dataset** ed è il passaggio che determina la qualità dei dati su cui verranno addestrati i modelli.

## 6.13 Black-box models e necessità di spiegazione

Molti algoritmi di apprendimento producono modelli **black-box**: ricevono un input e restituiscono una predizione, ma senza rendere trasparente il ragionamento interno che ha portato a quella decisione.

Per esempio, un modello può dire che una certa classe ha alta probabilità di essere difettosa, ma senza spiegare quali fattori abbiano portato a quel risultato.

### Nota del Prof

Un predittore come una rete neurale può essere molto accurato, ma non sa giustificare in modo leggibile il perché della sua scelta. Nell'ingegneria del software, invece, capire il motivo della difettosità è cruciale, perché consente al manager di agire sulla causa e non soltanto sul sintomo.

In contesto ingegneristico, quindi, la sola accuratezza non basta: il valore operativo di una predizione aumenta notevolmente quando essa è accompagnata da una spiegazione comprensibile.

## 6.14 XAI, GDPR, fairness, accountability, transparency

La necessità di spiegare i modelli è legata sia a motivazioni tecniche sia a motivazioni etiche e legali. In questo contesto si colloca il paradigma **FAT**:

- **Fairness**;
- **Accountability**;
- **Transparency**.

L'**Articolo 22 del GDPR** richiede che i processi decisionali automatizzati che hanno impatto sugli individui siano accompagnati da una spiegazione adeguata.

Questo porta a domande fondamentali:

- I sistemi di AI rispettano le leggi?
- I modelli producono decisioni discriminatorie?
- Comprendiamo davvero perché un modello fa una certa predizione?

### Nota del Prof

Una tecnica di Explainable AI consiste nell'affiancare a un modello black-box, come una rete neurale, un modello gemello spiegabile, ad esempio un albero decisionale.

Invece di addestrare l'albero direttamente sui dati originali, lo si addestra sugli output generati dalla rete neurale, così da mimarne il comportamento e renderne più leggibili le decisioni.

## In sintesi

La **XAI** nasce dall'esigenza di rendere i modelli comprensibili, verificabili e compatibili con vincoli legali ed etici come quelli imposti dal GDPR. In questo quadro, la spiegabilità non è un'aggiunta opzionale, ma una proprietà sempre più centrale.

## 6.15 CMMI Level 5 e process improvement

Il **CMMI (Capability Maturity Model Integrated)** è uno standard per misurare la maturità dei processi di sviluppo software.

I livelli principali sono:

1. **Initial**
2. **Managed**
3. **Defined**
4. **Quantitatively Managed**
5. **Optimizing**

Il **Level 5**, cioè *Optimizing*, rappresenta il livello più elevato: le performance del processo vengono migliorate in modo continuo tramite innovazione e controllo quantitativo.

### Nota del Prof

Il professore riporta un'esperienza diretta in cui ha aiutato un'azienda a mantenere il Livello 5 del CMMI. L'obiettivo era dimostrare statisticamente che certe strategie di miglioramento avessero prodotto benefici reali. I risultati hanno mostrato una forte riduzione dei difetti e un marcato aumento del profitto per dipendente.

Il riferimento al CMMI mostra bene come metriche, controllo statistico e miglioramento continuo possano essere integrati in una visione strutturata della qualità.

## 6.16 Process chart e controllo statistico

Per controllare la stabilità del processo si utilizzano i **process chart** o **control chart**. Essi servono a:

- identificare problemi mentre si verificano;
- predire l'intervallo atteso dei risultati del processo;
- verificare se il processo è statisticamente stabile;
- decidere se intervenire con correzioni locali o con cambiamenti più strutturali.

La costruzione di un process chart prevede:

1. scelta della variabile da osservare;
2. scelta dell'unità temporale;
3. calcolo della **media**;
4. calcolo dell'**Upper Control Limit** come media più tre deviazioni standard;
5. calcolo del **Lower Control Limit** come media meno tre deviazioni standard.

#### Nota del Prof

Finché i dati restano all'interno dei limiti di controllo, il processo è considerato statisticamente stabile. Se invece un punto supera uno dei limiti, si attiva una retrospettiva per analizzare la root cause dell'anomalia e capire come evitare che si ripeta.

I **process chart** permettono dunque di monitorare il processo nel tempo e di individuare rapidamente deviazioni anomale che richiedono analisi e intervento.

## 6.17 Casi applicativi mostrati nelle slide, anche se solo introdotti

Le lezioni presentano anche alcuni casi applicativi concreti.

### 6.17.1 Simulatore di release e defect prediction

#### Nota del Prof

È stata sviluppata un'interfaccia per supportare i manager nel **release planning**. Modificando alcuni slider relativi a variabili di input, come la percentuale di God Classes, la compliance architetturale o il numero di nuovi sviluppatori, il modello ricalcolava dinamicamente il numero atteso di difetti nella release successiva. Il sistema forniva anche intervalli di confidenza e scenari di best case e worst case.

### 6.17.2 Traceability recovery

#### Solo nelle slide (non spiegato a voce)

Nelle slide si introducono studi sull'uso di tecniche NLP e classificatori di ML per stimare il numero di link rimanenti nel recupero della tracciabilità dei requisiti.

### 6.17.3 Defectiveness prediction via JIT

Solo nelle slide (non spiegato a voce)

Le slide finali introducono il tema del miglioramento della defect prediction combinando informazioni **Just-In-Time** a livello di commit con metriche classiche a livello di classe e metodo.

Le tecniche discusse trovano applicazione sia in strumenti concreti di supporto decisionale per i manager, sia in linee di ricerca avanzate su **tracciabilità** e **predizione dei difetti**.

## 6.18 Riepilogo finale

I concetti chiave emersi sono i seguenti:

- Il **Machine Learning** apprende dai dati passati per prendere decisioni sul futuro.
- Le principali famiglie di problemi sono **associazioni**, **classificazione** e **regressione**.
- La **Software Analytics** usa dati provenienti da repository, issue tracker e pipeline di sviluppo per migliorare qualità, processi e produttività.
- La **defect prediction** serve a stimare quali classi o componenti sono più a rischio di contenere difetti.
- La pipeline fondamentale è: **Measure** → **Clean Data** → **Analyze** → **Model** → **Explain**.
- Il **labeling** delle classi dipende dal corretto collegamento tra ticket, commit e release.
- I modelli **black-box** sono potenti ma poco trasparenti, perciò diventa centrale la **Explainable AI**.
- La spiegabilità è importante non solo dal punto di vista tecnico, ma anche per ragioni di **fairness**, **accountability**, **transparency** e conformità al **GDPR**.
- Il **CMMI Level 5** e i **process chart** mostrano come il controllo quantitativo del processo possa tradursi in miglioramenti concreti di qualità e produttività.

# Capitolo 7

## SZZ: Identifying Buggy Entities

### 7.1 Contesto

#### 7.1.1 Cos'è la defect prediction

La **defect prediction** è una tecnica che mira a identificare gli **artefatti software** — per esempio classi, metodi o linee di codice — che hanno un'alta probabilità di contenere un difetto. L'obiettivo principale è **ridurre i costi e i tempi del testing e della code review**, permettendo agli sviluppatori di concentrare lo sforzo sulle parti più critiche del sistema.

##### Nota del Prof

Il testing, sia automatico sia manuale, può essere molto costoso in termini economici e computazionali. Il punto della defect prediction non è “indovinare i bug” in astratto, ma usare i dati storici per **prioritizzare** le attività di qualità. Anche se nel progetto del corso si lavora soprattutto a livello di classe, la stessa logica può essere applicata anche a commit, metodi o linee di codice.

#### 7.1.2 Perché la qualità del dataset è fondamentale

L'**affidabilità di un modello di predizione** dipende direttamente dalla **qualità del dataset** con cui il modello viene addestrato. Se le etichette sono sbagliate o rumorose, il classificatore apprenderà relazioni imprecise e tenderà a produrre risultati poco affidabili.

##### Nota del Prof

Qui vale in pieno il principio del *Garbage In, Garbage Out*: se i dati in ingresso sono sbagliati, incompleti o rumorosi, anche il modello finale sarà inevitabilmente distorto.

#### 7.1.3 Principali fonti di rumore nei dataset

Tra le principali fonti di rumore citate nelle slide compaiono:

- defect misclassification;
- defect origin.

Solo nelle slide (non spiegato a voce)

Le slide ricordano che lavori precedenti hanno già identificato queste fonti di rumore e hanno proposto tecniche per affrontarle.

### Nota del Prof

Un esempio tipico di **misclassification** è quando un ticket classificato come “bug” in realtà descrive una nuova funzionalità o un miglioramento. Un altro problema è il **linkage**: se il commit di fix non contiene l’ID del ticket, allora si perde il collegamento tra issue tracker e version control system, e diventa molto più difficile ricostruire quali classi fossero davvero difettose.

## 7.2 Cos’è SZZ

### 7.2.1 Definizione e obiettivo

**SZZ** è un processo algoritmico usato per identificare **quando un difetto è stato effettivamente introdotto** all’interno di un progetto software. In altre parole, non si limita a dire quale commit ha corretto il problema, ma cerca di risalire al commit precedente che ha inserito la modifica difettosa.

### Nota del Prof

Il nome **SZZ** deriva semplicemente dai cognomi degli autori del lavoro originario.

### 7.2.2 Uso del versioning e dell’annotation

Il metodo sfrutta il **meccanismo di annotation** dei sistemi di versionamento, ad esempio il comando `git blame` in Git. Questo meccanismo permette di associare ogni riga del codice alla revisione che l’ha modificata per ultima.

### Nota del Prof

L’annotation è importante perché consente di guardare una riga del file nello stato corrente e chiedersi: *chi l’ha scritta o modificata per ultimo, e in quale commit?* È proprio questa informazione che permette a SZZ di fare la ricerca all’indietro.

### 7.2.3 Idea di partire dal fix per risalire al bug-introducing change

L’idea centrale di SZZ è partire da un’informazione affidabile nel presente, cioè il **fix commit**, e usarla per ricostruire il passato. In particolare, l’algoritmo osserva **quali linee sono state cambiate per correggere il bug** e poi cerca di capire **quando quelle stesse linee erano state modificate l’ultima volta prima del fix**.

Il commit precedente individuato da questa ricerca viene considerato il **bug-introducing change**, cioè il candidato che ha introdotto il difetto.

## IMPORTANTE PER L'ESAME

Il punto chiave da ricordare è questo: **SZZ usa la correzione del bug come indizio per ricostruirne l'origine**. Non parte dal bug report in senso astratto, ma dalla porzione concreta di codice che è stata cambiata per eliminare il problema.

## 7.3 Come funziona SZZ

### 7.3.1 Identificazione del fix commit

Il primo passo consiste nell'identificare il **fix commit**, cioè il commit che corregge il difetto.

#### Nota del Prof

Operativamente, si parte dall'issue tracker, ad esempio Jira, si selezionano i ticket chiusi e classificati come bug, e si cercano i commit collegati a quei ticket. Quel commit rappresenta la **fix** osservabile nel repository.

### 7.3.2 Analisi delle linee modificate

Una volta trovato il fix commit, SZZ confronta la versione corretta con la versione immediatamente precedente e individua le **linee di codice effettivamente toccate dalla correzione**. Questo passaggio viene normalmente realizzato tramite una **diff**.

L'idea è che il difetto sia collegato proprio a quella parte di codice che è stata modificata nel fix. Perciò SZZ non analizza l'intero file in modo indistinto, ma si concentra sulla porzione di codice che ha subito la correzione.

#### Nota del Prof

SZZ non “capisce” il significato del bug. Non fa reasoning semantico. Vede solo che certe righe sono state corrette e assume che proprio lì si trovi la traccia storica utile per risalire all'origine del problema.

### 7.3.3 Ricerca all'indietro dell'ultima modifica precedente

Dopo aver isolato le linee corrette, l'algoritmo esegue una **backward search**. Per ogni riga coinvolta nel fix, usa il meccanismo di annotation per trovare **l'ultima modifica precedente** che aveva scritto o cambiato quella stessa riga.

Quel commit storico viene trattato come il miglior candidato disponibile per il **bug-introducing commit**. L'idea è quindi:

fix osservato nel presente → righe corrette → ricerca storica dell'ultima  
modifica precedente

## IMPORTANTE PER L'ESAME

SZZ è un algoritmo di **tracciamento storico**, non di comprensione semantica del bug. Funziona bene quando il problema è davvero contenuto nelle linee poi corrette;

funziona peggio quando il bug dipende da contesto, interazioni più ampie o codice mancante.

### 7.3.4 Differenza tra bug-fix change e bug-introducing change

È fondamentale distinguere chiaramente tra:

- **bug-fix change**: il commit che *risolve* il problema correggendo il codice;
- **bug-introducing change**: il commit passato che, secondo SZZ, ha *introdotto* il difetto, cioè la modifica storica da cui ha origine la porzione di codice poi corretta.

La differenza non è solo temporale, ma anche metodologica: il **bug-fix change** è osservabile con certezza, mentre il **bug-introducing change** è una **stima inferita** tramite backward search.

#### Nota del Prof

Nel dataset di defect prediction questa distinzione è decisiva. Non basta sapere quale commit ha riparato il bug: bisogna stimare da quale versione in poi una certa classe o un certo file debbano essere considerati difettosi.

## 7.4 Esempi nelle slide

### 7.4.1 Esempio 1: “When Do Changes Induce Fixes?”

#### Solo nelle slide (non spiegato a voce)

Il primo diagramma mostra una sequenza temporale in cui viene segnalato un bug e, in una revisione successiva, viene effettuato un fix. Nel fix commit vengono modificati tre metodi distinti, indicati come `a()`, `b()` e `c()`. SZZ traccia all'indietro la storia di ciascuna di queste porzioni di codice e individua tre revisioni precedenti differenti, ad esempio 1.11, 1.14 e 1.16.

Questo esempio è importante perché mostra che **un singolo bug fix può dipendere da più cambiamenti storici diversi**. Il fix avviene in un unico momento, ma le righe o i metodi corretti possono essere stati introdotti o alterati in revisioni differenti.

Il messaggio didattico è quindi duplice:

1. SZZ lavora a livello di **porzioni di codice**, non a livello astratto di “bug” nel suo complesso;
2. la genealogia del difetto può essere **distribuita** su più commit precedenti.

## 7.4.2 Esempio 2: “The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study”

### Nota del Prof

Nel secondo esempio compare un ciclo `for`. Nella versione che introduce il comportamento errato, la condizione è scritta come `for(int i=0; i<=4; i++)`. In seguito al bug report #123, il fix corregge il codice trasformandolo in `for(int i=0; i<4; i++)`.

L'esempio serve a visualizzare il funzionamento operativo di SZZ in modo molto concreto:

1. si individua il **fix commit** grazie al collegamento tra bug report e commit;
2. si esegue la **diff** rispetto alla versione precedente per isolare la riga corretta;
3. si traccia all'indietro la storia di quella riga per capire in quale commit era comparsa la forma difettosa.

Nel caso illustrato, la forma errata è la condizione `i<=4`, che viene poi corretta in `i<4`. SZZ usa proprio questa differenza come indizio per risalire al **bug-introducing change**.

Questo esempio chiarisce bene la logica standard dell'algoritmo: **si osserva il fix, si isolano le righe corrette, si segue la loro storia all'indietro**.

## 7.5 Limiti di SZZ

Le slide sottolineano che SZZ ha diverse **limitazioni** e che la sua accuratezza non è perfetta. Il problema di fondo è che l'algoritmo si basa su un'assunzione forte: *il bug è stato introdotto nelle stesse linee che vengono poi modificate per correggerlo*. Questa ipotesi è spesso utile, ma non sempre vera.

### 7.5.1 Code additions

#### Solo nelle slide (non spiegato a voce)

Le slide riportano che solo il **26–29%** dei **bug-introducing commits** può essere trovato con un algoritmo basato sull'assunzione che un bug sia stato introdotto dalle stesse linee di codice poi modificate per fixarlo.

Questo limite emerge quando il bug non viene corretto modificando una riga sbagliata già esistente, ma **aggiungendo codice che prima mancava**. Per esempio, il fix può consistere nell'inserire un controllo, una condizione o una gestione d'errore assente nella versione difettosa.

In questo caso, SZZ fallisce o diventa molto impreciso, perché non esiste una versione precedente di quella nuova riga da tracciare con `git blame`.

### Nota del Prof

Se il difetto dipende da qualcosa che mancava nel codice, l'algoritmo cerca nel passato una riga che in realtà non esisteva ancora. Questo spiega perché i casi di **code addition** siano così problematici per SZZ.

## 7.5.2 Regressive bugs

I **regressive bugs** sono difetti che emergono dopo modifiche successive del sistema, anche se la riga coinvolta in passato era corretta nel contesto originario.

Il problema è che SZZ tende a tornare alla **prima modifica storica trovata su quella riga**, attribuendo il bug a un commit antico che magari, al momento della sua introduzione, non era affatto errato. La riga può essere diventata problematica solo più tardi, quando il contesto del sistema è cambiato.

### Nota del Prof

Qui il limite è concettuale: una riga non è necessariamente “sbagliata in assoluto”. Può diventare sbagliata solo dopo l'introduzione di una nuova feature o dopo una modifica dell'ambiente o dei requisiti. SZZ, però, tende a incolpare chi ha scritto per primo quella riga.

## 7.5.3 Refactoring

Il **refactoring** cambia la struttura del codice senza modificarne, almeno idealmente, il comportamento funzionale. Esempi tipici sono rinominare variabili, spostare metodi o riorganizzare il codice.

Se la prima modifica precedente trovata da SZZ è proprio un refactoring, l'algoritmo rischia di etichettare quel commit come **bug-introducing**, anche se quel commit non ha realmente introdotto il difetto. In altre parole, il refactoring può **interrompere la pista storica** e nascondere il vero punto di introduzione del bug.

### Nota del Prof

SZZ segue la storia delle righe, non il significato profondo della funzionalità. Per questo un refactoring può far sembrare che il bug nasca lì, quando in realtà quella modifica ha solo riscritto o spostato codice senza introdurre il problema.

## 7.5.4 Imprecise backward search

La **imprecise backward search** indica che SZZ può essere abbastanza efficace nell'individuare **quale porzione di codice** sia collegata al difetto, ma molto meno preciso nello stabilire **da quanto tempo** quella porzione debba essere considerata buggy.

Una riga di codice può essere stata introdotta molto tempo prima in modo corretto e diventare difettosa solo in seguito a cambiamenti dei requisiti, del contesto o delle interazioni con altre parti del sistema. Se SZZ risale troppo indietro, rischia di etichettare come difettose versioni che, storicamente, erano ancora corrette.

### IMPORTANTE PER L'ESAME

Questo è uno dei limiti più importanti da ricordare: **SZZ** è spesso più affidabile nel dire “dove guardare” che nel dire con precisione “da quando” il codice sia diventato errato.

#### 7.5.5 Snoring

##### Solo nelle slide (non spiegato a voce)

Nelle slide lo **snoring** è semplicemente citato nell'elenco delle limitazioni, ma non viene spiegato nel dettaglio.

Nel quadro generale del corso, lo snoring è il fenomeno per cui alcune classi contengono già difetti, ma questi difetti non sono ancora stati scoperti o corretti. Di conseguenza, il dataset può continuare a etichettarle come non difettose.

Il collegamento con SZZ è che l'algoritmo dipende dall'esistenza di un **fix osservabile**. Se un difetto esiste ma non è ancora stato corretto, SZZ non ha alcuna traccia concreta da seguire e quindi non può ricostruirne l'origine.

##### Nota del Prof

In questa lezione lo snoring viene solo accennato. L'idea essenziale è che SZZ vede soltanto i bug che hanno già lasciato una traccia nella storia dei fix. I bug ancora “dormienti” sfuggono all'algoritmo e contribuiscono a sporcare il labeling del dataset.

### 7.6 Da ricordare per l'esame

- **Definizione di SZZ:** è un algoritmo che sfrutta il meccanismo di annotation dei sistemi di versionamento per risalire dal **fix commit** al possibile **bug-introducing commit**.
- **Idea chiave:** si parte dalle **linee corrette dal fix** e si cerca l'ultima modifica precedente di quelle stesse linee.
- **Differenza fondamentale:** il **bug-fix change** è il commit che ripara il problema; il **bug-introducing change** è il commit storico che, secondo SZZ, ha introdotto la modifica difettosa.
- **Passaggi essenziali:** identificare il fix commit, eseguire la diff, isolare le righe corrette, usare `git blame` per la backward search.
- **Principali limiti:** **code additions**, **regressive bugs**, **refactoring**, **imprecise backward search** e, in senso più ampio, i problemi legati ai bug non ancora osservabili come nel caso dello **snoring**.

# Capitolo 8

## Leveraging Defects Lifecycle for Labeling Defective Classes

### 8.1 Contesto e obiettivo del lavoro

#### 8.1.1 Defect prediction

La **defect prediction** ha l'obiettivo di identificare gli artefatti software, ad esempio classi o file, che hanno un'alta probabilità di manifestare un difetto. Lo scopo principale è ridurre i costi di testing e di code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.

#### 8.1.2 Importanza del dataset

Affinché un modello di predizione sia affidabile, è fondamentale costruire **dataset di difetti di alta qualità**. L'intero lavoro si concentra infatti non tanto sul classificatore in sé, quanto sulla fase precedente, cioè la **costruzione del dataset** e in particolare il **labeling** delle classi come difettose o non difettose.

Se il labeling è inaccurato, il modello apprenderà relazioni sbagliate tra metriche e defectiveness. Per questo motivo, la qualità del dataset non è un dettaglio secondario, ma una condizione necessaria per ottenere risultati credibili.

#### Nota del Prof

Ci troviamo nella fase iniziale di **misurazione** e creazione del dataset. Prima ancora di addestrare un modello di Machine Learning, bisogna capire come etichettare correttamente le classi che in passato sono state davvero affette da difetti.

#### 8.1.3 Labeling delle classi difettose

Le slide presentano due strategie principali per etichettare le classi come difettose o non difettose:

- usare **SZZ**;
- usare la **Earliest Affected Version (AV) in JIRA**, quando disponibile.

Il problema è che entrambe queste fonti possono essere incomplete o inaccurate. Da qui nasce la necessità di studiare metodi automatici alternativi per ricostruire il ciclo di vita del difetto.

### 8.1.4 Obiettivo generale dello studio

Lo studio ha un duplice obiettivo.

Da un lato, vuole capire **quanto siano disponibili e affidabili le informazioni di AV** inserite dagli sviluppatori nei ticket Jira. Dall'altro, propone un nuovo metodo automatizzato, chiamato **Proportion**, basato sull'idea che i difetti abbiano un **life-cycle relativamente stabile**.

In sintesi, il lavoro vuole:

1. misurare quanto sia realistico affidarsi ai dati di AV presenti in Jira;
2. proporre un metodo alternativo quando l'AV manca o è incoerente;
3. confrontare l'accuratezza del nuovo metodo con quella di diverse varianti di **SZZ**.

## 8.2 Concetti fondamentali

### 8.2.1 SZZ

**SZZ** è l'algoritmo classico per identificare il momento in cui un difetto è stato introdotto nel codice. A partire da un **fix commit**, SZZ usa il meccanismo di annotation del versioning system, per esempio **git blame**, per risalire all'ultima modifica precedente delle linee corrette.

### 8.2.2 Earliest AV in JIRA

L'approccio **Earliest AV in JIRA** consiste nell'usare la prima **Affected Version** riportata manualmente dagli sviluppatori nel sistema di tracciamento, ad esempio Jira. Se questa informazione fosse sempre presente e consistente, costituirebbe un riferimento molto utile per il labeling.

### 8.2.3 Defect lifecycle

L'idea centrale del paper è che ogni difetto attraversi un **ciclo di vita** che può essere descritto in termini di versioni del software. Questo ciclo di vita collega il momento in cui il difetto viene introdotto, quello in cui viene osservato e quello in cui viene definitivamente corretto.

### 8.2.4 Significato di IV, OV, FV e AV

Per descrivere il life-cycle del difetto, il paper usa quattro nozioni fondamentali.

- **IV (Injected Version)**: è la versione in cui il difetto viene introdotto nel codice.

#### Nota del Prof

È la prima versione in cui il bug esiste davvero nel sistema, anche se magari nessuno se n'è ancora accorto.

- **OV (Opening Version)**: è la versione in cui viene aperto il ticket del bug.

#### Nota del Prof

È il momento in cui la *failure*, cioè il malfunzionamento osservabile, viene riconosciuta e registrata.

- **FV (Fixed Version)**: è la prima versione rilasciata dopo il *fix commit*.

#### Nota del Prof

È la prima release in cui quel bug non dovrebbe più comparire.

- **AV (Affected Versions)**: sono tutte le versioni affette dal difetto, cioè l'intervallo di versioni comprese tra il momento di introduzione e quello di correzione.

In termini concettuali, le versioni difettose sono quelle comprese nell'intervallo  $[IV, FV)$ . Questo significa che la **Fixed Version** non è più affetta dal bug, mentre tutte le versioni precedenti a partire dalla **Injected Version** lo sono.

## 8.3 SZZ: funzionamento e limiti

### 8.3.1 Come funziona SZZ

SZZ individua i commit che hanno introdotto il bug tracciando la storia del codice sorgente. L'algoritmo parte da un **fix commit**, cioè da un commit che sappiamo correggere un bug, e usa quella correzione come indizio per cercare all'indietro il possibile **bug-introducing commit**.

### 8.3.2 Uso di `git blame` / `annotation mechanism`

Operativamente, SZZ prende le linee di codice modificate da un **fix commit** e usa `git blame` per risalire nel tempo, identificando il commit precedente che aveva introdotto o modificato quelle stesse righe.

L'assunzione di fondo è che il bug sia stato introdotto proprio nelle linee poi corrette dal fix. Quando questa assunzione è vera, SZZ può fornire una buona stima del momento di introduzione del difetto.

### 8.3.3 Bug introducing commit

Il commit individuato tramite backward search viene considerato il **bug-introducing commit**. Da quel momento in poi, la classe o il file coinvolto vengono etichettati come difettosi.

### Nota del Prof

In pratica, il commit che ha scritto la forma sbagliata del codice viene trattato come l'origine storica del bug. È un'inferenza plausibile, ma non sempre perfetta.

## 8.3.4 Limiti del metodo

Le slide chiariscono che SZZ ha limiti strutturali importanti.

- **Non è perfettamente accurato:** il commit trovato da SZZ è una stima, non una verità certa.
- **Non riesce a identificare bene i regression bugs:** un codice può diventare errato solo in seguito a cambiamenti successivi del sistema, anche se in passato era corretto.
- **Non funziona bene quando il fix consiste solo in aggiunta di codice:** se il bug viene corretto introducendo codice nuovo, senza modificare righe già esistenti, SZZ non ha una traccia storica affidabile su cui applicare `git blame`.

### Nota del Prof

SZZ può generare sia **falsi positivi** sia **falsi negativi**. Produce falsi positivi quando va troppo indietro nel tempo e attribuisce il difetto a linee che storicamente erano corrette; produce falsi negativi quando non riesce a risalire al vero punto di introduzione del bug.

## 8.4 Aim dello studio e Research Questions

### 8.4.1 Idea di stable life-cycle dei defect

L'intuizione centrale dello studio è che i difetti abbiano un **ciclo di vita stabile**. In altre parole, pur variando da bug a bug, sembrerebbe esistere una relazione abbastanza regolare tra:

- il numero di versioni che intercorrono tra introduzione del difetto e sua scoperta;
- il numero di versioni che intercorrono tra scoperta del difetto e sua correzione.

Questa ipotesi motiva il metodo **Proportion**: se tale proporzione è abbastanza stabile, allora è possibile stimare automaticamente la posizione dell'IV anche quando l'AV esplicita manca.

### 8.4.2 Research Questions

Le domande di ricerca dello studio sono quattro:

- **RQ1:** le **Affected Versions (AV)** sono disponibili e consistenti in Jira?
- **RQ2:** i vari metodi, ad esempio SZZ e Proportion, hanno accuratèzze diverse nell'etichettare le **Affected Versions**?

- **RQ3:** i metodi hanno accuratezze diverse nell'etichettare le **single classi difettose**?
- **RQ4:** l'uso di metodi diversi porta all'identificazione di **feature importanti** diverse nei modelli di predizione?

In questo capitolo ci fermiamo a **RQ1** e **RQ2**.

## 8.5 RQ1: Is the AV available and consistent?

### 8.5.1 Rationale

La prima domanda vuole verificare se il campo **Affected Version** sia effettivamente utilizzabile come fonte affidabile per il labeling. Ci sono infatti due problemi distinti:

- **availability:** gli sviluppatori potrebbero non compilare affatto il campo AV;
- **consistency:** il campo potrebbe essere compilato, ma in modo incoerente rispetto all'ordine temporale delle versioni.

#### Nota del Prof

Per esempio, se la versione più vecchia indicata come affetta risulta successiva all'apertura del ticket, allora quella AV non è coerente e non può essere usata come ground truth affidabile.

### 8.5.2 Design e selezione di progetti e bug

Per RQ1 sono stati analizzati **212 progetti Apache** collegati sia a **Git** sia a **Jira**. Sono stati considerati solo gli issue report che soddisfacevano simultaneamente i seguenti criteri:

- `Type == Bug`;
- `Status == Closed` oppure `Resolved`;
- `Resolution == Fixed`.

Inoltre, sono stati esclusi:

- i difetti che non avevano un **Git-related fix commit**;
- i difetti che **non erano post-release**.

Dal punto di vista sperimentale:

- la **variabile indipendente** è il singolo **defect**;
- la **variabile dipendente** è la **percentuale di AV disponibili** e di **AV disponibili e consistenti**.

### 8.5.3 Risultati

I risultati mostrano che il problema è molto rilevante. Sul totale dei difetti selezionati:

- il numero complessivo di bug chiusi e collegati a Git nei 212 progetti Apache è pari a **125.860**;
- **63.539 bug**, cioè circa il **51%**, risultano privi di AV utilizzabile oppure presentano un'AV inaffidabile.

Inoltre, per la maggior parte dei progetti, più del **25%** dei difetti non possiede una AV disponibile e consistente.

### 8.5.4 Discussione e implicazioni pratiche

La conclusione di RQ1 è netta: l'approccio basato esclusivamente sulla AV inserita in Jira **non è sufficiente** nella maggioranza dei casi.

Se più della metà dei bug non ha una AV affidabile, allora il labeling realistico basato solo sui dati espliciti del ticket diventa spesso impossibile. Questo giustifica la necessità di metodi automatici alternativi.

#### Nota del Prof

Se non abbiamo un'AV affidabile e non introduciamo un metodo compensativo, finiamo per etichettare come pulite molte classi che invece erano già affette da difetti. Questo produce inevitabilmente molti **falsi negativi** nel dataset.

## 8.6 RQ2: Do methods have different accuracy in AV labeling?

### 8.6.1 Razionale

Dato che RQ1 mostra che l'AV è spesso assente o inconsistente, RQ2 si chiede se esistano metodi automatici più accurati per **predire correttamente le versioni affette da un bug**.

La domanda non è più semplicemente "l'AV c'è?", ma: *quando manca, quale metodo ricostruisce meglio l'intervallo delle versioni difettose?*

### 8.6.2 Selezione dei progetti

Per valutare i metodi in modo affidabile, RQ2 restringe l'analisi a progetti per cui esiste una ground truth più solida. I criteri di selezione sono:

- progetti con più del **50%** di AV disponibili e consistenti;
- progetti con almeno **100 bug** collegati a un Git fix commit e con AV disponibile e consistente;
- progetti con almeno **6 versioni**.

Dopo questa selezione, rimangono **76 progetti Apache Jira**.

### 8.6.3 Formula della Proportion e significato di $P$

Il paper introduce una misura proporzionale del ciclo di vita del difetto:

$$P = \frac{FV - IV}{FV - OV}$$

Questa quantità misura, in termini di versioni, il rapporto tra:

- la distanza tra **Fixed Version** e **Injected Version**;
- la distanza tra **Fixed Version** e **Opening Version**.

Una volta stimata  $P$ , è possibile ricavare una **Predicted IV** mediante la formula:

$$\text{Predicted IV} = FV - (FV - OV) \cdot P$$

#### Nota del Prof

L'idea intuitiva è che, pur cambiando la durata assoluta dei singoli bug, la **proporzione** tra momento di scoperta e momento di correzione tenda a essere relativamente stabile. Per questo motivo si prova a usare  $P$  come “firma temporale” del lifecycle dei difetti.

### 8.6.4 Varianti di Proportion

Il paper confronta tre modi diversi di stimare  $P$ :

- **Cold Start:**  $P$  è calcolato come mediana dei difetti osservati negli altri progetti;
- **Increment:**  $P$  è calcolato come media dei difetti corretti nelle versioni precedenti dello stesso progetto;
- **Moving Window:**  $P$  è calcolato come media dell'ultimo 1% dei difetti corretti.

Le tre varianti riflettono tre strategie diverse: usare esperienza di altri progetti, usare solo il passato interno del progetto, oppure concentrarsi sul comportamento più recente del processo.

#### Nota del Prof

A lezione è stata discussa anche una variante chiamata **Total**, che usa tutti i difetti passati e futuri per stimare  $P$ . Questa variante non appartiene al paper, ma è utile per capire il problema del realismo temporale.

#### IMPORTANTE PER L'ESAME

La variante **Total** è concettualmente scorretta in ottica di Machine Learning realistico, perché usa informazioni del futuro per etichettare il passato. In un vero scenario predittivo, il futuro non è disponibile al momento della costruzione del dataset.

### 8.6.5 Metodi confrontati

I metodi messi a confronto in RQ2 sono:

- **Simple**: assume semplicemente  $IV = OV$ ;
- **SZZ\_B** (*Basic*);
- **SZZ\_U** (*Unleashed*);
- **SZZ\_RA** (*Refactoring Aware*);
- **Proportion\_ColdStart**;
- **Proportion\_Increment**;
- **Proportion\_MovingWindow**;
- le versioni **SZZ\_X+**, cioè ciascun metodo SZZ combinato con **Simple**.

### 8.6.6 Variabili dipendenti e definizione di TP, TN, FP, FN

L'unità di analisi di RQ2 non è ancora la classe-versione, ma la **versione rispetto a un singolo difetto**. In altre parole, si valuta se un metodo etichetta correttamente una certa versione come affetta

# Capitolo 9

## Defect Labeling, Evaluation and Metrics

### 9.1 RQ3: Accuratezza nel labeling delle classi

#### 9.1.1 Differenza rispetto a RQ2

Mentre in **RQ2** l'obiettivo era valutare l'accuratezza nell'individuare le **Affected Versions (AV)** di un difetto, in **RQ3** il livello di analisi diventa più fine. Lo scopo non è più soltanto capire se una *versione* sia affetta da un certo bug, ma costruire il dataset più accurato possibile per la **defect prediction a livello di classe**.

In altre parole, RQ2 ragiona ancora in termini di **versione-difetto**, mentre RQ3 passa al problema realmente utile per il dataset finale, cioè stabilire se una **specifica classe** in una **specifica versione** debba essere etichettata come difettosa oppure no.

#### 9.1.2 Rationale

Il rationale di questa ricerca è capire **quale metodo produca il dataset di defect prediction più accurato**. Nel contesto **within-project, across-version, class-level**, la domanda diventa:

una certa classe, in una certa release, deve essere considerata **defective** oppure **non-defective**?

Questa è la forma di labeling che serve davvero per addestrare un classificatore sulle classi software.

#### 9.1.3 Unità di analisi

L'unità di misura non è più la singola versione, ma la coppia **classe-versione**, per esempio *F1.java nella release 1*. Questo significa che ogni istanza del dataset finale rappresenta una classe osservata in una certa release.

#### 9.1.4 Design

Le **variabili indipendenti** rimangono le stesse di RQ2, cioè i diversi metodi di labeling:

- **Simple**;
- **SZZ\_B**, **SZZ\_U**, **SZZ\_RA**;
- **Proportion\_ColdStart**, **Proportion\_Increment**, **Proportion\_MovingWindow**;
- le varianti **SZZ\_X+**.

Le **variabili dipendenti** sono le stesse di RQ2, ma cambiano di significato perché l'unità di analisi non è più la versione, bensì la **defectiveness di una classe in una versione**.

Più precisamente:

- **True Positive (TP)**: la classe in una certa versione è realmente difettosa ed è etichettata come difettosa;
- **False Negative (FN)**: la classe in una certa versione è realmente difettosa, ma viene etichettata come non difettosa;
- **True Negative (TN)**: la classe in una certa versione è realmente non difettosa ed è etichettata come non difettosa;
- **False Positive (FP)**: la classe in una certa versione è realmente non difettosa, ma viene etichettata come difettosa.

Da queste quantità vengono poi calcolate le metriche standard:

- **Precision**;
- **Recall**;
- **F1**;
- **MCC**;
- **Kappa**.

### 9.1.5 Risultati

I risultati mostrano una struttura molto simile a quella già osservata in RQ2, ma con una differenza importante a livello di dominanza dei metodi.

- Come in RQ2, i metodi **Proportion** hanno **Precision** e accuratezza composita, cioè **F1**, **MCC** e **Kappa**, superiori a tutti i metodi **SZZ**.
- Come in RQ2, **SZZ\_U** ottiene la **Recall** più alta tra tutti i metodi.
- Come in RQ2, **SZZ\_B+** ha la più alta **Precision** all'interno della famiglia **SZZ**.
- Diversamente da RQ2, **Proportion\_MovingWindow** emerge come metodo dominante sulle **metriche composite**, risultando il più forte nel labeling delle classi.

### 9.1.6 Discussione

La discussione di RQ3 porta a tre osservazioni importanti.

La prima è che, confrontando i metodi **SZZ\_X** con le corrispondenti varianti **SZZ\_X+**, il guadagno nel **class labeling** è **inferiore all'1%**. Quindi, anche qui, l'integrazione con informazioni aggiuntive migliora poco i metodi SZZ sul piano pratico.

La seconda è che **tutti i metodi risultano più accurati nel labeling delle classi che nel labeling delle AV**. Questo è un punto metodologicamente importante: etichettare correttamente una classe è più facile che ricostruire perfettamente l'intero intervallo delle versioni affette da un difetto.

La terza è che questo miglioramento è quantificabile. Rispetto al task di AV labeling, nel task di class labeling si osserva un aumento mediano di:

- **13%** in Precision;
- **5%** in Recall;
- **16%** in F1;
- **27%** in MCC;
- **39%** in Kappa.

#### Solo nelle slide (non spiegato a voce)

La spiegazione proposta nelle slide è che una singola classe può essere affetta da **più difetti simultaneamente**. Di conseguenza, anche se un metodo manca un difetto specifico, può comunque etichettare correttamente la classe come difettosa grazie alla presenza di un altro difetto concomitante.

#### Nota del Prof

La Recall di SZZ tende a essere più alta perché SZZ va molto più indietro nel tempo rispetto a Proportion. Questo fa emergere più classi positive, ma al prezzo di moltissimi **falsi positivi**. Quindi SZZ recupera più positivi, ma sporca molto di più il dataset.

## 9.2 RQ4: Identificazione delle feature importanti

### 9.2.1 Razionale

RQ4 studia l'effetto a cascata del labeling sulla comprensione del problema. Non basta infatti costruire un dataset etichettato: bisogna anche capire se il metodo di labeling usato influenza **quali feature sembrano importanti** per spiegare la defectiveness.

Il razionale è quindi doppio:

1. capire **quali feature** vengono selezionate come rilevanti dal dataset costruito con un certo metodo;
2. capire se la relazione tra queste feature e la variabile target, cioè la **class defectiveness**, venga preservata correttamente.

Questo è molto importante dal punto di vista pratico, perché le metriche selezionate come più rilevanti guidano anche l'interpretazione ingegneristica del fenomeno: per esempio, si può voler capire se il rischio dipenda da **LOC**, **Churn**, **Age**, **Change Set Size** o altre metriche di processo e prodotto.

### 9.2.2 Feature selection

Per la **feature selection** si confrontano le feature selezionate usando il dataset prodotto da un certo metodo con le feature selezionate usando il dataset **actual**, cioè quello assunto come riferimento.

Lo scopo è misurare quanto fedelmente un metodo di labeling riesca a preservare la scelta delle feature rilevanti.

Nel materiale del corso viene indicato un approccio di **Exhaustive Search Feature Selection**, implementato tramite Weka e `CfsSubsetEval`. L'idea generale è scegliere feature:

- fortemente informative rispetto alla defectiveness;
- poco ridondanti tra loro.

Per confrontare la correttezza della feature selection vengono riutilizzate metriche analoghe a quelle già viste:

- **TP**;
- **TN**;
- **FP**;
- **FN**;
- **Precision**;
- **Recall**;
- **F1**;
- **MCC**;
- **Kappa**.

### 9.2.3 Correlation analysis

La seconda metà di RQ4 riguarda la **correlation analysis**. In questo caso non si confrontano più insiemi di feature selezionate, ma il valore della correlazione misurata:

- nel dataset prodotto da un certo metodo;
- nel dataset **actual**.

L'idea è verificare se un metodo di labeling alteri o meno la relazione statistica tra una feature e la defectiveness. Se il labeling è rumoroso, anche la correlazione osservata può risultare distorta.

Nel materiale del corso questa parte viene associata alla **correlazione di Spearman**.

### 9.2.4 Standard Mean Relative Error

Per quantificare lo scostamento tra la correlazione **predicted** e quella **actual**, viene usato lo **Standard Mean Relative Error**, espresso come:

$$\frac{|\text{Predicted} - \text{Actual}|}{\text{Actual}}$$

Questa formula misura l'errore relativo tra ciò che il metodo stima e ciò che si osserverebbe nel dataset di riferimento.

### 9.2.5 Risultati

I risultati di RQ4 mostrano in modo molto netto la superiorità dei metodi **Proportion** rispetto ai metodi **SZZ** anche su questo piano.

- I metodi **Proportion** hanno un'accuratezza superiore a tutti i metodi **SZZ** in **tutte e cinque le metriche** considerate.
- I metodi **Proportion** sono gli unici a mostrare una **mediana perfetta** sia in **Precision** sia in **Recall**.
- Diversamente da quanto accade in RQ2 e RQ3, tra le diverse varianti di **SZZ** non emergono grandi differenze.
- Il miglioramento ottenuto passando da **SZZ\_X** a **SZZ\_X+** resta ancora una volta **inferiore all'1%**.

### 9.2.6 Discussione

La discussione rende ancora più esplicita la portata dei risultati.

- I metodi **Proportion** hanno un errore che è circa il **25%** dell'errore commesso dai metodi **SZZ**. In altre parole, nel task di identificazione delle feature importanti, **Proportion** sbaglia molto meno.
- A differenza di quanto visto in RQ2 e RQ3, persino il metodo **Simple** risulta più accurato di tutti i metodi **SZZ** in questo task, con un errore che è circa **la metà** di quello di **SZZ**.
- Anche qui il guadagno ottenuto usando le varianti **SZZ\_X+** è **praticamente irrilevante**, restando sotto l'1%.

#### Solo nelle slide (non spiegato a voce)

Le slide sottolineano che la selezione delle feature e l'osservazione delle correlazioni risultano, nel complesso e in media, significativamente più accurate quando si usa il **bugs' life-cycle** rispetto a qualunque metodo **SZZ**.

## 9.3 Figure, tabelle e workflow del paper

Solo nelle slide (non spiegato a voce)

Le figure, le tabelle e i workflow di questa sezione derivano dalle slide e dal paper, ma non sono stati spiegati a voce in maniera estesa. Conviene quindi leggerli come supporto alla comprensione del flusso metodologico.

### 9.3.1 Diagramma del defect lifecycle (Fig. 1.1)

Il diagramma del **defect lifecycle** rappresenta la sequenza temporale fondamentale di un bug:

$$IV \rightarrow OV \rightarrow Fix Commit \rightarrow FV$$

dove:

- **IV** è la **Injected Version**;
- **OV** è la **Opening Version**;
- **FV** è la **Fixed Version**.

Le versioni affette dal difetto sono quelle comprese nell'intervallo  $[IV, FV)$ . Questo significa che il bug esiste già da **IV** e smette di essere presente a partire dalla **FV**.

### 9.3.2 Workflow del predicted AV (Fig. 3.1)

Il workflow del **predicted AV** mostra come le informazioni provenienti da **Git** e **Jira** vengano combinate.

In particolare:

- da **Git** si ricava il **Bug Fix**;
- da **Jira** si ricavano **Ticket Creation** e **Releases**;
- un certo **Method**, ad esempio **Proportion**, usa queste informazioni per costruire una **Predicted AV**;
- la **Predicted AV** viene confrontata con la **Actual AV** per ricavare **TP**, **TN**, **FP** e **FN**.

### 9.3.3 Workflow del class labeling (Fig. 3.2)

Il workflow del **class labeling** prende i file o le classi toccate dal **fix** e li combina con l'informazione sulle versioni affette per stabilire, release per release, se una certa classe debba essere considerata difettosa oppure no.

Questo è il passaggio che trasforma un'informazione a livello di **bug/versione** in un'informazione a livello di **classe-versione**, cioè nel formato realmente utile per la defect prediction.

### 9.3.4 Workflow della creazione dei complete datasets (Fig. 3.3)

Nel workflow dei **complete datasets** al class labeling viene aggiunta la fase di **Metric Collection**. Il risultato finale è una tabella in cui, per ogni istanza classe-versione, si raccolgono:

- l'identificativo della **release**;
- il nome del **file** o della **classe**;
- la label di **defectiveness**;
- varie metriche software, ad esempio **LOC**, **LOC Touched**, **Nfix**, **Churn**, **Age**, **Change Set Size**.

### 9.3.5 Feature selection e correlation analysis (Fig. 3.4 e Fig. 3.5)

Le figure dedicate a **feature selection** e **correlation analysis** mostrano due flussi paralleli:

- nel primo caso, il dataset predetto e quello actual vengono sottoposti a **feature selection**, e si confrontano le feature selezionate;
- nel secondo caso, sui due dataset si calcolano le correlazioni e si confrontano tramite l'errore relativo.

In entrambi i casi, l'idea è verificare quanto bene il metodo di labeling preservi l'informazione utile per la spiegazione del fenomeno.

### 9.3.6 Interpretazione di grafici, boxplot e tabelle comparative

I grafici e i boxplot delle slide trasmettono due messaggi centrali.

Il primo è che la variabile  $P$  risulta **molto più stabile** rispetto a **IV**, **OV** e **FV**. Le slide osservano infatti che la deviazione standard di  $P$  è circa **5** tra progetti diversi e circa **2** all'interno dello stesso progetto. Questo supporta l'ipotesi di **stable life-cycle**.

Il secondo è che, nelle metriche di accuratezza, i metodi **SZZ** tendono a mostrare valori più bassi e meno stabili, soprattutto su **MCC** e **Kappa**, mentre i metodi **Proportion** risultano mediamente più alti e più robusti.

## 9.4 Evaluation of Machine Learning Models

### 9.4.1 Training, testing e tuning

La costruzione di un modello di Machine Learning richiede almeno tre momenti distinti:

- **training**, in cui il modello viene addestrato;
- **testing**, in cui il modello viene valutato su dati indipendenti;
- **tuning**, in cui si regolano parametri o scelte modellistiche per migliorarne le prestazioni.

### 9.4.2 Perché l'errore sul training set non basta

L'errore osservato sul **training set** non è sufficiente per stimare la qualità del modello, perché il modello è stato costruito proprio su quei dati. Una performance alta in training può quindi riflettere semplice adattamento ai dati storici, non reale capacità di generalizzazione.

### 9.4.3 Training set vs test set

Per valutare seriamente un classificatore, bisogna separare:

- un **training set**, usato per apprendere;
- un **test set**, usato solo per verificare quanto il modello sappia comportarsi su dati non visti.

#### Nota del Prof

Tutti i dati della tabella sono di tipo *supervised*, quindi conosciamo già l'etichetta corretta. Per simulare il caso reale, si nasconde una parte dei risultati e si usa il modello addestrato sul training set per predirli. Solo dopo si confrontano *actual* e *predicted*.

### 9.4.4 Dati rappresentativi

Affinché la stima dell'errore abbia senso, sia il training set sia il test set devono essere **rappresentativi** dello stesso problema. Se training e test descrivono popolazioni diverse, la valutazione non misura più correttamente la capacità di generalizzazione.

### 9.4.5 Problema dei dati limitati

Idealmente, training e test dovrebbero essere grandi. In pratica, però, nei problemi di defect prediction i dati etichettati non sono infiniti, e questo rende necessarie tecniche di validazione che riusino i dati in modo intelligente senza violare il realismo sperimentale.

#### Nota del Prof

L'assioma implicito di tutta la validazione è che **il futuro sia simile al passato**. Usiamo il passato per stimare come il modello si comporterà su dati futuri che non abbiamo ancora osservato.

## 9.5 Tecniche di validazione non temporali

Queste tecniche si usano quando l'ordine temporale dei dati non è determinante oppure quando le istanze possono essere trattate come indipendenti.

### 9.5.1 Holdout

Nel metodo **holdout** il dataset viene diviso una sola volta, per esempio in 2/3 training e 1/3 testing.

Il vantaggio è la semplicità; lo svantaggio è che il campione selezionato per test potrebbe non essere rappresentativo.

#### Nota del Prof

Se una classe rara finisce tutta nel test set, il modello non l'ha mai vista in training e quindi non può imparare a riconoscerla.

### 9.5.2 Stratification

La **stratification** è una variante dell'holdout che cerca di mantenere in training e test proporzioni simili della classe di interesse, così da ridurre il rischio di campioni troppo sbilanciati.

### 9.5.3 Repeated holdout

Nel **repeated holdout** si ripete più volte l'holdout con partizioni diverse e si media l'errore. In questo modo si attenua la dipendenza da una singola divisione sfortunata dei dati.

Il limite è che i test set possono sovrapporsi, e il costo computazionale cresce rapidamente.

#### Nota del Prof

Se una singola run richiede 5 ore, ripetere l'holdout 5 volte porta facilmente a 25 ore di elaborazione.

### 9.5.4 K-fold cross-validation

La **k-fold cross-validation** divide il dataset in  $K$  parti uguali. A ogni iterazione, una parte viene usata per il test e le altre  $K - 1$  per il training. Il processo viene ripetuto  $K$  volte, così che ogni istanza finisca nel test set esattamente una volta.

#### Nota del Prof

L'analogia della pizza è molto efficace: si taglia la pizza in  $K$  spicchi, se ne usa uno come test e gli altri come training, poi si cambia spicchio e si ripete finché ogni spicchio è stato usato una volta come test.

### 9.5.5 Stratified k-fold cross-validation e repeated stratified cross-validation

La versione **stratified** cerca di mantenere la proporzione delle classi in ogni fold. Una pratica comune è il **10 times 10-fold stratified**, che offre stime molto stabili ma richiede molte esecuzioni.

### 9.5.6 Leave-one-out cross-validation

La **leave-one-out cross-validation** è il caso limite in cui  $K = n$ , cioè ogni test set contiene una sola istanza. Massimizza l'uso dei dati per il training, ma ha costi computazionali elevati e rende impossibile la stratificazione.

### 9.5.7 Bootstrap

Il **bootstrap** usa campionamento con reinserimento. Alcune istanze possono comparire molte volte nel training set, mentre altre possono non comparire affatto.

#### Nota del Prof

Nel contesto del corso questo metodo viene sconsigliato, perché tende a produrre campioni troppo distorti e poco realistici per la defect prediction.

## 9.6 Tecniche di validazione temporali

Queste tecniche si usano quando l'ordine temporale è parte integrante del problema, come accade nei dataset software basati sull'evoluzione delle release.

### 9.6.1 Time-series validation

La **time-series validation** preserva l'ordine cronologico dei dati, evitando che il training contenga informazioni provenienti dal futuro rispetto al test.

### 9.6.2 Ordered holdout

L'**ordered holdout** è un holdout che rispetta la cronologia: la parte iniziale del dataset va nel training set e la parte finale nel test set.

### 9.6.3 Walk-forward

Nel **walk-forward** il dataset viene diviso in blocchi cronologici, per esempio per release. A ogni passo, si addestra il modello su tutte le release precedenti e si testa su quella successiva.

#### Nota del Prof

Se l'obiettivo è predire la **Release 4**, allora il training set deve essere composto dall'unione di **Release 1, 2 e 3**. In questo modo il modello usa solo ciò che, realisticamente, era già noto prima della Release 4.

### 9.6.4 Importanza dell'ordine temporale

Nel contesto della defect prediction, ignorare il tempo significa violare il realismo sperimentale. Un metodo che usa informazioni provenienti dal futuro produce stime artificialmente ottimistiche.

### Nota del Prof

Questo è esattamente il problema della variante **Total** discussa a lezione: usare bug futuri per etichettare il passato significa dare al modello un vantaggio che in un vero scenario di predizione non avrebbe mai.

## 9.7 Performance metrics

### 9.7.1 Binary classifier

Un **binary classifier** è un modello che assegna ogni istanza a una di due classi, per esempio **Buggy** oppure **Non-Buggy**.

### 9.7.2 Confusion matrix e metriche base

Le quattro quantità fondamentali della **confusion matrix** sono:

- **TP (True Positive)**: predetto positivo, realmente positivo;
- **FP (False Positive)**: predetto positivo, realmente negativo;
- **TN (True Negative)**: predetto negativo, realmente negativo;
- **FN (False Negative)**: predetto negativo, realmente positivo.

### Nota del Prof

Un **False Positive** significa sprecare effort di testing su una classe che in realtà è sana.

### Nota del Prof

Un **False Negative** è molto più pericoloso: la classe è davvero difettosa, ma il modello la considera pulita, quindi rischia di arrivare in produzione senza essere testata adeguatamente.

### 9.7.3 Precision

La **Precision** è definita come:

$$\frac{TP}{TP + FP}$$

e misura quanto siano affidabili le predizioni positive del modello.

### Nota del Prof

È la logica di “al lupo al lupo”: se il modello segnala bug ovunque, finirà inevitabilmente con l’avere una Precision bassa.

### 9.7.4 Recall

La **Recall** è definita come:

$$\frac{TP}{TP + FN}$$

e misura quanti dei positivi reali vengono effettivamente recuperati dal classificatore.

### 9.7.5 Accuracy

L'**Accuracy** è:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

ma, da sola, può essere fuorviante in presenza di dataset molto sbilanciati.

### 9.7.6 Kappa

Il **Kappa** misura quanto il classificatore faccia meglio di un classificatore casuale o maggioritario. Vale:

- 1 in caso di classificazione perfetta;
- 0 se la performance equivale al caso;
- valori negativi se il classificatore è peggiore del caso.

#### Nota del Prof

Nel contesto del corso, un valore di Kappa sufficientemente alto suggerisce che ci sia davvero una relazione empirica non banale tra metriche software e difetti.

### 9.7.7 Dipendenza dal contesto applicativo

#### Solo nelle slide (non spiegato a voce)

Le metriche non vanno mai interpretate in isolamento. Per esempio, una Recall molto alta può essere banale da ottenere in dataset fortemente sbilanciati. Bisogna quindi sempre valutare le metriche nel contesto applicativo corretto.

## 9.8 ROC curves e AUC

### 9.8.1 Threshold dependence

Metriche come **Precision**, **Recall** e **F1** dipendono dalla soglia usata per trasformare una probabilità in una decisione binaria.

### 9.8.2 ROC curve

La **ROC curve** rappresenta il compromesso tra:

- **True Positive Rate (TPR)**;
- **False Positive Rate (FPR)**.

Ogni punto della curva corrisponde a una soglia diversa.

### 9.8.3 AUC

L'**AUC** è l'area sotto la ROC curve e misura, in termini probabilistici, la capacità del modello di assegnare un punteggio più alto a un positivo che a un negativo.

#### IMPORTANTE PER L'ESAME

Il limite principale dell'AUC, sottolineato a lezione, è che media le prestazioni su **tutte** le soglie possibili, comprese soglie che nel contesto reale potrebbero non avere alcun senso pratico. Per questo motivo, pur essendo utile, non sempre è la metrica più significativa dal punto di vista ingegneristico.

## 9.9 Collegamento tra i due PDF

Questa parte del materiale mette in relazione due livelli diversi dello stesso problema:

- da un lato il **labeling dei difetti**, cioè la costruzione del dataset;
- dall'altro la **valutazione dei modelli**, cioè il modo in cui giudichiamo la bontà di un classificatore.

### 9.9.1 Come il labeling influenza la qualità del dataset e del modello

Prima di poter valutare un modello, bisogna costruire un dataset con una ground truth il più possibile affidabile. Se il metodo di labeling è inaccurato, allora il classificatore verrà addestrato su esempi etichettati male e imparerà pattern sbagliati.

#### Nota del Prof

Questo è un caso classico di *Garbage In, Garbage Out*. Se SZZ sporca il dataset andando troppo indietro nel tempo o etichettando male le classi, il modello imparerà associazioni scorrette e fallirà anche se la fase di validazione è formalmente ben eseguita.

### 9.9.2 Realismo temporale e defect prediction

L'idea di **realismo temporale** accomuna sia i metodi di labeling sia le tecniche di validazione. Usare solo difetti passati per stimare  $P$ , come in **Proportion\_Increment**, e usare solo release passate per addestrare il modello, come nel **walk-forward**, riflette lo stesso principio: il modello non deve mai leggere il futuro.

### 9.9.3 Scegliere bene metriche e validazione

Se lo scopo è supportare decisioni reali su testing e code review, allora servono:

- un **dataset ben etichettato**;
- una **validazione coerente con il tempo**;
- metriche interpretabili, come **F1** e **Kappa**, da leggere insieme e non separatamente.

Un metodo di labeling migliore, come **Proportion**, rende anche più affidabile l'identificazione delle feature importanti, cioè delle vere cause del rischio di difettosità.

## 9.10 Conclusioni finali

I risultati del paper su **Proportion** e i concetti sulle metriche di valutazione convergono verso una stessa conclusione metodologica: nel Machine Learning applicato alla Software Engineering non basta scegliere un buon classificatore; occorre soprattutto essere rigorosi nella **costruzione del dataset** e nella **validazione del modello**.

### 9.10.1 Risultati del paper Proportion e limiti dell'approccio Jira-based

Su circa **125.000 difetti**, il lavoro mostra che affidarsi soltanto alle **Affected Versions** presenti in Jira è impossibile in circa il **51%** dei casi, a causa di AV mancanti o incoerenti.

### 9.10.2 Confronto Proportion vs SZZ

Nei dataset costruiti a livello di classi, i metodi **Proportion** risultano mediamente più accurati di **SZZ** in **Precision**, **F1**, **MCC** e **Kappa**. **SZZ** mantiene una Recall più alta, ma questo dipende dal fatto che tende a etichettare come positive molte più classi, pagando però un prezzo elevato in termini di falsi positivi.

### 9.10.3 Conclusioni su validazione e metriche

Nessuna metrica da sola è sufficiente. Per giudicare davvero la bontà di un modello di defect prediction bisogna:

- usare un labeling credibile;
- rispettare il tempo con tecniche come **ordered holdout** o **walk-forward**;
- interpretare le metriche in modo congiunto, tenendo conto del contesto applicativo.

Il messaggio finale è quindi che **dataset quality**, **realismo temporale** e **valutazione corretta** sono tre aspetti inseparabili dello stesso problema.

# Capitolo 10

## Snoring: rumore nei dataset di defect prediction

### 10.1 Introduzione e contesto

La **defect prediction** mira a identificare gli artefatti software, come classi o file, che hanno un'alta probabilità di presentare un difetto. Il suo obiettivo principale è ridurre i costi di testing e code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più rischiose del codice.

Il contesto dello studio è la **within-project defect prediction**, cioè la classificazione delle classi difettose all'interno di specifiche release di uno stesso progetto.

L'affidabilità di un modello predittivo dipende direttamente dalla qualità del dataset. Se il dataset contiene rumore, anche la valutazione e l'addestramento dei classificatori possono risultare distorti.

#### Nota del Prof

Il concetto centrale è il principio **Garbage In, Garbage Out**: se le etichette del dataset sono sbagliate, anche il modello più sofisticato apprenderà informazioni distorte.

#### Solo nelle slide (non spiegato a voce)

Le slide ricordano che lavori precedenti avevano già individuato altre fonti di rumore nei dataset di defect prediction, come la misclassification e l'origine errata dei difetti.

### 10.2 Sleeping defects e snoring classes

Uno **sleeping defect**, o **dormant bug**, è un difetto che viene scoperto o corretto diverse release dopo la sua introduzione effettiva nel codice. Il difetto è quindi già presente nel sistema, ma non è ancora visibile al costruttore del dataset.

Il ciclo di vita di un difetto può essere descritto tramite tre eventi principali:

- **I, Injected**: il difetto viene introdotto nel codice;
- **N oppure OV, Noticing/Opening Version**: il difetto viene osservato o viene aperto il ticket;

- **F oppure FV, Fixed Version:** il difetto viene corretto tramite fix commit.

Le **snoring classes** sono classi affette esclusivamente da difetti non ancora corretti. Poiché l'esistenza del difetto non è conoscibile prima del fix, queste classi vengono erroneamente considerate non difettose.

#### Nota del Prof

La metafora è utile: il difetto *dorme*, mentre la classe  *russa*, perché produce rumore nel dataset. Una classe russa solo se tutti i difetti che la colpiscono sono ancora dormienti. Se contiene almeno un difetto già noto e corretto, viene comunque etichettata come difettosa.

#### IMPORTANTE PER L'ESAME

Lo **snoring** introduce nei dataset **falsi negativi**, non falsi positivi. Una classe etichettata come difettosa è supportata da un fix osservato; una classe etichettata come non difettosa, invece, potrebbe contenere un difetto non ancora scoperto.

#### Solo nelle slide (non spiegato a voce)

Le slide mostrano un esempio con tre classi, C1, C2 e C3, osservate su più release. Una classe può essere realmente difettosa in una release intermedia, ma risultare *not defective* se il relativo fix avviene solo in una release successiva.

## 10.3 Assegnazione dei difetti alle classi

Per stabilire da quando una classe deve essere considerata difettosa, il processo può usare due fonti principali:

1. la prima **Affected Version** riportata nel sistema di issue tracking, ad esempio Jira;
2. un algoritmo di ricostruzione storica, come **SZZ**, quando l'Affected Version non è disponibile o non è affidabile.

Le nozioni principali sono:

- **IV, Injected Version:** release in cui il difetto viene introdotto;
- **OV, Opening Version:** release in cui viene aperto il ticket;
- **FV, Fixed Version:** release in cui il difetto viene corretto;
- **AV, Affected Version:** informazione che indica le versioni affette dal difetto.

#### Nota del Prof

Una classe può essere realmente difettosa ma apparire non buggy se, nello snapshot analizzato, il difetto è già presente ma il relativo ticket o fix sarà osservato solo in futuro. Questo è il punto in cui nasce il falso negativo.

## 10.4 SZZ: funzionamento e limiti

**SZZ** è un processo algoritmico usato per identificare quando un difetto è stato introdotto in un progetto software.

A partire da un **bug-fix commit**, SZZ analizza le linee modificate dalla correzione e usa meccanismi come `git blame` per risalire all'ultima modifica precedente di quelle stesse linee. Tale modifica viene interpretata come possibile **bug-introducing change**.

### Solo nelle slide (non spiegato a voce)

Nelle slide è mostrato un esempio in cui un bug di tipo “Array index out of range” viene corretto modificando la condizione di un ciclo da `i<=4` a `i<4`. SZZ parte dal fix e risale alla versione precedente in cui la riga errata era stata introdotta.

SZZ presenta diversi limiti:

- può fallire quando il fix consiste in una **code addition**;
- è sensibile ai **refactoring**;
- può produrre una **backward search** imprecisa;
- non gestisce perfettamente i **regressive bugs**;
- non elimina il problema dello **snoring**, perché i difetti non ancora corretti restano invisibili.

### Solo nelle slide (non spiegato a voce)

Le slide riportano che solo una parte dei bug-introducing changes può essere individuata tramite l'assunzione classica di SZZ, secondo cui il bug è stato introdotto nelle stesse linee poi modificate dal fix.

## 10.5 Research Questions dello studio

Lo studio sullo snoring si articola in cinque research questions:

1. **RQ1: To what extent do defects sleep?**
2. **RQ2: To what extent do classes snore?**
3. **RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?**
4. **RQ4: To what extent does snoring impact defect prediction accuracy?**
5. **RQ5: To what extent is no data better than snoring data?**

Le prime due domande misurano l'entità del fenomeno, mentre le ultime tre analizzano l'impatto dello snoring sulla valutazione e sull'addestramento dei classificatori.

## 10.6 RQ1: To what extent do defects sleep?

### 10.6.1 Rationale

La prima research question misura quante release trascorrono tra l'introduzione di un difetto e la sua correzione. L'obiettivo è quantificare quanto a lungo un difetto possa restare invisibile nel progetto.

### 10.6.2 Design

Sono stati selezionati **20 grandi progetti Apache**, con issue tracking in Jira e codice versionato in Git. I progetti considerati hanno almeno 10 release e un buon collegamento tra bug e codice.

### 10.6.3 Variabili

- **Variabile indipendente:** il bug o ticket in Jira.
- **Variabile dipendente:** numero di release per cui il bug ha dormito, misurato tra l'introduzione e il fix.

### 10.6.4 Risultati e discussione

Il risultato principale è che il bug medio dorme per circa **7 release**. Inoltre, nella maggior parte dei progetti, i difetti dormono per oltre il **19%** delle release esistenti.

**Solo nelle slide (non spiegato a voce)**

Le slide riportano casi estremi come **LUCENE-3672**, dormiente per 84 release, e **STRATOS-1653**, dormiente per 51 release su 53.

## 10.7 RQ2: To what extent do classes snore?

### 10.7.1 Rationale

Non ogni sleeping defect produce necessariamente una snoring class. Se una classe contiene anche un altro difetto già noto, la classe viene comunque etichettata come difettosa e non genera falso negativo. La RQ2 misura quindi quanto i difetti dormienti si traducano effettivamente in classi rumorose.

### 10.7.2 Design

Lo studio osserva come cambia la difettosità stimata man mano che l'osservatore avanza nel tempo. Nelle slide questo è rappresentato con i **binocoli**: più ci si sposta avanti nella storia del progetto, più difetti prima invisibili diventano osservabili.

### 10.7.3 Variabili

- **Variabile indipendente:** **Progress**, cioè il rapporto tra la release osservata e il numero totale di release.
- **Variabile dipendente:** **Missing Rate**, cioè:

$$\frac{FN}{FN + TP}$$

Il Missing Rate rappresenta la quota di classi realmente difettose che non vengono riconosciute come tali. Corrisponde a  $1 - Recall$ .

### 10.7.4 Risultati e discussione

Il Missing Rate diminuisce con il progresso temporale, ma resta significativo per molte release. Per la maggior parte dei progetti, il Missing Rate rimane superiore al **50%** se non si rimuove più del **20%** delle release finali.

#### Nota del Prof

Per ottenere un Missing Rate nullo bisogna in media ignorare una porzione molto ampia delle release finali. Questo mostra che la defectiveness osservata si stabilizza solo molto tempo dopo l'introduzione reale dei difetti.

## 10.8 RQ3: Impatto sulla valutazione dei classificatori

### 10.8.1 Rationale

La terza research question analizza se lo snoring alteri la valutazione dei classificatori. Se i dati di test sono rumorosi, si rischia di scegliere come migliore un classificatore che in realtà non lo è.

### 10.8.2 Design

Sono stati valutati **15 classificatori** tramite **ordered holdout**, mantenendo l'ordine cronologico dei dati: 66% per il training e 33% per il testing.

### 10.8.3 Metriche

Le metriche considerate includono:

- Precision;
- Recall;
- F1;
- Cohen's Kappa;
- AUC;

- Matthews Correlation Coefficient.

Viene inoltre calcolato il **Relative Bias**, cioè la distanza relativa tra accuratezza misurata su dati rumorosi e accuratezza misurata su dati privi di snoring.

## 10.8.4 Risultati e discussione

Lo snoring introduce un bias elevato nella valutazione. In particolare, usando dati affetti da snoring, il classificatore realmente migliore viene selezionato correttamente solo in una parte dei casi.

### Solo nelle slide (non spiegato a voce)

Le slide riportano che il classificatore migliore viene selezionato correttamente solo nel **45%** dei casi quando la valutazione è eseguita su dati affetti da snoring.

## 10.9 RQ4: Impatto sull'accuratezza predittiva

### 10.9.1 Rationale

La quarta research question valuta quanto lo snoring peggiori l'accuratezza predittiva reale dei modelli.

### 10.9.2 Design

I modelli vengono addestrati su dati affetti da snoring e poi valutati su un test set pulito. In questo modo si misura l'effetto del rumore presente nel training set.

### 10.9.3 Risultati e discussione

Lo snoring degrada le performance di tutti i classificatori e di tutte le metriche osservate. Le metriche più colpite sono **Recall**, **F1**, **Kappa** e **Matthews**. La **Precision** risulta meno sensibile.

### IMPORTANTE PER L'ESAME

Lo snoring danneggia soprattutto la **Recall**, perché introduce falsi negativi nel training set. Il modello impara quindi a classificare come sane molte classi che in realtà sono difettose.

## 10.10 RQ5: Quando no data è meglio di snoring data

### 10.10.1 Rationale

La quinta research question valuta se sia preferibile rimuovere alcune release recenti per ridurre il rumore, anche al costo di avere meno dati.

## 10.10.2 Design

Lo studio confronta scenari in cui vengono rimosse le ultime **1, 2, 3 o 4 release** dal training set.

## 10.10.3 Risultati e discussione

Rimuovere una release tende a migliorare la qualità della prediction. Tuttavia, rimuovere troppe release peggiora le performance perché riduce eccessivamente la dimensione del dataset.

### Nota del Prof

Il risultato mostra un trade-off tra **Garbage In, Garbage Out** e **dimensione del dataset**. Pochi dati puliti possono essere meglio di molti dati rumorosi, ma eliminare troppe release fa perdere informazione utile.

## 10.11 Conclusioni

Lo snoring è una fonte rilevante di rumore nei dataset di defect prediction. Il fenomeno nasce dal fatto che molti difetti restano dormienti per diverse release e diventano osservabili solo dopo il fix.

I risultati principali sono:

- molti difetti dormono per una porzione significativa della vita del progetto;
- le snoring classes introducono falsi negativi;
- la valutazione dei classificatori può diventare fortemente biasata;
- la Recall è particolarmente danneggiata;
- rimuovere alcune release recenti può migliorare la qualità del training set, ma rimuoverne troppe peggiora il modello.

## 10.12 Future works

Le possibili estensioni includono:

- tecniche più fine-grained per identificare e rimuovere il rumore;
- combinazione di più strategie di mitigazione;
- uso di etichette probabilistiche invece di etichette binarie rigide;
- replica dello studio nel contesto della **Just-In-Time defect prediction**.

## 10.13 Da ricordare

### IMPORTANTE PER L'ESAME

Lo snoring introduce falsi negativi: classi realmente difettose possono essere etichettate come non difettose perché il fix avverrà solo in futuro.

### IMPORTANTE PER L'ESAME

La metrica più penalizzata è la Recall, perché il modello impara a non riconoscere molti veri positivi.

### IMPORTANTE PER L'ESAME

Rimuovere alcune release recenti può ridurre il rumore, ma eliminare troppe release riduce eccessivamente la quantità di dati disponibili.

# Capitolo 11

## Effort-Aware Accuracy Metrics e valutazione dei classificatori

### 11.1 Contesto generale

Nel campo dell'**ingegneria del software**, i modelli di **defect prediction** sono classificatori di Machine Learning con lo scopo di identificare gli artefatti software, come classi o metodi, che hanno un'alta probabilità di manifestare un difetto.

#### 11.1.1 Defect prediction e testing activity context

Il fine ultimo della **defect prediction** è ridurre il costo delle attività di testing, analisi o *code review*. Prevedendo dove si annidano i bug, il team di sviluppo può dare priorità agli sforzi, concentrandoli su porzioni specifiche di codice, invece di ispezionare l'intero sistema alla cieca.

#### 11.1.2 Perché l'accuratezza classica non basta

Storicamente, i modelli predittivi venivano valutati tramite metriche classiche derivate dal mondo dell'**Information Retrieval**, come **Precision** o **Recall**.

##### Nota del Prof

Il professore sottolinea che, sebbene le metriche classiche siano utili a livello teorico, risultano inadeguate e poco pratiche nel contesto del testing. Sapere che un classificatore ha una Precision dello 0.90 o una Recall dello 0.89 non fornisce al manager alcuna indicazione pratica sull'impatto reale in termini di *effort* necessario per ispezionare il software. Per questo motivo, le metriche classiche risultano troppo astratte.

#### 11.1.3 Effort-aware accuracy metrics

Per colmare questo divario, sono nate le **Effort-Aware Accuracy Metrics (EAM)**. Si tratta di metriche di accuratezza che tengono esplicitamente in considerazione l'**effort**, cioè il costo o il tempo richiesto per ispezionare il codice suggerito dal classificatore, garantendo così una stima più realistica dei benefici apportati.

## 11.2 Concetti fondamentali

### 11.2.1 Definizione di EAM

Le **Effort-Aware Metrics** misurano i benefici forniti da un classificatore in base alla sua capacità di ordinare correttamente le entità candidate a essere difettose, bilanciando i bug trovati con lo sforzo speso per cercarli.

### 11.2.2 Ranking delle entità difettose

Poiché è impossibile analizzare tutto il software, file e classi vengono inseriti in un **ranking**, cioè una lista ordinata. Il tester ispezionerà il codice partendo dalla cima della lista e scendendo verso il basso finché non esaurisce le risorse a disposizione.

### 11.2.3 Relazione tra effort e LOC da ispezionare

#### Nota del Prof

Nell'approccio EAM si assume, come approssimazione, che l'effort necessario per testare un'entità sia linearmente proporzionale alla sua dimensione fisica, cioè al numero di **Linee di Codice (LOC)**. Ispezionare una classe enorme costerà quindi molto più effort rispetto a una classe molto piccola.

### 11.2.4 Differenza tra ranking per probabilità e ranking effort-aware

L'approccio classico ordina semplicemente le classi in base alla **probabilità** di contenere un bug. Un approccio realmente **effort-aware**, invece, ordina le classi tenendo conto anche di quanto *costa* ispezionarle in termini di LOC.

## 11.3 PofBx e NPofBx

### 11.3.1 Definizione di PofBx

La **PofBx** è la EAM non normalizzata più nota. Definisce la percentuale di bug che un tester può identificare ispezionando il top  $x\%$  delle linee di codice del sistema, seguendo un ranking basato esclusivamente sulla **probabilità predetta** di difetto.

### 11.3.2 Definizione di NPofBx

La **NPofBx** è la metrica proposta dagli autori del paper. Rappresenta la versione **normalizzata** della PofBx. Mantiene la stessa logica generale, ma utilizza un ranking differente.

### 11.3.3 Significato della normalizzazione

Invece di ordinare semplicemente per probabilità predetta, la NPofBx usa un ranking basato sulla **predicted defect density**, cioè una densità predetta dei difetti.

### 11.3.4 Ranking per predicted probability / size

La densità viene calcolata dividendo la probabilità predetta di un'entità per la sua dimensione:

$$\frac{\text{predicted probability}}{\text{size}}$$

### 11.3.5 Differenza pratica tra i due approcci

#### Nota del Prof

Senza normalizzazione, il modello potrebbe suggerire di testare per prima una classe enorme con probabilità del 90% di contenere un bug, esaurendo subito il budget di LOC analizzabili. Con la normalizzazione, il modello capisce che è più conveniente testare per prime diverse classi piccole ma promettenti, trovando molti più bug a parità di LOC lette.

## 11.4 Esempio introduttivo

Per chiarire l'effetto della normalizzazione, le slide e l'audio presentano un esempio pratico.

### 11.4.1 Dataset di esempio

Si considera un sistema di **1790 LOC**, diviso in **7 entità**, di cui **3** effettivamente difettose.

### 11.4.2 Significato di PofB20

Si assume di poter utilizzare il 20% del budget di ispezione. Il 20% di 1790 LOC corrisponde a **358 LOC**. Ordinando le 7 entità per semplice **probabilità predetta**, le prime due entità consumano tutto il budget disponibile. Analizzandole, si trova solo **1** delle **3** entità difettose. Di conseguenza:

$$PofB20 = 33\%$$

### 11.4.3 Significato di NPofB20

Applicando la normalizzazione, cioè ordinando le entità per Probabilità/Size, le classi piccole e promettenti risalgono la graduatoria. Con lo stesso tetto di 358 LOC, si riescono ad analizzare **3** entità anziché 2, trovando **2** delle **3** entità difettose. Di conseguenza:

$$NPofB20 = 66\%$$

### 11.4.4 Perché la normalizzazione migliora il numero di difetti trovati

Semplicemente cambiando il criterio di ordinamento del ranking, senza alterare i dati originali né il classificatore, il numero di bug trovati **raddoppia**.

## 11.5 Aim del lavoro e Research Questions

### 11.5.1 Problemi nell'uso delle EAM

Il lavoro nasce dall'osservazione che esiste una grave carenza, e talvolta un uso scorretto, delle EAM in letteratura, il che ne mina la validità e la generalizzazione.

### 11.5.2 Proposta di normalizzazione

L'obiettivo del lavoro è duplice:

1. evidenziare le problematiche storiche nell'uso delle EAM;
2. proporre e valutare la validità della metrica normalizzata **NPofB**, basata sulla **predicted defect density**.

### 11.5.3 Research Questions

Le quattro domande di ricerca sono:

- **RQ1**: quali EAM vengono usate nei paper di Ingegneria del Software?
- **RQ2**: la normalizzazione migliora effettivamente i valori di PofBs?
- **RQ3**: il ranking dei classificatori cambia se si normalizza la PofB?
- **RQ4**: il ranking dei classificatori cambia al variare della  $x$  nelle NPofBx normalizzate?

## 11.6 RQ1

Per rispondere alla prima domanda, gli autori hanno effettuato uno **Systematic Mapping Study**.

### 11.6.1 Numero di studi, progetti, EAM e classifier

Sono stati analizzati **152 primary studies** provenienti dai principali journal del settore. Lo studio finale è stato validato su **14 progetti** — 12 open-source e 2 industriali — confrontando **10** diverse soglie di EAM e usando **10 classificatori**.

### 11.6.2 Risultati principali

I risultati mostrano che:

- la grande maggioranza della letteratura ignora le EAM: **132 studi su 152** usano **0 EAM**, valutando i modelli solo tramite Precision o Recall;
- i pochi studi che usano EAM si limitano quasi sempre a **una sola metrica**, tipicamente **PofB20**, riducendo la generalizzabilità dei risultati;
- sette studi su venti presentavano un **mismatch nei nomi delle metriche**, sintomo di scarsa comprensione o assenza di standardizzazione.

## 11.7 RQ2

### 11.7.1 Intuizione di base

La RQ2 indaga se la **NPofB** ottenga performance matematicamente migliori rispetto alla **PofB**.

### 11.7.2 Variabili indipendente e dipendente

#### Nota del Prof

Il professore richiama una nozione cruciale di ingegneria degli esperimenti. In questo studio, la **variabile indipendente** è la presenza o assenza della normalizzazione, mentre la **variabile dipendente** è lo score della PofB calcolato con o senza normalizzazione.

### 11.7.3 PofB10–PofB90 e average

La valutazione è stata eseguita su un intero spettro di soglie, dal **10%** al **90%** a passi di 10, escludendo 0 e 100 perché banali, più una metrica media, **AveragePofB**. In totale vengono considerate **10 PofBs**.

### 11.7.4 Preprocessing

Prima dell'esperimento, i dati sono stati preprocessati tramite:

1. **Log10 normalization**;
2. **Feature subset selection**;
3. **SMOTE**.

#### Nota del Prof

La trasformazione in  $\log_{10}$  è fondamentale. Se si usano feature disomogenee, per esempio secondi, numero di bug o LOC, passare al logaritmo base 10 rende le distribuzioni più omogenee e aumenta l'accuratezza di base.

### 11.7.5 Walk-forward

#### Nota del Prof

Il **walk-forward** è vitale nell'ingegneria del software. Si dividono i dati in release sequenziali: per predire la Release 4 si usa come training set l'unione delle release 1, 2 e 3. Questo impedisce al modello di leggere il futuro e preserva il realismo temporale.

### 11.7.6 Classifier usati

Solo nelle slide (non spiegato a voce)

Sono stati usati 10 algoritmi ML ben noti: **Decision Table**, **IBk (k-NN)**, **J48**, **KStar**, **Naive Bayes**, **SMO**, **Random Forest**, **Logistic Regression**, **BayesNet** e **Bagging**.

### 11.7.7 Risultati

La normalizzazione incrementa i valori della PofB in modo statisticamente significativo e di svariati ordini di grandezza.

### 11.7.8 Relative gain

Il guadagno relativo viene calcolato come:

$$\frac{NPofB - PofB}{PofB}$$

Il **relative gain** cresce fortemente per soglie più basse: per esempio, il guadagno della **PofB10** è circa doppio rispetto a quello della **PofB20**.

### 11.7.9 Cliff's delta e p-value

#### IMPORTANTE PER L'ESAME

Il professore insiste molto sulla differenza tra **p-value** ed **effect size**. Il **p-value** indica quanto sia probabile commettere errore nel rigettare l'ipotesi nulla, ma *non* misura la grandezza dell'effetto. Il **Cliff's delta**, invece, misura proprio quanto l'effetto sia grande e importante.

#### Nota del Prof

Nel caso della normalizzazione, i valori di **Cliff's delta** risultano per lo più classificati come **Large**, cioè maggiori o uguali a 0.43, mostrando che l'effetto non è solo statisticamente significativo, ma anche sostanzialmente rilevante.

## 11.8 RQ3

### 11.8.1 Spearman rank correlation

La RQ3 si chiede se, normalizzando la PofB, il classificatore che prima risultava il migliore resti tale oppure se la graduatoria venga stravolta. Per confrontare i ranking si usa il coefficiente di correlazione di **Spearman**.

### 11.8.2 Confronto ranking con e senza normalizzazione

Il confronto viene fatto tra le classifiche dei classificatori ottenute con la stessa PofB, ma calcolata **con** e **senza** normalizzazione.

### 11.8.3 Best classifier

L'analisi si concentra anche su un aspetto pratico fondamentale: verificare se il **miglior classificatore** resti lo stesso prima e dopo la normalizzazione.

### 11.8.4 Risultati e interpretazione pratica

I risultati mostrano una correlazione solo **modesta** nella maggior parte dei casi. In pratica, identificare il miglior classificatore usando la vecchia PofB equivale spesso a scegliere il classificatore sbagliato rispetto alla più realistica NPofB.

Questo implica che molti studi passati basati sulla PofB non normalizzata risultano fuorvianti, perché potrebbero aver promosso come *miglior modello* un algoritmo che, in realtà, non fornisce il massimo beneficio effort-aware al tester.

## 11.9 RQ4

### 11.9.1 Confronto tra diverse NPofBx

Una volta stabilito che la normalizzazione è opportuna, la RQ4 si chiede se basti usare una sola metrica, per esempio **NPofB20**, oppure se il comportamento dei classificatori cambi al variare della soglia  $x$ .

### 11.9.2 Ruolo di $x$ da 10 a 90

L'analisi considera soglie che vanno dal **10%** al **90%**, oltre alla metrica media **Average NPofB**.

### 11.9.3 Average NPofB e confronto dei ranking

Le diverse classifiche vengono confrontate a coppie tramite **Spearman**, valutando anche il comportamento dell'**Average NPofB**.

### 11.9.4 Risultati e best classifier

I risultati dimostrano che i ranking dei classificatori sono molto lontani dall'essere identici al variare della soglia  $x$ . In circa l'**88%** dei casi, il classificatore migliore cambia se si cambia la quantità di codice da ispezionare.

In altre parole, il modello ottimale per ispezionare il **10%** del codice potrebbe non essere più il migliore se l'obiettivo è ispezionarne il **50%**.

## 11.10 Figure, tabelle e grafici

Solo nelle slide (non spiegato a voce)

Le tabelle e i grafici descritti qui sono presentati nelle slide come supporto visivo dei risultati, anche quando non vengono spiegati in dettaglio a voce.

### 11.10.1 Esempio PofB20 e NPofB20

Le tabelle iniziali del paper illustrano visivamente l'esempio dei 7 file, mostrando il passaggio da **33%** a **66%** di bug trovati grazie alla normalizzazione.

### 11.10.2 Tabella delle EAM usate negli studi precedenti

La **Tabella 4.4** mostra che, su **152** studi primari, **132** usano **0 EAM**. Nessun paper in letteratura ha usato **5 o più EAM**.

### 11.10.3 Tabella dei relative gains

La **Tabella 4.6** mostra il forte guadagno dovuto alla normalizzazione:

- +414% per **PofB10**;
- guadagni progressivamente decrescenti fino a +11% per **PofB90**;
- tutti i confronti hanno **p-value < 0.0001**.

### 11.10.4 Tabella degli equivalent best classifiers

La **Tabella 4.8** e il boxplot associato mostrano la proporzione di casi in cui il miglior modello resta lo stesso. Per esempio, si passa dal **15%** per **NPofB10** al **29%** per **NPofB90**. Questo dato è importante perché smonta l'idea che testare una sola EAM sia statisticamente e industrialmente corretto.

## 11.11 Main results e conclusioni

### 11.11.1 Superiorità delle NPofBs rispetto alle PofBs

Dal punto di vista statistico e dell'ordine di grandezza, la **NPofB** risulta nettamente superiore alla **PofB**. Fornisce valutazioni più realistiche perché allinea le stime matematiche al comportamento effettivo di uno sviluppatore, che nella pratica considera sempre anche il peso delle dimensioni del file.

### 11.11.2 Cambiamento del best classifier dopo la normalizzazione

L'impatto più forte della normalizzazione è che essa **ribalta il ranking dei classificatori**. Modelli ritenuti ottimi nella letteratura precedente perdono il primato quando si passa alla valutazione normalizzata.

### 11.11.3 Implicazioni pratiche

Chiunque valuti un classificatore in ingegneria del software dovrebbe:

- calcolare metriche **Effort-Aware normalizzate**, cioè **NPofB**;
- testare il classificatore su **molteplici soglie  $x$** , e non su una sola;
- evitare conclusioni basate esclusivamente su una singola metrica o su una sola soglia.

## 11.12 ACUME tool

### 11.12.1 Scopo

L'assenza di metriche EAM in letteratura è spesso dovuta alla mancanza di librerie standard per calcolarle. Gli autori propongono quindi **ACUME** (**ACcU**racy **ME**trics), un tool open-source in Python per automatizzare il processo.

### 11.12.2 Input e output

Il tool agisce alla fine della pipeline. Prende in input un file **CSV** predetto da strumenti di Machine Learning, come Weka o scikit-learn, con quattro colonne essenziali:

- **ID**
- **Size**
- **Predicted Probability**
- **Actual Defectiveness**

In output produce un singolo file CSV contenente:

- le metriche classiche di accuratezza;
- tutte le EAM discusse nel lavoro.

### 11.12.3 Utilità per i ricercatori

ACUME evita sprechi di tempo, riduce il rischio di mismatch nei nomi delle metriche, aumenta la generalizzazione dei risultati e migliora la riproducibilità degli esperimenti.

### In sintesi

Le metriche classiche, come Precision e Recall, risultano troppo astratte nel mondo reale del testing software. Questo studio mostra che pesare il ranking dividendo la probabilità per la grandezza del file, cioè usando **NPofB**, aumenta drasticamente la capacità di ritrovare i bug veri. Il risultato è un cambiamento profondo nelle modalità con cui l'accademia e i professionisti dovrebbero valutare i classificatori: mai più con una sola soglia e mai più senza normalizzazione.

# Capitolo 12

## Milestone 1: Dataset Creation

### 12.1 Obiettivo della Milestone 1

La **Milestone 1** richiede la costruzione di un dataset CSV per un progetto software assegnato. Il dataset deve essere usato per attività di **defect prediction**, cioè per predire se una classe in una specifica release sia difettosa oppure no.

L'unità di analisi è la coppia:

classe — release

Ogni riga del CSV rappresenta quindi una classe Java osservata in una determinata release del progetto.

#### IMPORTANTE PER L'ESAME

La Milestone 1 non consiste solo nell'estrarre metriche dal codice: il punto più delicato è costruire correttamente la colonna **Bugginess**, cioè stabilire se una classe deve essere etichettata come buggy o non buggy in una certa release.

### 12.2 Assegnazione del progetto

Ogni studente lavora su un progetto determinato tramite la prima lettera del cognome.

Il procedimento è il seguente:

1. si prende la prima lettera del cognome;
2. si converte la lettera nel numero corrispondente dell'alfabeto;
3. si calcola:

$$X = \text{numero} \bmod 6$$

4. si assegna il progetto secondo la seguente mappatura:

- $X = 0$ : AVRO;
- $X = 1$ : OPENJPA;
- $X = 2$ : STORM;
- $X = 3$ : ZOOKEEPER;

- $X = 4$ : SYNCOPÉ;
- $X = 5$ : TAJÓ.

## 12.3 Struttura del dataset CSV

Il dataset finale deve essere una tabella in formato CSV contenente, per ogni classe e release, le informazioni necessarie alla defect prediction.

Le colonne richieste sono:

- **Name of the project**: nome del progetto;
- **Name of the class**: path o nome della classe Java;
- **Release ID**: identificativo della release;
- **Feature**: circa 20 metriche descrittive della classe o del processo di sviluppo;
- **NSmells**: numero di code smells rilevati;
- **Bugginess**: etichetta finale, con valore **yes** o **no**.

### Nota del Prof

Il dataset deve essere piatto, cioè organizzato come una tabella classica: ogni riga è un'istanza, ogni colonna è una feature o una label. Questa struttura è necessaria per poter usare successivamente strumenti di machine learning.

## 12.4 Selezione delle release

Il workflow prevede di individuare tutte le release del progetto e le relative date. Successivamente bisogna ignorare l'ultimo **66%** delle release.

Di conseguenza, il dataset finale contiene solo il primo **34%** delle release più antiche.

### IMPORTANTE PER L'ESAME

Ignorare l'ultimo 66% delle release significa non inserirle come righe nel CSV finale. Tuttavia, i ticket e i commit successivi possono ancora essere necessari per capire se una classe appartenente al primo 34% era già difettosa.

## 12.5 Snapshot di release

Per ogni release selezionata è necessario trovare l'ultimo commit associato a quella release. A quel punto si effettua il checkout dell'intero codice alla revisione corrispondente.

Questo passaggio serve a costruire un inventario realistico delle classi Java effettivamente presenti nella release.

### Nota del Prof

Una classe può essere etichettata come buggy in una release solo se esiste realmente nello snapshot di quella release. Non basta sapere che quella classe sarà toccata da un fix in futuro.

## 12.6 Classi da considerare

Per ogni release selezionata bisogna individuare le classi Java di produzione presenti nello snapshot.

### IMPORTANTE PER L'ESAME

Devono essere escluse le **test classes**. Il dataset deve considerare le classi di produzione, tipicamente sotto percorsi come `src/main/java`, evitando file sotto `src/test/java`.

## 12.7 Class metrics

Le **class metrics** descrivono la storia e le caratteristiche della classe all'interno della release considerata. Le principali metriche richieste sono:

- **Size (LOC)**: numero di linee di codice;
- **LOC Touched**: somma delle linee aggiunte e cancellate nelle revisioni;
- **NR**: numero di revisioni;
- **Nfix**: numero di fix di difetti;
- **Nauth**: numero di autori;
- **LOC Added**: totale delle linee aggiunte;
- **Max LOC Added**: massimo numero di linee aggiunte in una revisione;
- **Average LOC Added**: media delle linee aggiunte per revisione;
- **Churn**: somma delle linee aggiunte e cancellate;
- **Max Churn**: massimo churn osservato;
- **Average Churn**: churn medio;
- **Change Set Size**: numero di file committati insieme alla classe;
- **Max Change Set**: massimo change set osservato;
- **Average Change Set**: change set medio;
- **Age**: età della classe o della release;
- **Weighted Age**: età pesata rispetto al LOC touched.

Solo nelle slide (non spiegato a voce)

Le slide indicano che alcune metriche possono essere calcolate entro la singola release oppure a partire dalla release 0, cioè considerando tutta la storia disponibile fino alla release corrente.

## 12.8 Commit metrics

Le **commit metrics** descrivono le caratteristiche dei cambiamenti effettuati nel repository. Esse sono utili perché alcune proprietà dei commit sono correlate alla probabilità di introdurre difetti.

Le metriche principali sono:

- **NS**: numero di sottosistemi modificati;
- **ND**: numero di directory modificate;
- **NF**: numero di file modificati;
- **Entropy**: distribuzione delle modifiche tra i file;
- **LA**: linee di codice aggiunte;
- **LD**: linee di codice cancellate;
- **LT**: linee di codice nel file prima del cambiamento;
- **FIX**: indica se il cambiamento è un defect fix;
- **NDEV**: numero di sviluppatori che hanno modificato i file coinvolti;
- **AGE**: intervallo medio tra l'ultima modifica e quella corrente;
- **NUC**: numero di cambiamenti unici ai file modificati;
- **EXP**: esperienza dello sviluppatore;
- **REXP**: esperienza recente dello sviluppatore;
- **SEXP**: esperienza dello sviluppatore sul sottosistema.

### Nota del Prof

Le metriche di processo sono importanti perché non misurano soltanto il codice, ma anche il modo in cui il codice è stato modificato. File toccati spesso, da molti sviluppatori o in commit molto grandi tendono a essere più rischiosi.

## 12.9 NSmells

La feature **NSmells** rappresenta il numero di code smells rilevati nella classe. Può essere calcolata tramite strumenti di analisi statica come SonarCloud, PMD o strumenti simili.

### Nota del Prof

NSmells è una feature del dataset, non la label. Una classe con molti smells non è automaticamente buggy, ma può avere una maggiore probabilità di diventarlo.

## 12.10 Labeling della bugginess

La colonna **Bugginess** indica se una classe in una specifica release deve essere considerata difettosa.

Il processo parte assumendo che tutte le classi siano inizialmente **no**. Successivamente, per i ticket di difetto validi, alcune coppie classe-release vengono trasformate in **yes**.

I ticket da considerare sono quelli che soddisfano condizioni del tipo:

- tipo uguale a defect o bug;
- stato **Closed** oppure **Resolved**;
- resolution uguale a **Fixed**.

### IMPORTANTE PER L'ESAME

Il labeling è il passaggio più critico della Milestone 1: se una classe difettosa viene etichettata come non buggy, il classificatore apprenderà da un esempio sbagliato.

## 12.11 Uso di SZZ e Proportion

Per identificare le classi buggy è necessario collegare ticket, fix commit e classi modificate.

Una strategia possibile è:

1. individuare i ticket di difetto validi;
2. trovare i commit che correggono quei ticket;
3. identificare le classi Java toccate dai fix commit;
4. stimare le release affette dal difetto;
5. marcare come buggy le coppie classe-release comprese nell'intervallo stimato.

Quando l'Affected Version è disponibile e affidabile, può essere usata per stimare l'inizio del difetto. Quando manca o non è affidabile, si ricorre a **SZZ** o a metodi come **Proportion Total**.

### Nota del Prof

L'idea di Proportion è stimare la Injected Version quando non è nota, usando l'andamento medio del ciclo di vita dei difetti osservati nel progetto.

## 12.12 ComputedAV e intervallo di release affette

Nel progetto è utile distinguere tra:

- **AV originale:** informazione grezza proveniente da Jira;
- **ComputedAV:** insieme di release calcolate come affette dal difetto.

La ComputedAV rappresenta l'intervallo di release in cui il difetto deve essere considerato presente. Operativamente, si può interpretare come un intervallo:

$$[IV, FV)$$

cioè dalla Injected Version inclusa fino alla Fixed Version esclusa.

### IMPORTANTE PER L'ESAME

L'AV originale di Jira non deve essere interpretata automaticamente come intervallo continuo. Va validata rispetto alla timeline delle release. La ComputedAV è invece la rappresentazione operativa usata per il labeling.

## 12.13 Workflow operativo completo

Il workflow della Milestone 1 può essere sintetizzato così:

1. identificare il progetto assegnato;
2. recuperare le release e le rispettive date;
3. selezionare solo il primo 34% delle release;
4. per ogni release selezionata:
  - (a) trovare l'ultimo commit della release;
  - (b) effettuare il checkout del codice;
  - (c) individuare le classi Java di produzione;
  - (d) calcolare le metriche di classe;
  - (e) calcolare le metriche di commit;
  - (f) calcolare NSmells;
  - (g) scrivere una riga nel CSV per ogni classe.
5. recuperare i ticket di difetto validi;
6. collegare ticket e fix commit;
7. identificare le classi toccate dai fix;
8. stimare IV, OV, FV e ComputedAV;
9. assegnare la label **yes/no**;
10. produrre il CSV finale.

## 12.14 Errori comuni da evitare

- **Scartare i ticket futuri:** anche se il CSV contiene solo il primo 34% delle release, i ticket successivi possono servire per etichettare correttamente quelle release.
- **Confondere AV e ComputedAV:** l'AV di Jira è dato grezzo; la ComputedAV è l'intervallo operativo usato nel progetto.
- **Etichettare classi non presenti:** una classe deve essere marcata buggy solo se esiste nello snapshot della release.
- **Includere classi di test:** il dataset deve riguardare classi di produzione.
- **Usare date incoerenti:** IV, OV, FV, resolution date e fix date devono essere trattate con una semantica chiara.
- **Sovrascrivere informazioni grezze:** i dati originali, come AV da Jira, vanno mantenuti distinti dai dati calcolati.

### IMPORTANTE PER L'ESAME

L'errore più grave è costruire il dataset usando solo i ticket presenti entro il primo 34% delle release. Questo fa apparire come non buggy classi che saranno corrette in seguito, reintroducendo falsi negativi nel labeling.

## 12.15 Da ricordare

### IMPORTANTE PER L'ESAME

La riga del dataset rappresenta una coppia classe-release, non una classe in generale.

### IMPORTANTE PER L'ESAME

Il CSV finale contiene solo il primo 34% delle release, ma la bugginess deve essere calcolata usando tutti i ticket e i commit disponibili.

### IMPORTANTE PER L'ESAME

La label **Bugginess** deve essere coerente con la presenza reale della classe nello snapshot della release.

### IMPORTANTE PER L'ESAME

NSmells è una feature, non una label: serve al classificatore per apprendere, ma non determina da sola se una classe è buggy.

# Capitolo 13

## Balancing

### 13.1 Problema del class imbalance

Il **class imbalance** si verifica quando i possibili valori della variabile di interesse non sono distribuiti in modo equilibrato dal punto di vista della numerosità. Per esempio, se in un dataset sono presenti il 5% di classi *buggy* e il 95% di classi *non buggy*, il dataset è fortemente sbilanciato.

Nella maggior parte delle applicazioni reali, lo sbilanciamento è una caratteristica naturale del problema. L'evento che si vuole identificare, cioè la classe positiva, è spesso raro.

Solo nelle slide (non spiegato a voce)

Il class imbalance è frequente in applicazioni come **fraud detection**, **intrusion detection**, **risk management**, classificazione di testi e diagnosi o monitoraggio medico.

I classificatori standard tendono a essere dominati dalla classe maggioritaria. Poiché nel training set osservano molte più istanze della maggioranza, imparano a predire soprattutto quella classe. Il risultato è un'alta accuratezza predittiva sulla classe maggioritaria, ma una bassa capacità predittiva sulla classe minoritaria.

Un input bilanciato può aiutare il classificatore perché rende più visibili i pattern della classe minoritaria, mettendo il modello in condizione di riconoscere meglio i positivi.

### 13.2 Effetto dello sbilanciamento sui classificatori

In presenza di class imbalance, un classificatore può ottenere una **accuracy** apparentemente alta predicendo quasi sempre la classe maggioritaria. Tuttavia, questo comportamento può essere inutile rispetto all'obiettivo reale del problema.

Nota del Prof

Se lo scopo è trovare frodi o diagnosticare una malattia, un modello che predice sempre "sano" o "non frode" può ottenere un'accuracy elevata quando i positivi sono pochi, ma fallisce proprio nel compito più importante: identificare i casi rari.

Nel contesto della **defect prediction**, la classe minoritaria è spesso quella delle classi *buggy*. Il problema non è riconoscere le molte classi sane, ma individuare le poche classi realmente difettose.

Se il training set contiene pochi difetti, il modello tenderà a predire pochi difetti anche nel testing set, riducendo soprattutto la capacità di trovare veri positivi.

## 13.3 Undersampling

L'**undersampling** è una tecnica di bilanciamento che riduce il numero di istanze della classe maggioritaria, preservando tutte le istanze della classe minoritaria.

L'idea è estrarre un sottoinsieme più piccolo della maggioranza, in modo che la distribuzione tra le classi diventi più equilibrata.

### Solo nelle slide (non spiegato a voce)

Nel diagramma delle slide, un'icona a forma di forbice taglia le righe in eccesso dal blocco della *majority class*, riducendolo fino a renderlo confrontabile con il blocco della *minority class*.

Il vantaggio principale dell'undersampling è la semplicità: il dataset diventa più bilanciato e spesso anche più piccolo, riducendo il costo computazionale dell'addestramento.

Lo svantaggio principale è la **perdita di informazione**. Eliminando istanze della classe maggioritaria, si possono rimuovere esempi utili per descrivere correttamente il problema.

### Nota del Prof

L'undersampling può peggiorare le prestazioni del classificatore perché fornisce meno dati al modello. Bilanciare il dataset eliminando esempi significa anche rinunciare a una parte della conoscenza disponibile.

## 13.4 Oversampling

L'**oversampling** è una tecnica di bilanciamento che aumenta il numero di istanze della classe minoritaria.

Nella sua forma più semplice, l'oversampling duplica le istanze minoritarie già presenti, fino a rendere la classe minoritaria numericamente confrontabile con la classe maggioritaria.

### Solo nelle slide (non spiegato a voce)

Nel diagramma delle slide, una lente di ingrandimento ispeziona il piccolo blocco della *minority class* e lo espande, creando copie fino a raggiungere una dimensione simile a quella della *majority class*.

Il vantaggio principale dell'oversampling è che non elimina dati storici: tutte le istanze originali vengono mantenute.

Lo svantaggio è il rischio di **overfitting**. Il classificatore può attribuire troppa importanza a istanze duplicate, scambiando ripetizioni artificiali per informazione reale.

### Nota del Prof

Duplicare dati non significa aggiungere nuova conoscenza. Se un evento raro viene copiato molte volte, il modello può credere che quel pattern sia più rappresentativo di quanto non sia realmente.

## 13.5 SMOTE

**SMOTE** significa **Synthetic Minority Oversampling Technique**. A differenza dell'oversampling semplice, SMOTE non si limita a duplicare istanze esistenti, ma genera nuove istanze sintetiche della classe minoritaria.

La generazione avviene usando le similarità nello **spazio delle feature** tra istanze minoritarie già presenti. In pratica, SMOTE crea nuovi punti intermedi tra esempi minoritari simili.

### Solo nelle slide (non spiegato a voce)

Nel diagramma delle slide, i *majority samples* sono rappresentati come quadrati tratteggiati, i *minority samples* come pallini neri e i *synthetic samples* come pallini rossi. SMOTE genera esempi sintetici posizionandoli lungo segmenti che collegano istanze minoritarie simili.

Il vantaggio di SMOTE è che introduce maggiore variabilità rispetto alla semplice duplicazione. Tuttavia, esiste il rischio che le istanze sintetiche non siano realistiche.

### Nota del Prof

Il professore sconsiglia l'uso disinvolto di SMOTE. Creare istanze sintetiche significa affidarsi a un algoritmo esterno per inventare nuovi esempi, inserendo dati artificiali nel training set di un altro predittore. Questo può ridurre il realismo dell'esperimento.

## 13.6 Confronto tra undersampling, oversampling e SMOTE

Le tre tecniche modificano il dataset in modi diversi:

- **undersampling**: riduce la classe maggioritaria;
- **oversampling**: duplica la classe minoritaria;
- **SMOTE**: genera nuove istanze sintetiche della classe minoritaria.

L'undersampling può essere utile quando il dataset è molto grande e si vuole ridurre il costo computazionale. L'oversampling può essere utile quando non si vuole perdere informazione storica. SMOTE prova a superare la duplicazione semplice, ma introduce dati artificiali.

I principali rischi sono:

- **perdita di informazione**, tipica dell'undersampling;
- **overfitting**, tipico dell'oversampling semplice;
- **istanze sintetiche non realistiche**, tipiche di SMOTE.

## 13.7 Lab

Il laboratorio richiede di confrontare l'accuratezza dei modelli di machine learning con e senza applicazione dell'oversampling.

Il workflow richiesto è:

1. prendere un dataset;
2. usare tre classificatori:
  - **RandomForest**;
  - **NaiveBayes**;
  - **IBk**;
3. applicare una validazione tramite **66/33 ordered split**;
4. confrontare i risultati con e senza oversampling.

### Nota del Prof

In un progetto software reale, lo split casuale può essere poco realistico. Se si vuole predire una release futura, il modello deve essere addestrato sulle release precedenti. Mischiare release passate e future può introdurre conoscenza futura nel training, falsando l'esperimento.

## 13.8 Weka API

### 13.8.1 Undersampling

#### Solo nelle slide (non spiegato a voce)

In Weka, l'undersampling può essere eseguito con il filtro:

```
weka.filters.supervised.instance.SpreadSubsample -M 1.0
```

### 13.8.2 Oversampling

#### Solo nelle slide (non spiegato a voce)

In Weka, l'oversampling può essere eseguito con il filtro:

```
weka.filters.supervised.instance.Resample
```

Per l'oversampling, le slide indicano i seguenti parametri:

- `noReplacement=false`, per permettere la duplicazione;
- `biasToUniformClass=1.0`;
- `sampleSizePercent=Y`.

Il parametro  $Y$  si calcola come:

$$Y = 100 \cdot \frac{\text{majority} - \text{minority}}{\text{minority}}$$

Solo nelle slide (non spiegato a voce)

Per il dataset diabetes, le slide riportano l'esempio:

```
weka.filters.supervised.instance.Resample -B 1.0 -Z 130.3
```

### 13.8.3 SMOTE

Solo nelle slide (non spiegato a voce)

In Weka, SMOTE può richiedere l'installazione di un package aggiuntivo tramite il gestore dei pacchetti di Weka.

## 13.9 Collegamento con il progetto

Il balancing è rilevante nei dataset di defect prediction perché le classi *buggy* sono spesso una minoranza rispetto alle classi *non buggy*.

Senza balancing, un classificatore può imparare a predire quasi sempre **no**, ottenendo una buona accuracy apparente ma fallendo nel riconoscimento delle classi difettose.

Valutare solo l'accuracy è quindi fuorviante. In un dataset con il 95% di classi sane, un classificatore che predice sempre **no** ottiene il 95% di accuracy, ma non individua alcun difetto.

#### Nota del Prof

Dopo il balancing ci si può aspettare un aumento della **Recall**, perché il modello riconosce più positivi reali. Tuttavia, può diminuire la **Precision**, perché aumentando le predizioni positive possono aumentare anche i falsi allarmi.

Per valutare correttamente i modelli, oltre all'accuracy, è opportuno osservare metriche come:

- **Precision**;
- **Recall**;
- **F1**;
- **AUC**;
- **Kappa di Cohen**.

## 13.10 Da ricordare

### IMPORTANTE PER L'ESAME

Il class imbalance è centrale nella defect prediction perché le classi buggy sono spesso rare. Un modello non bilanciato può sembrare accurato, ma ignorare proprio i difetti.

### IMPORTANTE PER L'ESAME

L'undersampling riduce la classe maggioritaria ma può perdere informazione. L'oversampling mantiene i dati ma può causare overfitting. SMOTE crea dati sintetici, ma questi possono non essere realistici.

### IMPORTANTE PER L'ESAME

Tutte le attività di bilanciamento devono essere applicate esclusivamente al **training set**. Non si deve mai bilanciare il testing set. Bilanciare l'intero dataset prima dello split significa falsare la valutazione, perché il classificatore può essere testato su istanze duplicate o artificiali collegate a quelle viste in addestramento.

### IMPORTANTE PER L'ESAME

In presenza di class imbalance, l'accuracy da sola non basta. È necessario osservare metriche come Recall, Precision, F1, AUC e Kappa.

# Capitolo 14

## Weka GUI e Feature Selection

### 14.1 Introduzione a Weka

**Weka** è un toolkit per il **machine learning** e il **data mining** scritto in Java e distribuito con licenza GNU. Viene usato per ricerca, didattica e applicazioni pratiche, ed è collegato al libro *Data Mining* di Witten e Frank.

Le componenti principali di Weka sono:

- **Explorer**: permette di esplorare visivamente i dati, applicare filtri e usare algoritmi di classificazione;
- **Experimenter**: consente di confrontare più algoritmi su uno o più dataset, anche con test statistici;
- **Knowledge Flow**: permette di progettare graficamente pipeline di machine learning;
- **Simple CLI**: interfaccia da riga di comando per eseguire direttamente classi Java di Weka.

#### Nota del Prof

Nel corso si usano soprattutto **Explorer** ed **Experimenter**. Explorer è utile per prendere confidenza con dataset, distribuzioni, filtri e classificatori; Experimenter è più adatto per confrontare più modelli in modo sistematico e ottenere risultati statisticamente confrontabili.

### 14.2 Formato dati in Weka

Weka lavora principalmente con file **flat**, cioè dataset organizzati come tabelle relazionali singole.

#### Solo nelle slide (non spiegato a voce)

I formati supportati includono **ARFF**, **CSV**, **C4.5** e **binary**.

Il formato proprietario di Weka è **ARFF**, che si basa su tre parti principali:

- **@relation**: nome del dataset;
- **@attribute**: definizione degli attributi, cioè nome e tipo di ogni colonna;
- **@data**: valori delle singole istanze, separati da virgola.

Weka distingue tra attributi **numerici** e attributi **nominali**. Per impostazione predefinita, l'ultimo attributo viene considerato la **classe target** da predire.

#### Nota del Prof

Nel progetto è consigliabile importare i CSV e salvarli subito in formato ARFF. Il CSV non specifica esplicitamente i tipi delle colonne, quindi Weka potrebbe interpretare alcuni attributi in modo errato. Inoltre, Weka non gestisce direttamente le scale ordinali, come *low*, *medium*, *high*: queste vanno trasformate con attenzione, ad esempio tramite codifica binaria o numerica.

## 14.3 Explorer: preprocessing e classificazione

Nel tab **Preprocess** dell'Explorer è possibile importare i dati tramite il pulsante *Open file*. Dopo l'importazione, Weka permette di visualizzare gli attributi, le distribuzioni, il numero di valori distinti, i valori mancanti e alcune statistiche descrittive.

I filtri di preprocessing permettono di trasformare il dataset prima dell'addestramento.

#### Solo nelle slide (non spiegato a voce)

Tra i filtri disponibili nelle slide sono citati **discretization**, **normalization**, **resampling** e **attribute selection**.

Selezionando un attributo, Weka mostra informazioni come minimo, massimo, media, deviazione standard e un istogramma colorato rispetto alla classe target.

Nel tab **Classify**, Weka permette di scegliere diversi classificatori.

#### Solo nelle slide (non spiegato a voce)

Tra i classificatori citati nelle slide compaiono alberi decisionali come **J48** e **RandomForest**, classificatori instance-based come **IBk**, SVM, logistic regression e Bayes nets, tra cui **NaiveBayes**.

Weka include anche **meta-classifiers**, cioè classificatori costruiti sopra altri classificatori.

#### Solo nelle slide (non spiegato a voce)

Tra i meta-classifiers sono citati **bagging**, **boosting** e **stacking**.

Cliccando sul nome del classificatore è possibile aprire l'*object editor* e configurare i parametri. Le principali opzioni di test sono:

- **Use training set**;

- **Supplied test set**;
- **Cross-validation**, ad esempio 10-fold;
- **Percentage split**.

## 14.4 Experimenter

L'**Experimenter** permette di confrontare diversi learning schemes in modo più sistematico rispetto all'**Explorer**. È utilizzabile sia per problemi di classificazione sia per problemi di regressione.

### Nota del Prof

L'**Experimenter** è particolarmente utile quando bisogna confrontare più classificatori, più dataset o più configurazioni. I risultati possono essere esportati, ad esempio in formato CSV, per analisi successive.

### Solo nelle slide (non spiegato a voce)

Le slide indicano che l'**Experimenter** supporta **cross-validation**, **learning curve** e **hold-out**. Inoltre può iterare su diversi parametri e include strumenti di **significance testing**.

Il significance testing permette di capire se la differenza tra due classificatori è statisticamente significativa, invece di limitarsi a confrontare valori medi di accuratezza.

## 14.5 Feature Selection: obiettivi

La **feature selection**, o selezione degli attributi, serve a scegliere un sottoinsieme di feature informative rispetto al task predittivo.

Gli obiettivi principali sono:

- ridurre il costo di learning, diminuendo il numero di attributi;
- migliorare le performance rispetto all'uso del *full attribute set*;
- ridurre il costo di misurazione delle feature.

### IMPORTANTE PER L'ESAME

Nel contesto della Software Engineering, il costo più importante non è sempre il tempo di addestramento del modello, ma il costo di misurazione. Estrarre decine o centinaia di metriche per ogni classe può richiedere molto effort. Ridurre il numero di feature significa ridurre anche il costo di raccolta dei dati.

Nei dataset di defect prediction è frequente avere metriche ridondanti o sovrapposte. Per esempio, LOC, numero di metodi e numero di attributi possono misurare indirettamente la dimensione della classe. La feature selection permette di mantenere solo le feature più utili.

## 14.6 Filter approach

Il **filter approach** valuta gli attributi partendo dai dati, senza considerare il classificatore che verrà poi usato.

I filtri assegnano alle feature un **merit score**, cioè un punteggio che misura quanto una feature sia utile per separare le classi o per descrivere la variabile target.

La valutazione può avvenire in due modi:

- valutando ogni feature singolarmente;
- valutando sottoinsiemi di feature.

Valutare le feature singolarmente è veloce, ma può essere sub-ottimale perché ignora le relazioni tra attributi. Valutare i subset è più potente, ma introduce un problema combinatorio: il numero di possibili sottoinsiemi cresce in modo esponenziale.

### Nota del Prof

I filtri sono molto veloci e utili quando il numero di attributi è grande. Il limite è che il punteggio prodotto, ad esempio un valore di Info Gain, non sempre indica chiaramente dove fissare la soglia per eliminare le feature.

## 14.7 Info Gain e Spearman

### 14.7.1 Information Gain

L'**Information Gain** misura quanta informazione una feature fornisce sulla classe target. Più precisamente, quantifica la riduzione dell'incertezza, misurata tramite entropia, quando si conosce il valore di un attributo.

Un valore alto di Information Gain indica che la feature è utile per separare le istanze in classi distinte.

### Nota del Prof

Una feature con alto Information Gain aiuta il classificatore perché riduce l'incertezza su quale classe assegnare a un'istanza.

### 14.7.2 Spearman correlation

La **Spearman correlation**, o coefficiente rho di Spearman, è una misura non parametrica della forza e della direzione dell'associazione tra due variabili ordinate.

### Solo nelle slide (non spiegato a voce)

A differenza di Pearson, che misura relazioni lineari sui valori grezzi, Spearman valuta relazioni monotone sui ranghi. Il coefficiente varia da  $-1$  a  $+1$ .

Una relazione monotona esiste quando una variabile tende ad aumentare al crescere dell'altra, oppure tende a diminuire al crescere dell'altra, anche se non necessariamente a ritmo costante.

### Nota del Prof

Metriche come Spearman sono interpretabili, ma non risolvono completamente il problema della ridondanza. Due feature possono essere entrambe molto correlate con la classe target, ma misurare quasi la stessa informazione.

## 14.8 Wrapper approach

Il **wrapper approach** seleziona le feature usando il classificatore come una *black box*. A differenza dei filtri, il wrapper dipende dal classificatore specifico.

Il processo generale è:

1. scegliere un subset di feature;
2. creare training e test set basati su quel subset;
3. addestrare il classificatore sul training set;
4. valutare l'accuratezza su un validation set;
5. ripetere il processo per più subset e scegliere quello con il miglior risultato.

Il vantaggio è che le feature vengono ottimizzate per uno specifico classificatore. Lo svantaggio è il costo computazionale elevato, perché il modello deve essere addestrato e testato molte volte.

### Nota del Prof

I wrapper sono più lenti dei filtri, ma possono produrre feature subset più adatti a un classificatore specifico.

## 14.9 Search strategies

Poiché il numero di possibili subset di feature è esponenziale, nei wrapper si usano strategie di ricerca.

- **Exhaustive search:** prova tutte le combinazioni possibili. È completa, ma spesso impraticabile;
- **Forward search:** parte da un insieme vuoto e aggiunge progressivamente la feature che migliora di più il risultato;
- **Backward search:** parte dall'insieme completo e rimuove progressivamente la feature la cui eliminazione danneggia meno l'accuratezza;
- **Greedy search:** prende decisioni locali cercando un buon compromesso tra accuratezza e costo computazionale.

Solo nelle slide (non spiegato a voce)

Le slide indicano che la forward search e la backward search hanno costo  $O(n^2 \cdot \text{learning/testing time})$  e che esistono anche altre euristiche.

### IMPORTANTE PER L'ESAME

Se si ha molta fiducia nel set di feature iniziale, può convenire usare **Backward Search**, perché parte da tutte le feature e ne elimina poche. Se invece si sospetta che molte feature siano inutili o ridondanti, può convenire usare **Forward Search**, perché costruisce un subset più ristretto partendo da zero.

## 14.10 Train, validation e test

È fondamentale distinguere tra:

- **training set**: usato per addestrare il modello;
- **validation set**: usato per tuning, scelta delle feature e scelta del modello;
- **test set**: usato solo per la valutazione finale.

### IMPORTANTE PER L'ESAME

Il test set non deve essere usato per scegliere feature o parametri. Se si usa il test set durante la selezione, si introduce data leakage e la valutazione finale non è più realistica.

Il validation set serve a prendere decisioni sul modello senza contaminare il test set. Solo alla fine il modello selezionato viene valutato sul test set.

## 14.11 Feature Selection in Weka

In Weka, la selezione degli attributi può essere eseguita tramite il pannello **Select attributes**. I metodi di attribute selection sono composti da due parti:

- **Search Method**: metodo con cui si esplora lo spazio dei subset;
- **Attribute Evaluator**: criterio con cui si valuta la qualità delle feature o dei subset.

Solo nelle slide (non spiegato a voce)

Tra i search method citati nelle slide compaiono **best-first**, **forward selection**, **random**, **exhaustive**, **genetic algorithm** e **ranking**.

Solo nelle slide (non spiegato a voce)

Tra gli evaluation method citati nelle slide compaiono metodi **correlation-based**, **wrapper**, **information gain** e **chi-squared**.

### Nota del Prof

Weka è flessibile perché permette di combinare quasi arbitrariamente search method ed evaluator.

## 14.12 Lab e collegamento con il progetto

Il laboratorio richiede di confrontare l'accuratezza con e senza feature selection.

Il workflow richiesto è:

1. prendere un dataset;
2. usare tre classificatori:
  - **RandomForest**;
  - **NaiveBayes**;
  - **IBk**;
3. eseguire una **10-fold cross validation**;
4. confrontare i risultati con e senza feature selection;
5. discutere se la valutazione è fair;
6. valutare se usare API o GUI.

### IMPORTANTE PER L'ESAME

Fare feature selection sull'intero dataset prima della cross-validation non è fair. Significa permettere alla selezione delle feature di vedere anche le fold che dovrebbero essere usate come test. Questo produce data leakage.

### Nota del Prof

In Weka, per evitare data leakage, è preferibile usare il meta-classificatore **AttributeSelectedClassifier**. In questo modo la feature selection viene eseguita separatamente dentro ogni fold di training, senza usare la fold di test.

### Nota del Prof

La GUI è utile per capire il dataset, osservare le distribuzioni e sperimentare rapidamente. Le API o la CLI sono invece più adatte al progetto finale, perché permettono di automatizzare esperimenti ripetibili.

### Nota del Prof

Se si usano sia feature selection sia balancing, è opportuno prestare attenzione all'ordine delle operazioni. La feature selection dovrebbe essere valutata senza contaminare il test set; anche il balancing deve essere applicato solo sul training set.

## 14.13 Da ricordare

### IMPORTANTE PER L'ESAME

In Software Engineering, la feature selection è importante soprattutto perché riduce il costo di misurazione delle metriche, non solo il tempo di training del modello.

### IMPORTANTE PER L'ESAME

I filtri, come Info Gain e Spearman, sono veloci e indipendenti dal classificatore. I wrapper sono più lenti ma ottimizzano le feature per uno specifico classificatore.

### IMPORTANTE PER L'ESAME

Il test set non deve mai essere usato per scegliere feature, parametri o modelli. Qualsiasi forma di selezione deve avvenire usando solo training e validation set.

### IMPORTANTE PER L'ESAME

In Weka, `AttributeSelectedClassifier` è utile perché consente di eseguire feature selection in modo corretto dentro la cross-validation, evitando data leakage.

### IMPORTANTE PER L'ESAME

La GUI serve per comprendere e ispezionare il problema; API e CLI servono per automatizzare esperimenti ripetibili e scalabili.

# Capitolo 15

## Milestone 2: Classifiers Accuracy

### 15.1 Obiettivo della Milestone 2

La **Milestone 2** ha come obiettivo principale il confronto dell'accuratezza di tre classificatori di Machine Learning applicati al dataset costruito nella Milestone 1.

L'input della Milestone 2 è quindi il file CSV prodotto nella Milestone 1, contenente le metriche delle classi e la relativa etichetta di **bugginess**.

Le metriche da confrontare non si limitano alla semplice accuracy, ma includono:

- **Precision**;
- **Recall**;
- **AUC**;
- **Kappa**;
- **NPofB20**.

#### IMPORTANTE PER L'ESAME

La Milestone 2 non richiede solo di eseguire tre classificatori, ma di confrontarli criticamente usando più metriche. Il miglior classificatore può cambiare a seconda della metrica considerata.

### 15.2 Classificatori da confrontare

Solo nelle slide (non spiegato a voce)

I tre classificatori richiesti per l'esperimento sono **RandomForest**, **NaiveBayes** e **IBk**.

#### 15.2.1 RandomForest

**RandomForest** è un classificatore basato su un insieme di alberi decisionali. Ogni albero produce una predizione e il modello finale combina i risultati. In genere è robusto e adatto a dataset complessi.

### 15.2.2 NaiveBayes

**NaiveBayes** è un classificatore probabilistico basato sul teorema di Bayes. Assume, in modo semplificato, che le feature siano indipendenti tra loro.

#### Nota del Prof

NaiveBayes è un classificatore molto veloce in Weka. Questa caratteristica può renderlo utile come baseline o come modello semplice da confrontare con algoritmi più complessi.

### 15.2.3 IBk

**IBk** è l'implementazione Weka del metodo *k-Nearest Neighbours*. È un classificatore *instance-based*: una nuova istanza viene classificata in base alle classi delle  $k$  istanze più simili presenti nel training set.

## 15.3 Validazione sperimentale

La tecnica richiesta è la **10 times 10-fold cross-validation**.

Nella **10-fold cross-validation** semplice, il dataset viene diviso in 10 parti. A turno, 9 parti vengono usate per il training e 1 per il testing, fino a usare ciascuna fold come test set una volta.

Nella **10 times 10-fold cross-validation**, l'intero processo viene ripetuto 10 volte con partizioni differenti, ottenendo complessivamente 100 run.

Ripetere la cross-validation serve a ridurre l'effetto della casualità nella suddivisione delle fold e a ottenere una stima più stabile delle performance.

#### IMPORTANTE PER L'ESAME

La 10 times 10-fold cross-validation è richiesta dalla Milestone 2. Tuttavia, nella Software Engineering è utile ragionare anche su split temporali, come ordered hold-out o walk-forward, perché mischiare release passate e future può produrre una stima ottimistica dell'accuratezza.

## 15.4 Feature Selection

La Milestone 2 richiede l'uso di filtri come tecnica di **feature selection**. La feature selection permette di selezionare solo le metriche più utili, eliminando attributi irrilevanti o ridondanti.

I principali vantaggi sono:

- ridurre il rumore nel dataset;
- migliorare, in alcuni casi, le performance del modello;
- ridurre il costo di estrazione e misurazione delle metriche;
- semplificare l'interpretazione del modello.

### Nota del Prof

Se dopo la feature selection l'accuratezza migliora, significa che alcune feature generavano confusione nel modello. Se invece l'accuratezza diminuisce leggermente, si può comunque ottenere un vantaggio pratico: usare meno metriche e ridurre il costo di raccolta dati.

### IMPORTANTE PER L'ESAME

La feature selection non deve essere applicata sull'intero dataset prima della cross-validation. Se il filtro vede anche le fold di test, si introduce **data leakage** e la valutazione diventa falsata.

In Weka, una soluzione corretta è usare il meta-classificatore:

`AttributeSelectedClassifier`

In questo modo la feature selection viene applicata separatamente sulle fold di training, senza usare informazioni provenienti dalla fold di test.

## 15.5 Balancing

Nei dataset di defect prediction la classe **buggy** è spesso minoritaria. Questo sbilanciamento può spingere il classificatore a predire quasi sempre la classe maggioritaria, cioè non **buggy**.

Per mitigare il problema si possono usare tecniche di **balancing**, come:

- **undersampling**: riduzione della classe maggioritaria;
- **oversampling**: aumento della classe minoritaria;
- **SMOTE**: generazione di istanze sintetiche della classe minoritaria.

### IMPORTANTE PER L'ESAME

Il balancing deve essere applicato solo al **training set**, mai al test set. Bilanciare il test set significa alterare artificialmente la distribuzione reale dei dati su cui si valuta il modello.

In Weka, il balancing può essere gestito tramite il meta-classificatore:

`FilteredClassifier`

### IMPORTANTE PER L'ESAME

Se si usano sia feature selection sia balancing, bisogna evitare che uno dei due passaggi contaminino il test set. Le operazioni devono essere incapsulate nei meta-classificatori, in modo che vengano applicate solo alle fold di training durante la cross-validation.

### Nota del Prof

Quando si combinano feature selection e balancing, è importante ragionare sull'ordine delle operazioni. Il balancing altera artificialmente le istanze, quindi può influenzare la valutazione statistica delle feature se applicato nel momento sbagliato.

## 15.6 Metriche di valutazione

### 15.6.1 Precision

La **Precision** misura quanti degli elementi predetti come buggy sono effettivamente buggy:

$$Precision = \frac{TP}{TP + FP}$$

Una Precision alta indica pochi falsi positivi.

### 15.6.2 Recall

La **Recall** misura quanti degli elementi realmente buggy sono stati trovati dal classificatore:

$$Recall = \frac{TP}{TP + FN}$$

Una Recall alta indica pochi falsi negativi.

### 15.6.3 AUC

L'**AUC**, cioè Area Under the ROC Curve, valuta la capacità del classificatore di distinguere tra classi positive e negative al variare della soglia decisionale.

### 15.6.4 Kappa

Il **Kappa di Cohen** confronta le prestazioni del classificatore con quelle ottenibili per caso. Un valore maggiore di zero indica che il modello sta facendo meglio di una classificazione casuale o banale.

### Nota del Prof

Il classificatore ZeroR predice sempre la classe maggioritaria. In dataset sbilanciati può ottenere un'accuracy alta, ma un Kappa pari a zero, perché non sta realmente imparando un pattern predittivo.

### 15.6.5 NPofB20

Solo nelle slide (non spiegato a voce)

**NPofB20** è una metrica effort-aware normalizzata. Valuta quanti difetti si riescono a trovare ispezionando il 20% delle LOC del sistema.

La logica è ordinare le classi in base a una priorità di ispezione, tipicamente legata alla probabilità predetta di difettosità normalizzata rispetto alla dimensione della classe.

#### IMPORTANTE PER L'ESAME

L'accuracy da sola non basta nei dataset sbilanciati. Un modello che predice sempre **non buggy** può sembrare accurato, ma non trovare nessuna classe difettosa. Per questo sono fondamentali metriche come Recall, Kappa e NPofB20.

## 15.7 Domande da rispondere nella Milestone 2

La Milestone 2 richiede di rispondere a domande sperimentali come:

- quale classificatore è più accurato degli altri?
- il miglior classificatore cambia in base al dataset?
- il miglior classificatore cambia in base al numero di release considerate?
- il miglior classificatore cambia in base alla metrica?
- quale classificatore è migliore per una determinata metrica?

Non esiste necessariamente un classificatore migliore in assoluto. Un modello può avere alta Recall ma bassa Precision, oppure buona AUC ma Kappa modesto.

#### IMPORTANTE PER L'ESAME

Nel report è importante discutere i trade-off. Il docente non cerca solo il valore numerico migliore, ma la capacità di giustificare le scelte sperimentali e interpretare i risultati.

## 15.8 Weka e API

### 15.8.1 Uso della GUI

La GUI di Weka, in particolare Explorer ed Experimenter, è utile per comprendere il workflow, testare rapidamente i classificatori e confrontare configurazioni diverse.

L'Experimenter permette di lanciare esperimenti ripetuti, come la 10 times 10-fold cross-validation, e salvare i risultati in una tabella esportabile.

### Nota del Prof

Dal CSV prodotto dall'Experimenter può essere utile costruire una colonna descrittiva del modello, combinando classificatore e opzioni, ad esempio `RandomForest_FS_Balanced`.

## 15.8.2 Limite della GUI per NPofB20

La GUI di Weka non calcola nativamente la **NPofB20**. Per calcolarla servono informazioni a livello di singola istanza, come:

- ID dell'istanza;
- LOC o dimensione della classe;
- classe reale;
- probabilità predetta;
- ordinamento delle classi secondo la priorità effort-aware.

### Nota del Prof

Le API Java di Weka o uno script esterno sono utili per stampare le predizioni istanza per istanza e calcolare automaticamente NPofB20.

## 15.9 Collegamento con il progetto

L'input della Milestone 2 è il CSV finale prodotto nella Milestone 1. L'output è una tabella di confronto tra classificatori e configurazioni sperimentali.

Per ciascuno dei tre classificatori si possono confrontare diverse varianti:

- modello base;
- modello con feature selection;
- modello con balancing;
- modello con feature selection e balancing.

Questo produce una tabella comparativa su più metriche, come Precision, Recall, AUC, Kappa e NPofB20.

### IMPORTANTE PER L'ESAME

L'obiettivo finale della Milestone 2 è identificare e giustificare un **best classifier**, cioè la combinazione di algoritmo e preprocessing più adatta al proprio dataset. Questo modello sarà poi usato nella Milestone 3.

## 15.10 Da ricordare

#### **IMPORTANTE PER L'ESAME**

Feature selection e balancing devono essere applicati senza contaminare il test set. In Weka è opportuno usare meta-classificatori come `AttributeSelectedClassifier` e `FilteredClassifier`.

#### **IMPORTANTE PER L'ESAME**

Il balancing va applicato solo al training set. Bilanciare il test set significa falsare la valutazione.

#### **IMPORTANTE PER L'ESAME**

La 10 times 10-fold cross-validation è richiesta dalla Milestone 2, ma bisogna essere consapevoli che nei dati software l'ordine temporale delle release ha significato.

#### **IMPORTANTE PER L'ESAME**

L'accuracy non è sufficiente per valutare modelli su dataset sbilanciati. Bisogna considerare Precision, Recall, AUC, Kappa e NPofB20.

#### **IMPORTANTE PER L'ESAME**

Il miglior classificatore può cambiare in base alla metrica. Per questo il risultato deve essere discusso, non solo riportato numericamente.

# Capitolo 16

## What If I Had No Smells?

### 16.1 Introduzione ai code smells

#### 16.1.1 Definizione ed esempi

I **code smells** sono artefatti del codice che rappresentano sintomi di scelte di progettazione o implementazione non ottimali. Non coincidono necessariamente con un difetto osservabile, ma segnalano la presenza di strutture che possono rendere il software più difficile da comprendere, modificare o mantenere.

Tra gli esempi riportati nel materiale compaiono:

- **God classes**, cioè classi troppo grandi e sovraccariche di responsabilità;
- **code clones**, cioè duplicazioni di codice;
- **low comment frequency**, cioè una frequenza troppo bassa di commenti;
- **non-intuitive variable naming**, cioè nomi di variabili poco chiari o poco intuitivi.

#### 16.1.2 Legame con il technical debt

I **code smells** contribuiscono all'accumulo di **technical debt**, cioè codice non pienamente buono o non pienamente pulito che viene comunque rilasciato nel prodotto software. Questo debito tecnico non implica necessariamente un malfunzionamento immediato, ma può ridurre progressivamente la qualità del prodotto e rendere più costose le attività future di evoluzione e manutenzione.

#### 16.1.3 Analisi storica e analisi controfattuale

Gran parte della letteratura tradizionale ha studiato il problema in termini di **analisi storica**: si osservano i repository e si cerca di capire se gli smells siano correlati alla qualità del prodotto o al numero di difetti.

Questo studio propone invece una **analisi controfattuale**. La domanda non è più soltanto *che cosa è successo*, ma *che cosa sarebbe successo se gli smells fossero stati evitati*. Il passaggio concettuale è quindi da una logica descrittiva a una logica ipotetica e decisionale.

## 16.2 Obiettivo del lavoro

### 16.2.1 Domanda centrale del paper

La domanda centrale del lavoro è la seguente: **il numero di file difettosi diminuisce se i code smells vengono evitati, mantenendo inalterate le altre caratteristiche di processo e di prodotto?**

### 16.2.2 Significato della stima

L'obiettivo è definire un metodo per **stimare il numero di defective files** in uno scenario ipotetico in cui gli smells siano assenti. Non si tratta quindi di misurare direttamente un intervento reale di refactoring, ma di costruire uno scenario sintetico in cui la variabile relativa agli smells venga modificata per valutare quale impatto avrebbe avuto sul numero di file difettosi.

### 16.2.3 Interesse pratico e di ricerca

Dal punto di vista della ricerca, questo problema è interessante perché gli esperimenti controllati sugli smells sono costosi, difficili da scalare e poco adatti a scenari multivariati. Dal punto di vista industriale, invece, i manager hanno bisogno di strumenti che supportino decisioni pratiche su qualità, costi ed effort, senza richiedere necessariamente esperimenti complessi o interventi invasivi sul codice.

## 16.3 What-if analysis method

### 16.3.1 Significato di what-if scenario

La **what-if analysis** è una tecnica che costruisce scenari ipotetici modificando il valore di alcune variabili per osservare come cambierebbero i risultati finali. Nel caso di questo studio, lo scenario ipotetico è quello di un software in cui i file non presentino code smells.

### 16.3.2 Azzerare gli smells mantenendo invariate le altre caratteristiche

L'idea metodologica consiste nel modificare i dati storici in modo da porre a zero il numero di smells, mantenendo però invariate le altre caratteristiche del file e del processo di sviluppo, come dimensione, churn, numero di revisioni, autori e altre change metrics.

#### Nota del Prof

È importante capire perché la feature `NSmells` sia stata scelta come variabile da manipolare. Non tutte le feature sono realmente *actionable*. Se un modello dicesse che i difetti aumentano con la dimensione del codice, non potremmo semplicemente ridurre il numero di righe eliminando funzionalità utili. Invece, la presenza di code smells può essere controllata attivamente, per esempio tramite refactoring o quality gates, senza modificare i requisiti funzionali del sistema.

### 16.3.3 Uso di modelli appresi su dati reali e applicati a dati sintetici

Il metodo prevede di addestrare modelli di classificazione su dati reali e poi applicarli a dati sintetici ottenuti azzerando la variabile relativa agli smells. In questo modo si cerca di stimare la difettosità attesa in uno scenario in cui il codice, a parità di altre condizioni, non presenti smells.

## 16.4 Contesto industriale

### 16.4.1 Progetti e tipi di smell analizzati

Lo studio è stato applicato su dati provenienti da **cinque progetti industriali** della durata di circa dieci anni, appartenenti all'azienda **Keymind**. Complessivamente sono stati analizzati **106 tipi diversi di smells**.

### 16.4.2 Ruolo di SonarQube e dei quality gates

Nel contesto industriale illustrato, **SonarQube** viene usato per identificare gli smells nel codice e per supportare la gestione del debito tecnico. Inoltre, vengono utilizzati i **quality gates**, cioè regole che bloccano automaticamente l'accettazione di commit o push request quando il codice supera determinate soglie di qualità.

**Solo nelle slide (non spiegato a voce)**

Nel materiale è mostrato anche un esempio di output di SonarQube, in cui viene segnalato uno smell dovuto alla presenza di linee di codice commentate invece che rimosse.

### 16.4.3 Avoiding smells vs removing smells

Lo studio insiste sulla differenza tra due strategie:

- **removing smells**, cioè eliminare gli smells dopo che sono già entrati nella codebase, tipicamente tramite refactoring;
- **avoiding smells**, cioè impedire che entrino fin dall'inizio, per esempio bloccando il codice non conforme tramite quality gates.

La seconda strategia è considerata particolarmente interessante perché impedisce l'accumulo del debito tecnico e riduce il rischio di dover poi rifattorizzare codice già integrato.

### 16.4.4 Vantaggi e svantaggi dell'evitare smells a priori

Evitare gli smells a priori ha diversi vantaggi: impedisce l'accumulo progressivo di technical debt, evita parte dei rischi connessi al refactoring e rende più semplice mantenere il codice sotto controllo nel tempo.

Esistono però anche svantaggi importanti. Questa strategia richiede infrastrutture e tool adeguati, è applicabile soprattutto al nuovo codice e può entrare in conflitto con

obiettivi di business come il **time-to-market**. In alcuni contesti aziendali, accettare una certa quantità di smells può quindi essere una scelta intenzionale e ragionevole.

## 16.5 Metodo dello studio

Il metodo proposto si articola in **cinque step** principali:

1. **metric selection**;
2. **metric collection**;
3. **dataset manipulation**;
4. **model selection**;
5. **estimation results**.

L'intero studio può essere letto come una pipeline in cui si parte dalla scelta delle metriche, si raccolgono i dati, si costruiscono scenari sintetici, si seleziona il classificatore più adatto e infine si stima l'impatto dell'assenza di smells sulla difettosità.

## 16.6 Metric selection e metric collection

### 16.6.1 Perché servono metriche oltre a Size e NSmells

Per costruire un predittore credibile non basta conoscere la dimensione del file o il numero di smells presenti. È necessario considerare anche metriche capaci di descrivere l'evoluzione del file e le attività di manutenzione subite durante una release.

### 16.6.2 Ruolo delle change metrics

Le **change metrics** sono state selezionate perché ampiamente usate in letteratura per la **defect estimation**. Esse descrivono sia aspetti di prodotto sia aspetti di processo, e aiutano a catturare il comportamento storico del file durante lo sviluppo.

### 16.6.3 Tabella delle metriche

Solo nelle slide (non spiegato a voce)

La tabella delle metriche include, oltre a **NSmells**, anche metriche come **Size**, **LOC\_touched**, **NR**, **NFix**, **NAuth**, **LOC\_added**, **Churn**, **ChgSetSize**, **Age** e **WeightedAge**. Nel complesso, queste variabili descrivono quanto un file sia grande, quanto venga modificato, da quanti sviluppatori venga toccato e quanto sia esposto a dinamiche di manutenzione.

### 16.6.4 Raccolta dei dati con SonarQube

La raccolta delle metriche è stata eseguita usando **SonarQube**, che nel contesto del paper viene descritto come capace di identificare fino a **1297 tipi di smells**. Nel dataset studiato sono stati raccolti **6912 smells** complessivi.

### 16.6.5 Differenza temporale tra smells e difetti

Un punto metodologico cruciale è la differenza temporale tra le misurazioni:

- gli **smells** e la **dimensione del file** vengono misurati all'**inizio** della release;
- le **change metrics** e i **difetti** vengono misurati alla **fine** della release.

Questa scelta serve a rispettare la direzione temporale del fenomeno: gli smells devono essere già presenti prima che possano influenzare il lavoro degli sviluppatori e contribuire all'introduzione di difetti.

#### Nota del Prof

L'idea è che gli smells funzionino come una sorta di trappola o di fattore di rischio. Per poter influenzare negativamente chi modifica il codice, devono esistere già nel momento in cui inizia la manutenzione del file.

### 16.6.6 Significato del diagramma di raccolta dati

#### Solo nelle slide (non spiegato a voce)

Il diagramma di raccolta dati per release e file mostra visivamente una *release window* in cui gli smells sono osservati in prossimità della **first revision**, mentre churn e defects vengono osservati alla **last revision**. Il messaggio principale è che le cause potenziali vengono misurate prima degli effetti osservati.

## 16.7 Dataset manipulation

### 16.7.1 Definizione dei dataset A, B+, B e C

La manipolazione del dataset introduce quattro insiemi distinti:

- **A**: il dataset originale, non alterato;
- **B+**: la porzione di A che contiene solo file con almeno uno smell, cioè  $N\text{Smells} > 0$ ;
- **C**: la porzione di A che contiene file senza smells, cioè  $N\text{Smells} = 0$ ;
- **B**: il dataset sintetico ottenuto da B+ azzerando artificialmente la feature  $N\text{Smells}$ .

### 16.7.2 Perché B rappresenta lo scenario sintetico

Il dataset **B** rappresenta il vero **what-if scenario**. I file sono gli stessi di B+, ma la presenza di smells viene forzata a zero. In questo modo si costruisce una versione artificiale dei file originariamente “smelly”, come se tali smells fossero stati evitati fin dall'inizio.

### 16.7.3 Media delle feature e correlazione

Solo nelle slide (non spiegato a voce)

Nel materiale viene mostrata una tabella con la media delle feature nei dataset A, B e C e con la correlazione rispetto a **NSmells** e **Defectiveness**. Il messaggio importante è che **NSmells** ha una correlazione significativa ma non fortissima con la defectiveness, e non mostra una collinearità dominante con le altre change metrics. Questo suggerisce che la feature trasporta un'informazione aggiuntiva e non banalmente ridondante.

### 16.7.4 Messaggio metodologico della manipolazione

La manipolazione del dataset non ha soltanto un valore pratico, ma anche metodologico. Il paper giustifica l'uso del modello appreso su A per fare predizioni su B mostrando che, salvo eccezioni limitate, A e B non risultano trattabili come popolazioni radicalmente diverse. Ciò rende più plausibile applicare il classificatore addestrato sul dataset reale a uno scenario sintetico ottenuto modificando la sola feature **NSmells**.

## 16.8 Model selection

### 16.8.1 Classificatori confrontati

I modelli confrontati sono i seguenti:

- Bagging;
- BayesNet;
- J48;
- Logistic.

### 16.8.2 Uso della leave-one-out cross-validation

Per misurare in modo rigoroso le performance dei modelli, viene utilizzata la **leave-one-out cross-validation**. Questa tecnica è particolarmente adatta quando il dataset disponibile non è enorme e si vuole sfruttare al massimo l'informazione presente, mantenendo comunque una validazione severa.

### 16.8.3 Criterio di scelta del modello migliore

Il criterio di scelta si basa soprattutto sul bilanciamento tra **Precision** e **Recall**. L'idea non è selezionare un modello che sia semplicemente aggressivo nel trovare positivi, ma uno che riesca anche a farlo in modo sufficientemente affidabile.

### 16.8.4 Perché Bagging risulta il migliore

Il modello **Bagging** risulta il migliore nel materiale perché ottiene i valori migliori nel confronto mostrato, con una **Precision pari a 0.79** e una **Recall pari a 0.70**. Rispetto agli altri tre classificatori, offre quindi il compromesso più convincente tra capacità di trovare file difettosi e affidabilità delle predizioni positive.

## 16.9 Results

### 16.9.1 Applicazione del modello ai dataset A, B+, B e C

Una volta scelto il modello migliore, cioè **Bagging**, esso viene addestrato sul dataset A e poi applicato ai dataset **A**, **B+**, **B** e **C** per stimare il numero di **defective files** nei diversi scenari.

### 16.9.2 Interpretazione del passaggio da 66 a 38 defective files in B+

Nel sottoinsieme **B+**, cioè tra i file che originariamente avevano almeno uno smell, il numero di file difettosi passa da **66** a **38** quando si considera il dataset sintetico **B**, in cui gli smells sono stati azzerati.

Questo significa che **28 file** sarebbero potenzialmente stati risparmiati, con una riduzione del:

$$\frac{66 - 38}{66} = 42\%$$

Questa percentuale va letta come una riduzione interna al solo gruppo dei file che in origine erano affetti da smells.

### 16.9.3 Riduzione complessiva del 20% su A

Nel dataset complessivo **A** erano presenti **135 defective files**. Se 28 di essi risultano potenzialmente evitabili nello scenario senza smells, allora la riduzione globale stimata sull'intero dataset è pari a:

$$\frac{66 - 38}{135} \approx 20\%$$

Il risultato complessivo del paper è quindi che circa **il 20% dei file difettosi totali** potrebbe essere stato evitato se gli smells fossero stati prevenuti.

### 16.9.4 Differenza tra riduzione sui file smelly e riduzione sull'intero dataset

#### Nota del Prof

È fondamentale non confondere la riduzione del **42%** con quella del **20%**. Il 42% riguarda soltanto il sottoinsieme dei file originariamente smelly, cioè **B+**. Il 20% riguarda invece l'intero dataset **A**. I file presenti in **C** erano già privi di smells ma potevano comunque essere difettosi. Questi rappresentano bug che non sarebbero

stati evitati semplicemente eliminando gli smells, e spiegano perché la riduzione complessiva sia molto più bassa di quella osservata nel solo sottoinsieme smelly.

## 16.10 Limiti e messaggi chiave

### 16.10.1 Perché il risultato non implica causalità certa

Il risultato ottenuto non dimostra una causalità certa tra smells e difetti. La **what-if analysis** costruisce una stima controfattuale basata su modelli statistici, non un esperimento controllato in cui le condizioni vengono realmente manipolate nel mondo reale.

### 16.10.2 Natura ipotetica dello scenario senza smells

L'intero ragionamento si basa su una storia ipotetica: si immagina un software in cui il numero di smells sia pari a zero, ma si mantengano inalterate tutte le altre caratteristiche osservate. Questa è un'assunzione forte e semplificatrice, utile per costruire l'analisi ma non pienamente realistica.

#### Nota del Prof

Nella realtà, correggere uno smell non significa soltanto cambiare il valore di `NSmells`. Spesso comporta modifiche strutturali che alterano anche altre feature, come la dimensione del file, il churn, il numero di autori o la complessità complessiva. Per questo lo scenario sintetico è utile come strumento di ragionamento, ma non va interpretato come una fotografia perfetta di ciò che sarebbe davvero successo.

### 16.10.3 Evitare alcuni smells potrebbe non ridurre i difetti

Non tutti gli smells vanno interpretati come fattori che, se rimossi, riducono automaticamente il numero di bug. In alcuni casi, evitare o rimuovere certi smells potrebbe non produrre alcun beneficio rilevante sulla difettosità, e in altri casi potrebbe persino peggiorare la situazione se il codice risultante diventasse più complesso o meno comprensibile.

### 16.10.4 Casi in cui accettare smells può essere ragionevole

Dal punto di vista industriale, esistono casi in cui accettare una certa quantità di smells può essere una scelta razionale. Per esempio, un'azienda può preferire tollerare alcuni problemi di qualità interna pur di rispettare scadenze rigide, contenere i costi o arrivare rapidamente sul mercato.

### 16.10.5 Trade-off con costi, effort e time-to-market

Il tema centrale, quindi, non è inseguire sempre e comunque l'obiettivo di **zero smells**, ma bilanciare:

- qualità del codice;
- effort richiesto per prevenire o rimuovere gli smells;

- **time-to-market**;
- **costo complessivo della decisione.**

### 16.10.6 Contributo principale del paper

Il contributo principale del lavoro non è solo il valore numerico finale del **20%**, ma soprattutto la proposta di un metodo basato su **what-if analysis** per supportare il **decision making** manageriale. Il paper mostra come i dati storici e i modelli predittivi possano essere usati non solo per descrivere il passato, ma anche per ragionare in modo quantitativo su scenari alternativi.

### 16.10.7 Valore della what-if analysis e ruolo degli smells

Il valore della **what-if analysis** sta nel trasformare il problema degli smells in una domanda decisionale concreta: *vale la pena imporre regole più severe per prevenire gli smells?* Il lavoro suggerisce che gli smells possono rappresentare un fattore collegato alla difettosità e che, almeno in alcuni contesti, evitarli potrebbe ridurre in modo significativo il numero di file difettosi.

### 16.10.8 Importanza di contestualizzare i risultati

Il messaggio finale è che i risultati devono sempre essere letti nel loro contesto. Non esiste una regola universale secondo cui eliminare gli smells sia sempre la scelta migliore. La bontà della decisione dipende dal tipo di progetto, dai vincoli aziendali, dal processo di sviluppo e dagli obiettivi di business.

In sintesi, il paper propone una metodologia utile per ragionare su un trade-off reale della Software Engineering: **quanto conviene investire oggi nella qualità interna del codice per ridurre i costi e i difetti di domani.**

# Capitolo 17

## Characterizing Automated Refactoring

### 17.1 Obiettivo della Milestone 4

#### 17.1.1 Significato di *characterizing automated refactoring*

L'obiettivo di questa fase finale del progetto è studiare e caratterizzare la capacità dei **Large Language Models (LLM)** di svolgere attività di **automated refactoring**. In particolare, si vuole capire se sia possibile ottenere codice con **zero smells** e quale tipo di input, in termini di prompt e test, sia necessario fornire al modello per permettergli di completare correttamente il task.

#### 17.1.2 Collegamento con *maintainability* e *smells*

La rimozione degli **smells** dal codice sorgente ha come finalità il miglioramento della **maintainability** del sistema software.

#### IMPORTANTE PER L'ESAME

L'intera logica della milestone si inserisce in una storia metodologica più ampia: capire quanti difetti si sarebbero potuti prevenire se il codice avesse avuto zero smells e verificare se sia realisticamente possibile avvicinarsi a questo stato usando il refactoring automatico guidato da LLM.

### 17.2 Class selection

#### 17.2.1 Ranking delle classi dell'ultima release

La prima fase operativa richiede di selezionare le classi su cui effettuare l'esperimento. Per farlo, si considera l'**ultima release attuale del progetto** e si costruisce un ranking delle classi in base al numero di smells presenti, cioè in base a **NSmells**.

#### 17.2.2 Esclusione delle classi troppo piccole o semplici

Dal ranking devono essere escluse le classi troppo banali, per esempio quelle caratterizzate da pochi metodi o da una struttura troppo semplice.

### Nota del Prof

Questa scelta è inevitabilmente soggettiva e va giustificata nel report. L'idea è che una classe troppo piccola renderebbe il task di refactoring banale anche per l'LLM, togliendo valore sperimentale all'analisi.

## 17.2.3 Algoritmo di selezione delle due classi

Una volta filtrata la lista, si selezionano due classi tramite un algoritmo basato sulla prima lettera del proprio nome:

1. si prende il numero corrispondente alla prima lettera del nome;
2. si calcola  $X = \text{numero} \bmod 5$ ;
3. in base al valore di  $X$ , si scelgono due classi in posizioni simmetriche del ranking.

Per esempio, se  $X = 0$ , si selezionano la prima e l'ultima classe; se  $X = 1$ , la seconda e la penultima; e così via.

### Nota del Prof

Questo meccanismo serve a garantire che l'LLM venga testato su due casi di difficoltà differente: una classe molto problematica, in alto nel ranking, e una classe meno problematica, in basso. In questo modo è possibile osservare se il comportamento del modello cambia al variare della gravità iniziale degli smells.

## 17.3 Automated refactoring con Microsoft Copilot

### 17.3.1 Uso dell'account universitario

Per svolgere il task è richiesto l'uso di **Microsoft Copilot** tramite l'account universitario, così da poter utilizzare una versione avanzata del modello.

### 17.3.2 Significato di $C_0$ e $C_X$

Nel processo di refactoring,  $C_0$  rappresenta la **classe originale**, cioè la classe di partenza con i suoi smells. Invece,  $C_X$  rappresenta la **nuova classe refactored** prodotta dall'LLM in risposta al prompt.

### 17.3.3 Struttura del prompt e vincoli

Il prompt suggerito imposta il modello come un esperto sviluppatore Java e gli chiede di migliorare la **maintainability** della classe  $C_0$ , producendo una nuova versione  $C_X$ . Il prompt deve imporre alcuni vincoli precisi:

1. mantenere invariata la funzionalità;
2. rimuovere gli smells indicati nel report diagnostico di SonarCloud;

3. fare in modo che la classe passi una specifica **test suite**;
4. evitare modifiche non richieste;
5. garantire che  $C_X$  possa sostituire  $C_0$  in modo *plug and play*, continuando a lavorare correttamente con il resto del sistema.

#### Nota del Prof

La frase con cui si chiede al modello di prendersi tutto il tempo necessario per fornire una risposta accurata non è un dettaglio stilistico: può influenzare concretamente la qualità dell'output generato.

## 17.4 Versioni refactored e tabella $C_X$

### 17.4.1 Significato delle versioni

Lo studio prevede la generazione di più versioni del refactoring, così da capire quale livello di informazione debba essere fornito all'LLM per ottenere risultati convincenti. Le versioni considerate sono:

- $C_0$ : classe originale;
- $C_1, C_2, C_3, C_4$ : versioni refactored generate modificando l'input del prompt.

### 17.4.2 Differenza tra classe originale e versioni refactored

La classe  $C_0$  è il file reale estratto dal progetto, mentre le classi  $C_1, C_2, C_3, C_4$  sono output prodotti dal modello di intelligenza artificiale in condizioni sperimentali diverse.

### 17.4.3 Spiegazione della tabella mostrata nelle slide

Nelle slide è presente una tabella che mette in relazione le versioni della classe con i **test sviluppati a partire da  $C_0$** . Le colonne riportano le sigle **Test BB**, **Test CF**, **Test MT**, **Test RND**, **Test ES** e **Test LLM**.

#### Nota del Prof

BB indica *Black Box*. Ai fini metodologici, però, il punto principale non è tanto entrare nei dettagli di ciascun tipo di test, quanto osservare che cosa accade alla qualità dell'output man mano che aumentano le informazioni fornite all'LLM.

#### Solo nelle slide (non spiegato a voce)

Le sigle **MT**, **RND**, **ES** e **LLM** compaiono nella tabella come parte della progressione sperimentale, ma non vengono spiegate nel dettaglio nell'audio della lezione.

#### 17.4.4 Interpretazione della progressione No/Yes

I valori **No/Yes** nella tabella non indicano se la classe passa o fallisce i test dopo la generazione. Indicano invece se quel test è stato **fornito come input nel prompt** per produrre la relativa versione  $C_X$ .

Più precisamente:

- per  $C_1$ , nessun test è fornito in input;
- per  $C_2$ , viene fornito solo il test BB;
- per  $C_3$ , vengono forniti BB e CF;
- per  $C_4$ , vengono forniti BB, CF e MT.

Il significato metodologico della tabella è quindi mostrare una **crescita progressiva dell'informazione** data al modello.

### 17.5 Analisi finale dei risultati

Una volta ottenute le classi  $C_X$ , esse devono essere analizzate con tre domande principali.

#### 17.5.1 Compilazione

La prima domanda è se  **$C_X$  compili**. Inoltre, va verificato se compili in isolamento oppure se compili correttamente quando reintegrata nel sistema.

#### 17.5.2 Presenza di smells

La seconda domanda è se  **$C_X$  presenti ancora degli smells**. In particolare, bisogna distinguere tra:

- smells rimasti dalla versione originale;
- smells effettivamente rimossi;
- nuovi smells eventualmente introdotti dal refactoring.

#### 17.5.3 Confronto tra $C_X$ e $C_0$

La terza domanda è relativa al confronto tra la versione originaria  $C_0$  e le versioni refactored  $C_X$ . Lo scopo è capire se il refactoring automatico abbia prodotto un miglioramento reale oppure solo apparente.

## 17.6 Feature e bugginess

### 17.6.1 Feature positivamente correlate con la bugginess

L'analisi non può fermarsi al fatto che il codice compili o che alcuni smells siano spariti. Occorre ricalcolare anche le **feature architetturali** e confrontarle con quelle osservate in  $C_0$ .

In particolare, bisogna considerare le feature che nelle milestone precedenti erano risultate **positivamente correlate con la bugginess**. Se tali feature aumentano in  $C_X$ , allora il refactoring potrebbe avere solo nascosto il problema, peggiorando in realtà la maintainability.

Un esempio tipico è la **size**: eliminare uno smell ma ottenere una classe molto più grande o complessa non rappresenta necessariamente un miglioramento.

### 17.6.2 Feature negativamente correlate con la bugginess

Se invece aumenta una feature che era risultata **negativamente correlata con la bugginess**, allora il refactoring può essere interpretato come un miglioramento reale della maintainability.

#### Nota del Prof

Non è necessario ricalcolare da zero tutte le correlazioni. È sufficiente usare i risultati già emersi nelle milestone precedenti e osservare se, nelle versioni refactored, le metriche rilevanti siano salite o scese rispetto a  $C_0$ .

## 17.7 Significato metodologico della milestone

### 17.7.1 Che cosa significa caratterizzare un refactoring automatico

**Characterizing automated refactoring** significa studiare in modo empirico il comportamento dell'intelligenza artificiale durante il refactoring, cercando di capire non solo se il modello riesca a modificare il codice, ma anche **quale livello di informazione gli serve** per farlo bene.

### 17.7.2 Refactoring sintatticamente corretto e refactoring realmente utile

#### IMPORTANTE PER L'ESAME

Un refactoring automatico non può essere considerato riuscito soltanto perché il codice compila o perché SonarCloud non segnala più gli smells originari. Se la nuova classe non passa più i test, oppure degrada pesantemente altre metriche rilevanti, allora il refactoring è da considerarsi fallito.

### 17.7.3 Importanza di una valutazione olistica

La validazione finale del task richiede quindi una prospettiva **olistica**. Per stabilire se il refactoring automatico abbia davvero migliorato il sistema, bisogna considerare contemporaneamente:

- la **compilazione**;
- il passaggio della **test suite**;
- la presenza o assenza di **smells**;
- l'andamento delle **metriche software** correlate con la bugginess.

Solo combinando questi quattro elementi è possibile concludere se  $C_X$  rappresenti davvero una versione migliore di  $C_0$  dal punto di vista della maintainability.

# Capitolo 18

## Introduzione e concetti generali di Software Testing

### 18.1 Qualità, Verifica e Validazione

Nei sistemi software esiste spesso una discrepanza tra ciò che il cliente desidera, ciò che viene progettato, ciò che viene implementato e ciò che è realmente necessario. Questa distanza rende necessario introdurre attività sistematiche di garanzia della qualità.

Citando Boehm, le attività di Verifica e Validazione rispondono a due domande diverse:

- **Verifica:** *Are we building the product right?* Controlla se il prodotto è conforme alle specifiche tecniche.
- **Validazione:** *Are we building the right product?* Controlla se il prodotto soddisfa le attese del committente o dell'utente.

L'obiettivo delle attività di V&V è rassicurare l'utilizzatore che il sistema sia adatto allo scopo. Il livello di fiducia fornito non è assoluto, ma dipende dal contesto.

#### Nota del Prof

Il livello di confidenza richiesto varia in base alla funzione del software. Un gestionale per documenti richiede garanzie diverse rispetto a un sistema safety-critical che può prendere decisioni vitali.

### 18.2 Modello formale $P, S, D, C$

Per formalizzare il problema della verifica si considerano quattro elementi:

- $S$ : specifica, cioè descrizione del comportamento atteso;
- $D$ : dominio degli input;
- $C$ : codominio degli output;
- $P$ : programma, cioè funzione che mappa elementi di  $D$  in  $C$ .

Un programma  $P$  soddisfa la specifica  $S$  se e solo se, per ogni input  $d \in D$ , vale:

$$P(d) = S(d)$$

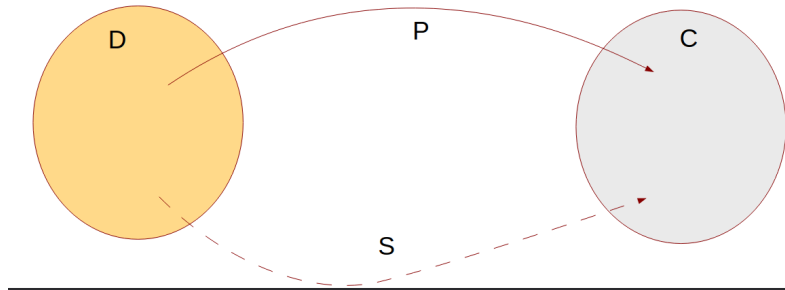


Figura 18.1: Rappresentazione del dominio  $D$ , del codominio  $C$ , del programma  $P$  e della specifica  $S$ . Il programma è corretto rispetto alla specifica quando, per ogni input del dominio, il comportamento prodotto da  $P$  coincide con quello atteso da  $S$ .

## 18.3 Verifica formale, testing e debugging

Esistono approcci diversi per obiettivi diversi.

La **verifica formale** mira a dimostrare matematicamente che il programma  $P$  soddisfa la specifica  $S$  su ogni possibile input.

### Nota del Prof

La verifica formale richiede competenze specialistiche, ad esempio in logica, grafi o algebra. Spesso opera su modelli astratti del sistema e non direttamente sul codice eseguito in produzione.

Il **software testing** mira invece a cercare differenze tra  $P$  e  $S$  tramite esperimenti su un sottoinsieme di input.

Il **debugging** è l'attività che segue l'osservazione di una difformità: serve a individuare la causa del problema e a rimuoverla.

### IMPORTANTE PER L'ESAME

Testing e debugging non sono la stessa cosa. Il testing cerca di evidenziare una failure, cioè di mostrare che il programma si comporta diversamente dalla specifica. Il debugging interviene dopo, per capire la causa e correggere il difetto.

## 18.4 Failure, Fault, Error

La definizione banale “corretto = senza errori” non è sufficiente. È necessario distinguere tra tre concetti:

- **Failure** o **malfunzionamento**: comportamento non corretto e osservabile dall'esterno;

- **Fault, bug o difetto:** problema presente nel codice o nel modello;
- **Error o errore:** causa originaria del fault, spesso dovuta a un errore umano.

La relazione causale è:

$$\text{Error} \rightarrow \text{Fault} \rightarrow \text{Failure}$$

Un errore umano introduce un fault nel codice; il fault, se attivato in certe condizioni, può manifestarsi come failure.

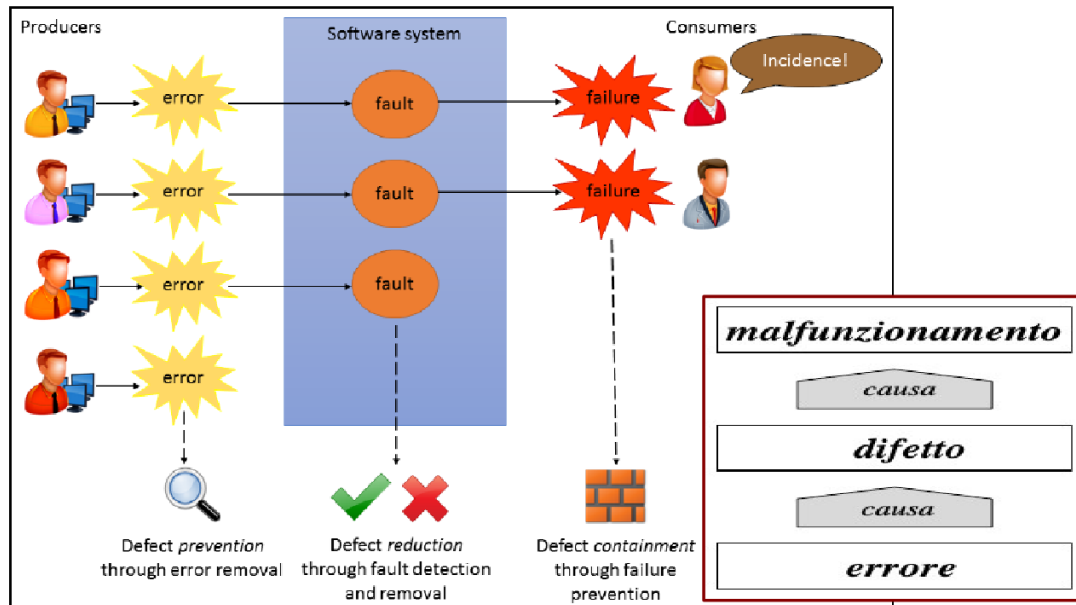


Figura 18.2: Relazione tra *error*, *fault* e *failure*: l'errore umano introduce un difetto nel sistema software, che può manifestarsi all'utente come malfunzionamento osservabile. La figura evidenzia anche tre strategie di intervento: prevenzione degli errori, rilevazione e rimozione dei fault, e contenimento delle failure.

### 18.4.1 Esempio makeItDouble

Si consideri la funzione:

```
int makeItDouble(int param) {
    int result;
    result = param * param;
    return(result);
}
```

La chiamata `makeItDouble(3)` restituisce 9, ma se la specifica richiede il raddoppio del parametro, il risultato atteso sarebbe 6.

In questo caso:

- 9 è la **failure**, cioè il risultato osservabile errato;
- `param * param` è il **fault**, cioè il difetto nel codice;
- l'**error** può essere un typo oppure un errore di interpretazione tra “double” e “power”.

## 18.5 Software Testing

Il **software testing** consiste nell'esecuzione di esperimenti in un ambiente controllato, con lo scopo di acquisire sufficiente fiducia sul funzionamento del sistema.

Il testing riguarda tipicamente aspetti funzionali, ma può riguardare anche caratteristiche extra-funzionali.

I principali vantaggi sono:

- verificare se il sistema risponde alle esigenze per cui è stato creato;
- rendere osservabili eventuali malfunzionamenti.

Il limite fondamentale è che il testing non può dimostrare l'assenza di difetti.

### IMPORTANTE PER L'ESAME

Secondo Dijkstra, il software testing può dimostrare la presenza di guasti, ma non la loro assenza. Se un test fallisce, sappiamo che esiste un problema; se tutti i test passano, sappiamo solo che non abbiamo trovato problemi nei casi testati.

## 18.6 Esempio myBuggyFact e dominio di input

La specifica dell'esempio richiede il calcolo del fattoriale su tutti gli interi, interpretando il fattoriale di un numero negativo come il fattoriale del suo opposto.

La funzione mostrata è:

```
int myBuggyFact(int p){
    int result = abs(p);
    if (result > 1){
        int next = result - 1;
        result = result * myBuggyFact(next);
    }
    return result;
}
```

Il dominio degli input è molto ampio: per un intero a 32 bit ci sono  $2^{32}$  possibili valori. L'esplorazione esaustiva è quindi impraticabile.

Per questo motivo occorre selezionare in modo intelligente solo alcune parti del dominio, ad esempio usando classi di equivalenza.

## 18.7 Domande fondamentali del testing

La progettazione dei test richiede di rispondere ad alcune domande fondamentali.

### 18.7.1 Quali e quanti input selezionare?

Poiché non è possibile testare tutto il dominio, bisogna scegliere valori rappresentativi. Una strategia consiste nel partizionare il dominio in classi di equivalenza rispetto a un certo obiettivo di test.

### 18.7.2 Quando smettere di testare?

Servono criteri di copertura e valori minimi di accettazione. Tra i criteri possibili ci sono:

- copertura degli statement;
- copertura degli archi;
- copertura delle condizioni;
- copertura dei flussi;
- copertura delle funzionalità.

### 18.7.3 Come sapere se il risultato è corretto?

Questo è l'**oracle problem**: per stabilire se un test passa o fallisce, bisogna conoscere l'output atteso.

#### Nota del Prof

L'oracolo può derivare da specifiche, dati storici, tabelle note o sistemi equivalenti già in esecuzione. Tuttavia, in generale, sapere automaticamente quale sia l'output corretto non è sempre banale.

## 18.8 Testing come campionamento

In generale, il dominio  $D$  è molto grande e l'esplorazione esaustiva non è praticabile. Inoltre, solo una piccola parte del dominio può causare errori.

Di conseguenza, il testing può essere visto come l'esecuzione del programma su un **campione** di dati di input.

Testing = eseguire un programma su un campione di input

Questo approccio permette di aumentare la confidenza nel sistema, ma non fornisce certezza assoluta di correttezza.

## 18.9 Correttezza vs Reliability

La **correttezza** di un programma indica la completa aderenza alle specifiche per ogni elemento del dominio di input. Può essere asserita tramite prove formali, oppure richiederebbe un testing esaustivo, impraticabile nella maggior parte dei casi.

La **reliability**, invece, è un attributo statistico del sistema. Indica la probabilità che un'interazione con il programma termini con successo, assumendo un certo profilo di utilizzo.

La formula è:

$$Reliability = 1 - PFD$$

dove  $PFD$  è la *Probability of Failure on Demand*.

## IMPORTANTE PER L'ESAME

Un sistema può avere alta reliability senza essere matematicamente corretto. La reliability dipende da come il sistema viene usato, mentre la correttezza richiede aderenza alle specifiche su tutto il dominio.

### 18.10 Operational Profile

L'**operational profile** è una descrizione numerica di come il sistema viene effettivamente utilizzato.

Può variare in funzione delle tipologie di utenti e può essere:

- **stimato**, tramite requisiti, specifiche e aspettative di sviluppatori o stakeholder;
- **misurato**, tramite monitoraggio delle richieste al sistema, log e report.

#### Nota del Prof

La stessa funzione può avere reliability diverse in base al profilo d'uso. Se gli utenti usano spesso input che attivano il fault, la reliability sarà bassa; se quegli input sono rari, la reliability percepita può essere alta.

#### 18.10.1 Esempi su myBuggyFact

Nel caso di **myBuggyFact**, il difetto si manifesta quando l'input è 0. Per questo motivo, la **reliability** dipende dalla probabilità con cui l'utente usa proprio quel valore.

Nel primo operational profile gli input sono:

$[-5, -3, 0, 2, 4, \text{othersNum}]$

con probabilità d'uso:

$[0.2, 0.2, 0.2, 0.2, 0.2, 0]$

Poiché l'input 0 ha probabilità 0.2, la probabilità di fallimento è:

$$PFD = 0.2$$

quindi:

$$Reliability = 1 - PFD = 1 - 0.2 = 0.8$$

Nel secondo operational profile gli input sono gli stessi, ma cambiano le probabilità d'uso:

$[0, 0, 0.002, 0.499, 0.499, 0]$

In questo caso l'input 0 ha probabilità molto bassa:

$$PFD = 0.002$$

quindi:

$$Reliability = 1 - PFD = 1 - 0.002 = 0.998$$

## IMPORTANTE PER L'ESAME

Lo stesso programma può avere reliability diverse a seconda dell'operational profile considerato. La reliability non misura la correttezza assoluta del codice, ma la probabilità che il programma funzioni correttamente rispetto al modo in cui viene effettivamente usato.

### 18.11 Collegamento con progetto ed esame

Il testing non dimostra l'assenza di bug, perché esplora solo una parte del dominio. Serve quindi progettare i test in modo ingegneristico, scegliendo input rappresentativi e criteri di copertura adeguati.

L'oracolo è necessario per stabilire se un test è passato o fallito, mentre l'operational profile è fondamentale per stimare l'affidabilità reale del sistema.

È importante non confondere i concetti:

- il testing osserva le **failure**;
- il debugging cerca e rimuove i **fault**;
- i fault derivano da **error** umani;
- la correctness è una proprietà assoluta rispetto alla specifica;
- la reliability è una proprietà statistica rispetto all'uso.

# Capitolo 19

## Test Automation e Continuous Testing

### 19.1 Software Quality Assurance

La **Software Quality Assurance** (SQA) è definita come l'insieme di attività sistematiche che forniscono evidenza dell'adeguatezza all'uso (*fitness*) del prodotto software complessivo.

I termini principali della definizione sono:

- **Systematic activities**: indica la presenza di un **processo** regolato;
- **Providing evidence**: indica la presenza di **punti di controllo** per dimostrare la conformità;
- **Fitness**: indica la presenza di **obiettivi misurabili** da raggiungere.

#### Nota del Prof

Un processo di SQA è un insieme di piani e attività per il monitoraggio e il controllo del processo di sviluppo software. Il suo scopo è assicurarsi che i vari attori stiano lavorando nella giusta direzione.

Gli obiettivi della SQA sono fornire sufficiente confidenza che il prodotto software:

- soddisfi le sue specifiche;
- sia utilizzabile nel contesto considerato;
- venga sviluppato secondo le modalità stabilite.

Le componenti della SQA includono **software testing**, **quality control**, **software configuration management**, standard, procedure, convenzioni e specifiche.

#### IMPORTANTE PER L'ESAME

Il **software testing** è solo una delle componenti della SQA. Le attività di testing devono essere sempre collegate a un piano di qualità più ampio che definisce cosa significhi qualità per quello specifico progetto.



Figura 19.1: Il software testing come una delle componenti della Software Quality Assurance: la figura evidenzia che il testing si inserisce in un quadro più ampio che comprende anche quality control, procedure e software configuration management.

## 19.2 Esempio myBuggyFact

Nelle slide viene mostrata una funzione ricorsiva per il calcolo del fattoriale:

```
int myBuggyFact(int p){
    int result = abs(p);
    if (result > 1){
        int next = result - 1;
        result = result * myBuggyFact(next);
    }
    return result;
}
```

Poiché il parametro in ingresso è un `int`, testare esaustivamente la procedura richiederebbe  $2^{32}$  possibili test.

### Nota del Prof

Rintracciare l'input che fa fallire la funzione tra  $2^{32}$  possibilità, senza supporto automatico, equivale a cercare un ago in un pagliaio. L'automazione è quindi fondamentale per eseguire validazioni in tempi ragionevoli.

## 19.3 Automated Testing

L'**automated testing** consiste nello svolgimento delle attività di test principalmente tramite infrastrutture o framework di supporto.

Le attività che possono essere automatizzate includono:

- esecuzione dei test;
- generazione dei report;
- selezione dei casi di test da eseguire;

- prioritizzazione dei test;
- valutazione della validità o bontà di un insieme di casi di test.

L'automated testing si contrappone al **manual testing**, in cui i test sono svolti da tester umani, membri del team di QA o utenti finali.

#### Nota del Prof

I framework automatici migliorano efficienza ed efficacia del testing: riducono costi e durata delle fasi di test, aumentano la ripetibilità degli esperimenti, migliorano l'attendibilità statistica e producono una documentazione più leggibile delle failure.

## 19.4 Continuous Testing e Continuous Integration

Il **Continuous Testing** consiste nell'integrare i processi di software testing con:

- ambienti di sviluppo;
- strumenti di build;
- sistemi di versioning.

I test vengono eseguiti automaticamente sulla versione corrente del sistema, sia in configurazioni locali allo sviluppo sia in ambienti simili a quelli di produzione.

#### Nota del Prof

Nel Continuous Testing lo sviluppo non si interrompe: a ogni evoluzione significativa della code-base si attiva una catena automatica di controlli.

Gli sviluppatori non devono avviare esplicitamente i test remoti, ma ricevono notifiche asincrone sui risultati.

#### Solo nelle slide (non spiegato a voce)

Le slide evidenziano la necessità di policy per selezione, prioritizzazione e orchestrazione dei casi di test da eseguire.

Il ciclo **CI + CT** può essere sintetizzato così:

Source control → Trigger → Build server → Report → Development

Il **build server** si occupa di:

- configurare un ambiente tipo;
- eseguire la build del sistema;
- eseguire i casi di test.

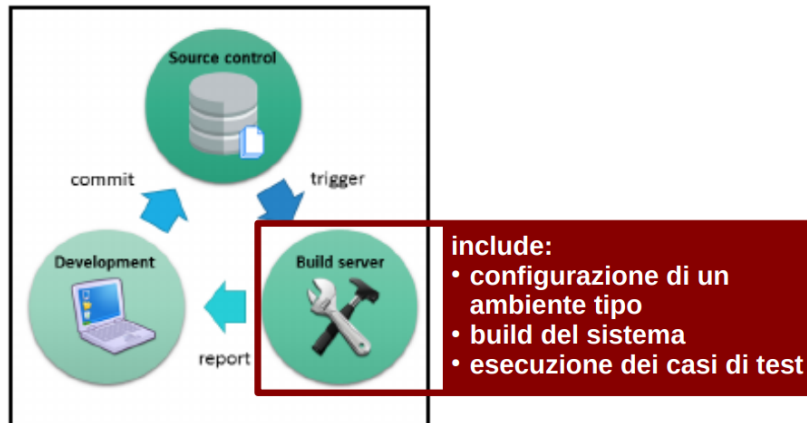


Figura 19.2: Schema concettuale di integrazione tra Continuous Integration e Continuous Testing: il build server configura l'ambiente, costruisce il sistema ed esegue automaticamente i casi di test.

## 19.5 Git e Source Control

**Git** è un sistema di controllo versione open-source e distribuito.

### Nota del Prof

Nel progetto si utilizza un approccio basato su **blessed repository**. Lo sviluppatore crea un **fork** del repository principale, lavora sul proprio repository locale, esegue il **push** sul fork remoto e propone l'integrazione tramite **Pull Request** all'integration manager.

I concetti principali sono:

- **local repository**: copia locale del repository;
- **fork**: copia personale del repository remoto;
- **push**: invio delle modifiche al repository remoto;
- **pull request**: richiesta di integrazione delle modifiche;
- **blessed repository**: repository principale;
- **integration manager**: responsabile dell'integrazione delle modifiche.

### IMPORTANTE PER L'ESAME

I controlli di qualità e la Continuous Integration vengono tipicamente attivati quando si apre una Pull Request o quando si effettua un push sul repository.

## 19.6 Maven

**Maven** è un sistema di build sviluppato da Apache Software Foundation, rivolto principalmente ad ambienti Java.

Uno dei suoi ruoli principali è la gestione delle dipendenze.

### Nota del Prof

La caratteristica fondamentale di Maven è la gestione esplicita e transitiva delle dipendenze. Lo sviluppatore dichiara le librerie necessarie e le loro versioni; Maven scarica automaticamente tali librerie e anche le dipendenze indirette richieste.

Un progetto Maven segue in genere una struttura standard di directory, pensata per distinguere chiaramente il codice applicativo, i test e i risultati prodotti dalla build.

La directory **target** contiene tipicamente i risultati della build.

Il file principale di configurazione è il **pom.xml**, che raccoglie le informazioni fondamentali del progetto e le direttive necessarie per gestirne dipendenze, build, test e deploy.

Le principali fasi Maven sono:

- `validate`;
- `compile`;
- `test`;
- `package`;
- `integration-test`;
- `verify`;
- `install`;
- `deploy`;
- `clean`;
- `site`.

### IMPORTANTE PER L'ESAME

Le fasi Maven sono sequenziali. Un errore in una fase, ad esempio un test fallito durante la fase **test**, comporta il fallimento dell'intero processo di build.

## 19.7 JUnit

**JUnit** è un framework open-source per l'implementazione di programmi di test in Java. È essenziale per rendere la build **self-testing**.

JUnit ha contribuito a rendere il test, soprattutto quello di unità, una parte centrale della programmazione e ha favorito tecniche come il **Test-Driven Development** (TDD).

Gli obiettivi principali di JUnit sono:

- supportare la definizione di casi di test utili;
- supportare test ripetibili nel tempo;
- ridurre i costi di scrittura dei test tramite pratiche orientate al riuso.

## IMPORTANTE PER L'ESAME

JUnit fornisce l'infrastruttura per scrivere ed eseguire test, ma **non** decide la strategia di test, **non** seleziona i valori di input e **non** indica quali test siano più significativi. Queste decisioni spettano all'ingegnere.

La versione di riferimento per il corso è **JUnit 4**, che si basa sull'uso di annotazioni per definire e organizzare i casi di test.

### Solo nelle slide (non spiegato a voce)

JUnit 5 introduce una maggiore modularità architetturale, permette la composizione esplicita di più runner e distingue tra API per scrivere test e SPI per estendere strategie di scoperta, scelta ed esecuzione dei test.

## 19.8 Annotazioni JUnit 4 e Maven

In JUnit 4, un caso di test è un metodo pubblico annotato con `@Test`.

Le annotazioni principali sono:

- `@Test`: identifica un metodo come caso di test;
- `@Before`: metodo eseguito prima di ogni test;
- `@After`: metodo eseguito dopo ogni test;
- `@BeforeClass`: metodo eseguito una sola volta prima di tutti i test della classe;
- `@AfterClass`: metodo eseguito una sola volta dopo tutti i test della classe.

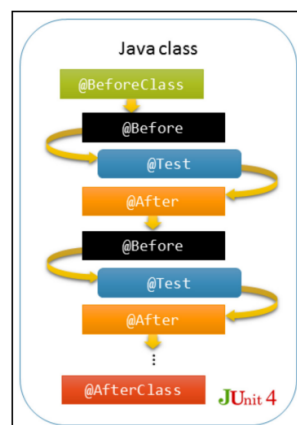


Figura 19.3: Ciclo di esecuzione di un test in JUnit 4: la figura distingue le operazioni di setup e teardown a livello di classe da quelle eseguite prima e dopo ogni singolo caso di test.

Solo nelle slide (non spiegato a voce)

L'ordine di esecuzione tra più metodi con la stessa annotazione, ad esempio più metodi `@Before`, non è specificato.

Il flusso logico di esecuzione è:

$$\text{@BeforeClass} \rightarrow (\text{@Before} \rightarrow \text{@Test} \rightarrow \text{@After}) \rightarrow \text{@AfterClass}$$

Affinché Maven riconosca ed esegua automaticamente i test durante la fase `test`, le classi di test devono rispettare specifici pattern di nomenclatura:

- `Test*`;
- `*Test`;
- `*Tests`;
- `*TestCase`.

### IMPORTANTE PER L'ESAME

Se una classe di test non rispetta i pattern riconosciuti da Maven, può essere ignorata durante la fase di test. In questo caso la CI potrebbe segnalare successo senza aver realmente eseguito quei test.

## 19.9 Framework CI/CD

Tra i principali framework di Continuous Integration e Continuous Delivery citati nelle slide ci sono:

- **Travis CI**;
- **GitHub Actions**;
- **GitLab CI/CD**;
- **CircleCI**.

Solo nelle slide (non spiegato a voce)

Questi framework sono offerti come servizi remoti e prevedono policy e costi di utilizzo. Spesso non ci sono costi per progetti open-source.

### IMPORTANTE PER L'ESAME

Per i progetti universitari open-source, i limiti degli account gratuiti sono generalmente sufficienti. Non è richiesta una sottoscrizione a pagamento.

Ogni framework richiede una configurazione specifica rispetto al sistema di versioning utilizzato, spesso tramite file di configurazione dedicati.

## 19.10 Visione complessiva e progetto

La visione complessiva collega **Git**, **Maven**, **JUnit** e un framework di **CI/CD**.

Il flusso tipico è:

1. lo sviluppatore lavora localmente;
2. usa Maven per compilare e JUnit per testare;
3. effettua il push sul proprio fork;
4. apre una Pull Request verso il repository principale;
5. la Pull Request attiva il server di Continuous Integration;
6. il server scarica le dipendenze tramite Maven;
7. il server esegue la build;
8. il server lancia i test JUnit;
9. viene prodotto un report di successo o fallimento.

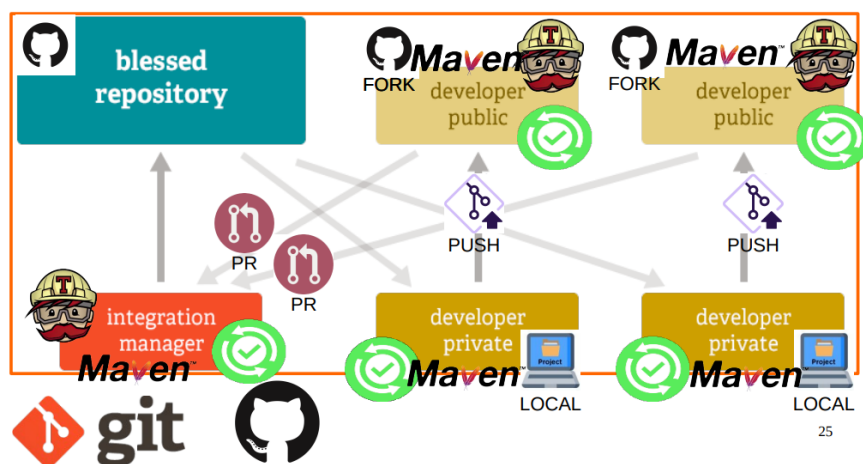


Figura 19.4: Visione complessiva del flusso di Continuous Integration e Continuous Testing: commit, push, pull request, build automatica ed esecuzione dei test concorrono a mantenere sotto controllo la qualità del software.

### Nota del Prof

L'automazione serve a verificare che il codice non funzioni solo sulla macchina dello sviluppatore, ma anche in ambienti puliti, riproducibili ed eterogenei.

Gli errori da evitare nel progetto sono:

- configurare in modo errato il `pom.xml`;
- usare nomi di classi di test non riconosciuti da Maven;
- lasciare dipendenze mancanti;

- avere una build non riproducibile;
- non integrare automaticamente i test nella build.

# Capitolo 20

## Unit Testing, Integration Testing, JUnit, Mockito e Maven

### 20.1 Dimensioni del testing

Le attività di software testing possono essere classificate lungo diverse dimensioni, tra loro **ortogonali**. Questo significa che ogni dimensione descrive un aspetto diverso del test e può essere combinata liberamente con le altre.

Le principali dimensioni sono:

- **Livello del test:** Unit, Integration, System, Acceptance;
- **Metodo di test:** Black-box, White-box, Non-functional;
- **Tipo di test:** Manual, Automated.

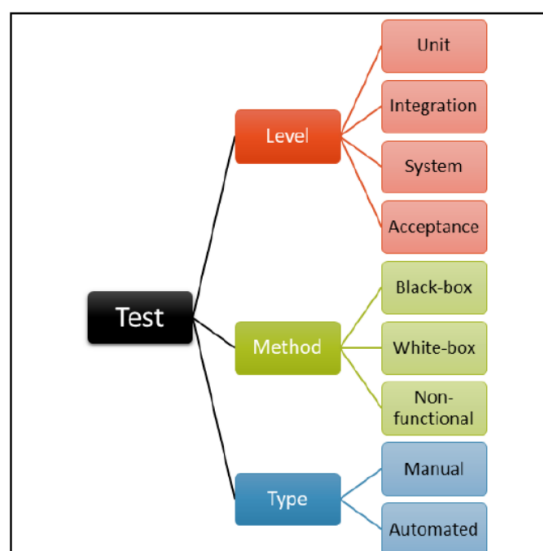


Figura 20.1: Dimensioni ortogonali del software testing: un test può essere classificato in base al livello, al metodo e al tipo. Queste dimensioni sono indipendenti e possono combinarsi tra loro, ad esempio in un test di unità automatico white-box o in un test di sistema manuale black-box.

### Nota del Prof

Le dimensioni sono indipendenti. Non bisogna pensare che, ad esempio, i test automatici siano sempre white-box o che i test manuali siano sempre black-box. Sono classificazioni diverse che descrivono proprietà diverse del test.

Nel metodo **black-box**, il sistema viene considerato come una scatola nera: il tester conosce input, output attesi e specifiche dell'interfaccia, ma non usa direttamente la conoscenza dell'implementazione interna.

Nel metodo **white-box**, invece, il tester conosce la struttura interna del codice e può usare questa conoscenza per progettare casi di test mirati, ad esempio per coprire specifici rami, condizioni o statement.

## 20.2 Livelli di test e attività di V&V

I livelli di test possono essere rappresentati come una piramide:

Unit → Integration → System → Acceptance

Alla base ci sono i **test di unità**, che sono numerosi, relativamente economici e più facili da automatizzare. Salendo nella piramide, i test diventano progressivamente più costosi, meno numerosi e spesso meno facilmente automatizzabili.

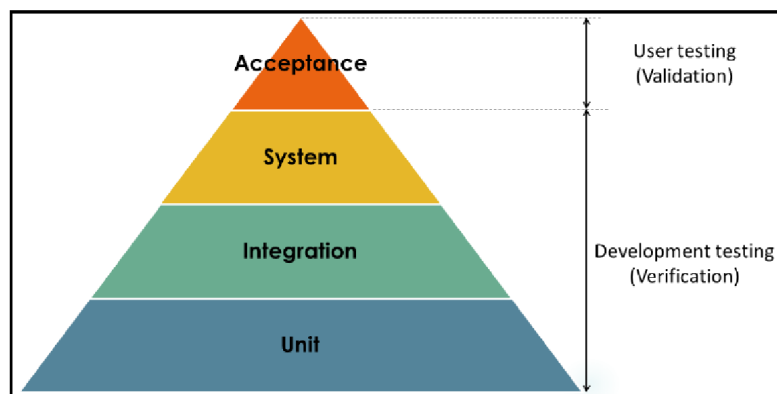


Figura 20.2: Piramide dei livelli di test: i test di unità, integrazione e sistema rientrano principalmente nel development testing, orientato alla verifica; i test di accettazione rientrano invece nello user testing, orientato alla validazione.

### Solo nelle slide (non spiegato a voce)

La piramide dei test distingue tra **Development Testing**, associato prevalentemente alla verifica, e **User Testing**, associato prevalentemente alla validazione.

I livelli inferiori, cioè unit, integration e system testing, rientrano tipicamente nelle attività di **verification**. Il livello di acceptance testing è invece più legato alla **validation**.

- **Verification:** controlla se il prodotto è stato costruito correttamente rispetto alle specifiche. Risponde alla domanda: *Are we building the product right?*

- **Validation:** controlla se il prodotto costruito è effettivamente quello desiderato dall'utente o dal committente. Risponde alla domanda: *Are we building the right product?*

#### Nota del Prof

La verifica guarda alla conformità rispetto alle specifiche tecniche. La validazione guarda invece alla corrispondenza tra il prodotto finale e le reali aspettative dell'utente.

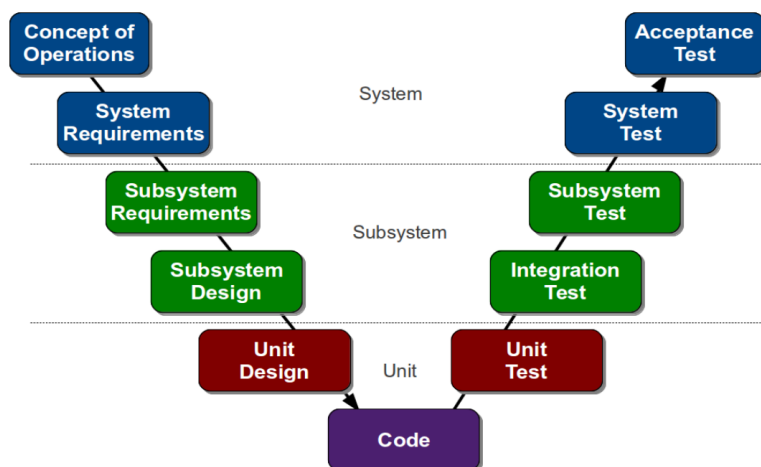
## 20.3 V-model e Continuous Testing

Il **V-model** mette in relazione le fasi di specifica e progettazione con le corrispondenti fasi di test.

Nel ramo discendente della V si definiscono requisiti, design e codice. Nel ramo ascendente si progettano ed eseguono i test corrispondenti.

Le corrispondenze principali sono:

- **Unit Design**  $\leftrightarrow$  **Unit Test**;
- **Subsystem Design / Subsystem Requirements**  $\leftrightarrow$  **Integration Test**;
- **System Requirements**  $\leftrightarrow$  **System Test**;
- **Concept of Operations**  $\leftrightarrow$  **Acceptance Test**.



4

Figura 20.3: V-model: relazione tra le fasi di specifica e progettazione, sul ramo discendente, e i corrispondenti livelli di test, sul ramo ascendente. Ogni livello di progettazione trova una controparte nei test: unit, integration, system e acceptance test.

#### Solo nelle slide (non spiegato a voce)

Il diagramma del V-model mostra che la progettazione dei test non dovrebbe essere vista come un'attività finale, ma come un processo collegato alle specifiche e al design delle funzionalità.

Nel contesto del **Continuous Testing**, i casi di test vengono eseguiti automaticamente in momenti rilevanti del ciclo di sviluppo, ad esempio:

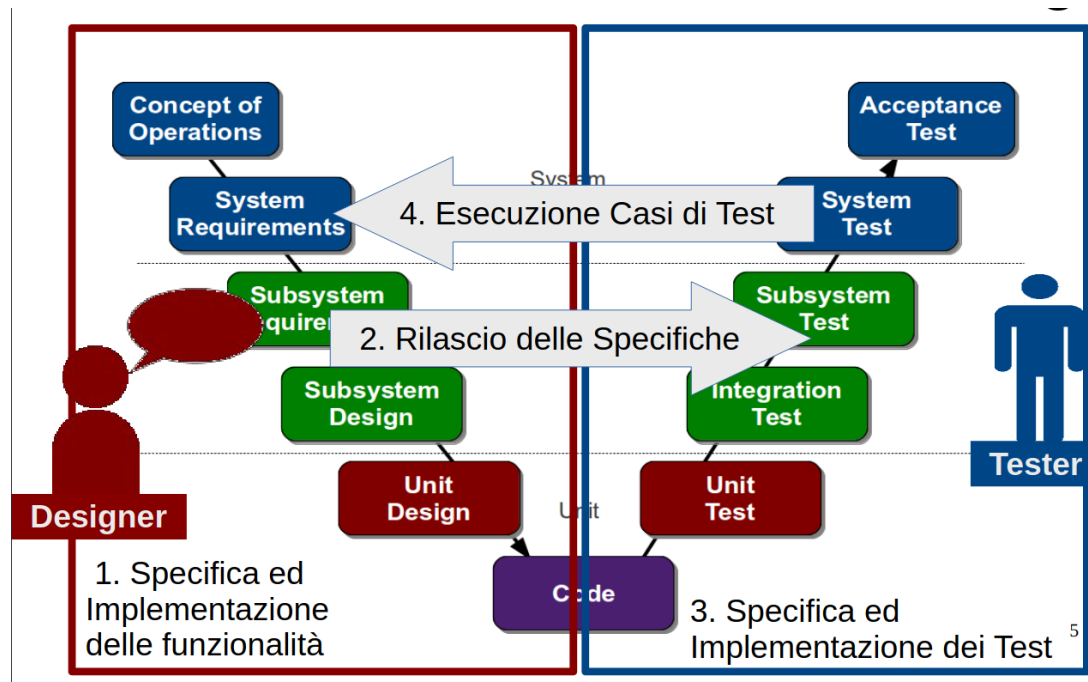


Figura 20.4: V-model nel contesto del Continuous Testing: il designer specifica e implementa le funzionalità, rilascia le specifiche al tester, il tester progetta e implementa i casi di test, che vengono poi eseguiti automaticamente durante il ciclo di sviluppo.

- a ogni proposta di modifica, come commit o Pull Request;
- al rilascio di una nuova funzionalità;
- al rilascio di una nuova versione.

#### Nota del Prof

Il processo di testing vive in parallelo allo sviluppo. Il designer produce specifiche e funzionalità; il tester usa tali informazioni per progettare casi di test. In un contesto continuo, i test vengono rieseguiti automaticamente ogni volta che il sistema cambia.

## 20.4 Test di unità

I **test di unità** hanno l'obiettivo di rivelare malfunzionamenti relativi a un singolo modulo o a una singola unità funzionale del **System Under Test**.

Lo scopo è acquisire confidenza che ogni unità, se eseguita in isolamento, si comporti come atteso.

Una unità può essere, a seconda del contesto:

- una funzione;
- un metodo;

- una classe;
- un modulo;
- un package;
- una piccola unità funzionale del sistema.

Generalmente i test di unità sono definiti direttamente dagli sviluppatori delle unità del SUT.

Possono essere progettati secondo due prospettive:

- **White-box:** usando come riferimento l'implementazione realizzata;
- **Black-box:** usando come riferimento la funzionalità astratta da realizzare, il modello comportamentale o gli esempi descritti dalle specifiche.

#### Solo nelle slide (non spiegato a voce)

Le slide richiamano anche il **Test Driven Development** (TDD), in cui i test possono essere sviluppati prima della scrittura del codice.

L'adeguatezza di una suite di unit test può essere valutata in funzione di:

- numero di funzionalità controllate;
- numero di requisiti controllati;
- numero di aspetti rilevabili dalle specifiche;
- metriche di copertura del codice sorgente.

#### IMPORTANTE PER L'ESAME

Il test di unità verifica che una singola unità funzioni correttamente in isolamento. Non è pensato per verificare la correttezza dell'interazione tra più moduli: quello è compito del test di integrazione.

## 20.5 JUnit: obiettivi e filosofia

**JUnit** è un framework che supporta l'implementazione di unit test in ambienti Java.

I principali obiettivi di JUnit sono:

- supportare la definizione di casi di test utili;
- supportare la definizione di casi di test ripetibili nel tempo;
- ridurre i costi di scrittura dei casi di test tramite pratiche orientate al riuso del codice.

JUnit non è un linguaggio separato, ma un framework Java basato su classi, metodi, annotazioni e asserzioni.

## IMPORTANTE PER L'ESAME

JUnit è uno strumento per **implementare** ed **eseguire** casi di test e test suite. Non decide quale strategia di test usare, quali test siano più significativi o quali valori di input selezionare.

Questo significa che JUnit fornisce l'infrastruttura esecutiva, ma la qualità del test dipende comunque dal lavoro del progettista dei test.

## 20.6 Esempi JUnit: test parametrizzati

I **test parametrizzati** permettono di eseguire lo stesso test più volte con coppie o tuple diverse di input e output attesi.

In JUnit 4, una classe di test parametrizzata può usare:

```
@RunWith(Parameterized.class)
```

Questa annotazione indica a JUnit che la classe deve essere eseguita tramite il runner parametrizzato.

La struttura tipica prevede:

- attributi privati per memorizzare i parametri del test;
- un metodo annotato con `@Parameters`;
- un costruttore con tanti argomenti quanti sono i parametri;
- uno o più metodi annotati con `@Test`.

Il metodo annotato con `@Parameters` deve essere statico e restituire una `Collection` contenente le tuple di valori da usare nei test.

### Nota del Prof

Il metodo `@Parameters` deve essere statico perché JUnit deve recuperare i parametri prima di creare le istanze della classe di test.

Un esempio schematico è:

```
@RunWith(Parameterized.class)
public class ParameterizedTest {

    private double expected;
    private double valueOne;
    private double valueTwo;

    @Parameters
    public static Collection<Integer[]> getTestParameters() {
        return Arrays.asList(new Integer[][] {
            {2, 1, 1},
            {3, 2, 1},
        });
    }
}
```

```

        {4, 3, 1}
    });
}

public ParameterizedTest(double expected,
                        double valueOne,
                        double valueTwo) {
    this.expected = expected;
    this.valueOne = valueOne;
    this.valueTwo = valueTwo;
}

@Test
public void sum() {
    Calculator calc = new Calculator();
    assertEquals(expected, calc.add(valueOne, valueTwo), 0);
}
}

```

#### Solo nelle slide (non spiegato a voce)

Nell'esempio delle slide, il test parametrizzato usa il runner JUnit, parametri come attributi privati, un metodo di configurazione che restituisce una `java.util.Collection` e un costruttore con tanti argomenti quanti sono i parametri.

## 20.7 Pre-testing e post-testing globale

JUnit fornisce annotazioni per definire azioni da eseguire prima o dopo i test.

Le annotazioni globali sono:

- `@BeforeClass`: metodo eseguito una sola volta prima di tutti i test della classe;
- `@AfterClass`: metodo eseguito una sola volta dopo tutti i test della classe.

Questi metodi vengono usati per operazioni di setup o teardown globale, come initialize risorse condivise o chiudere risorse alla fine dell'intera suite.

#### Solo nelle slide (non spiegato a voce)

Le slide precisano che non è specificato un ordine di esecuzione tra più metodi annotati con `@BeforeClass`, né tra più metodi annotati con `@AfterClass` ovvero è **NON deterministico**.

Le annotazioni locali sono invece:

- `@Before`: metodo eseguito prima di ogni singolo test;
- `@After`: metodo eseguito dopo ogni singolo test.

### Nota del Prof

`@BeforeClass` e `@AfterClass` lavorano una volta per tutta la classe. `@Before` e `@After` lavorano invece prima e dopo ogni singolo test, garantendo maggiore isolamento tra i casi di test.

Il flusso logico può essere rappresentato così:

$$\text{@BeforeClass} \rightarrow (\text{@Before} \rightarrow \text{@Test} \rightarrow \text{@After}) \rightarrow \text{@AfterClass}$$

## 20.8 Designer dei test e programmatore dei test

Nel processo di testing è importante distinguere tra due responsabilità logiche:

- **designer dei casi di test;**
- **programmatore dei casi di test.**

Il **designer** identifica dettagliatamente i valori attesi da un test. Può farlo usando:

- valori puntuali;
- specifiche;
- un oracolo;
- una funzione di predizione;
- un sistema esterno equivalente.

Il **programmatore** implementa invece le scelte progettuali attraverso istruzioni di verifica e controllo, ad esempio usando `assertEquals`, `assertTrue` o altre asserzioni.

### IMPORTANTE PER L'ESAME

Un test senza valori attesi e senza assert significative è incompleto. Chiamare un metodo senza verificare il risultato non basta: bisogna stabilire un oracle e codificarlo nel test.

### Nota del Prof

Nello unit testing, designer e programmatore dei test spesso coincidono con lo sviluppatore. Tuttavia, concettualmente sono ruoli diversi: il designer decide cosa testare e quale risultato aspettarsi; il programmatore decide come codificare la verifica.

## 20.9 Test di integrazione

I **test di integrazione** hanno l'obiettivo di far emergere malfunzionamenti derivanti dall'interazione tra due o più moduli o componenti del SUT.

Questi test servono ad acquisire confidenza sul fatto che:

- esistano le dovute interconnessioni tra moduli o componenti;
- le interazioni rispettino i contratti funzionali;
- i protocolli e le specifiche di interazione siano rispettati.

### Nota del Prof

Un esempio classico è un sensore di temperatura progettato in Europa che invia gradi Celsius a una centralina sviluppata negli Stati Uniti che si aspetta gradi Fahrenheit. I due moduli possono funzionare perfettamente se testati isolatamente, ma fallire quando vengono integrati.

L'assunzione alla base dell'integration testing è che ogni modulo o unità funzionale, preso in isolamento, si comporti già correttamente. In altre parole, si assumono superati i test di unità.

I test di integrazione possono essere scritti da:

- sviluppatori;
- integratori;
- tester.

La scelta dipende dagli scenari di integrazione considerati.

I test possono essere progettati:

- in modalità **white-box**, assumendo disponibilità del codice dei moduli;
- in modalità **black-box**, assumendo disponibilità solo delle specifiche di interazione;
- in modalità mista, che nella pratica è molto frequente.

### Nota del Prof

Nella pratica spesso si conosce bene il proprio modulo, ma si trattano le dipendenze esterne come scatole nere. Per questo l'approccio reale è spesso misto.

### IMPORTANTE PER L'ESAME

Il test di integrazione non serve a ritestare la logica interna del singolo modulo. Serve a verificare che più moduli comunichino correttamente e rispettino i contratti stabiliti.

## 20.10 Stub, mock e test driver

Nei test incrementali si considerano solo parti del SUT. Di conseguenza, alcune dipendenze potrebbero:

- non essere ancora disponibili;
- essere troppo costose da usare;

- essere lente o instabili;
- non dover essere realmente invocate nello specifico test.

Per questo motivo si usano stub, mock e test driver.

### 20.10.1 Stub e mock

Uno **stub** è un elemento software definito in fase di test. Può essere una classe, un modulo, una unità o un componente fittizio.

Le sue caratteristiche sono:

- non è soggetto a test;
- fa parte dell'ambiente di test;
- simula il comportamento di una unità mancante;
- spesso restituisce comportamenti banali, costanti o controllati.

Il termine **mock** viene spesso usato per indicare un oggetto finto che simula una dipendenza e consente anche di verificare le interazioni avvenute.

#### Nota del Prof

Se si vuole testare un componente che dipende da un database, può essere utile usare un mock del database. In questo modo si verifica la logica del componente senza dipendere realmente dal database.

### 20.10.2 Test driver

Il **test driver** è l'esecutore del test. Invoca i moduli coinvolti e spesso codifica parte della logica di integrazione che si vuole ottenere dai moduli composti.

Un driver semplice si limita a chiamare il codice da testare. Un driver più avanzato può occuparsi anche di:

- configurazione dell'ambiente di test;
- coordinamento delle risorse;
- clean-up dopo l'esecuzione;
- gestione del contesto necessario al test.

#### IMPORTANTE PER L'ESAME

Stub e mock simulano dipendenze richieste dal SUT. Il test driver, invece, guida l'esecuzione del test e coordina l'interazione tra i componenti.

# Capitolo 21

## Unit and Integration Testing: Frameworks

### 21.1 Test di integrazione

Il **test di integrazione** ha l'obiettivo di far emergere malfunzionamenti derivanti dall'interazione tra due o più moduli o componenti del **System Under Test** (SUT).

In particolare, serve ad acquisire confidenza che:

- esistano le dovute interconnessioni tra i moduli o componenti del SUT;
- le interazioni rispettino i **contratti funzionali** stabiliti dai protocolli o dalle specifiche di interazione;
- i moduli comunichino correttamente quando vengono composti.

L'assunzione di partenza è che ogni modulo, testato in isolamento, si comporti correttamente. In altre parole, i **test di unità** devono essere già stati superati.

I test di integrazione possono essere progettati secondo approcci diversi:

- **white-box**, se si conosce il codice dei moduli coinvolti;
- **black-box**, se ci si basa solo sulle specifiche di interazione;
- **misto**, situazione molto frequente nella pratica.

#### Nota del Prof

Nella pratica si usano spesso approcci misti. Da un lato serve conoscere l'API esposta, quindi il contratto visibile dall'esterno; dall'altro può essere necessario conoscere alcune meccaniche interne per configurare correttamente l'interazione tra componenti.

Poiché il test di integrazione è spesso incrementale, non sempre tutti i moduli sono disponibili o desiderabili nell'ambiente di test. Per questo motivo si usano **stub**, **mock** e **test driver**.

## 21.2 Stub, mock e test driver

Uno **stub** o **mock** simula il comportamento di un'unità mancante, non disponibile o non considerata nel test. Spesso tale comportamento è semplice, costante o controllato artificialmente.

Uno stub o mock:

- non è il soggetto del test;
- fa parte dell'ambiente di test;
- simula una dipendenza richiesta dal SUT;
- permette di isolare l'interazione che si vuole verificare.

### Nota del Prof

Se si vuole testare un componente che dipende da un database, può essere utile usare un mock del database. In questo modo si verifica la logica del componente senza dipendere realmente da un database esterno, che potrebbe essere lento, instabile o difficile da configurare.

Un **test driver** è invece un elemento software che esegue o coordina il test, configurando l'ambiente, preparando le risorse, attivando le interazioni e svolgendo operazioni di clean-up dopo l'esecuzione.

Un test driver avanzato può occuparsi anche di:

- configurazione dell'ambiente di test;
- inizializzazione delle risorse;
- coordinamento tra componenti;
- clean-up dopo l'esecuzione;
- teardown dell'ambiente.

### IMPORTANTE PER L'ESAME

Stub e mock rispondono simulando dipendenze richieste dal SUT. Il test driver, invece, guida l'esecuzione del test e coordina l'interazione tra i componenti.

## 21.3 Strategie di integration testing

Identificare una strategia ottima di integrazione è, in generale, un problema difficile.

### Solo nelle slide (non spiegato a voce)

Le slide indicano che la scelta ottima può essere formulata come un problema **NP-complete**.

Le strategie comuni sono:

- **big bang**;
- **top-down**;
- **bottom-up**.

### 21.3.1 Big bang

La strategia **big bang** consiste nell'integrare tutte le componenti in un unico passo e testare il sistema risultante.

#### Nota del Prof

È la soluzione più semplice da descrivere, ma spesso è sconsigliata. Se il sistema fallisce dopo aver integrato tutto insieme, diventa molto difficile capire quale interazione abbia causato il problema. Inoltre, più difetti possono mascherarsi a vicenda.

#### IMPORTANTE PER L'ESAME

Il limite principale del big bang è la bassa diagnosabilità: quando qualcosa non funziona, è difficile capire se il problema dipenda da un singolo componente, da una specifica interfaccia o dall'interazione tra più moduli.

### 21.3.2 Top-down

La strategia **top-down** parte dalle interfacce esposte e dalle funzionalità più generali, cioè quelle più vicine ai confini del SUT.

Le sequenze di test sono definite a partire dagli scenari d'uso del sistema. Il raffinamento avviene progressivamente:

- partendo dai moduli di livello superiore;
- simulando i moduli inferiori non ancora disponibili tramite stub;
- sostituendo gli stub con implementazioni reali;
- aggiungendo moduli interni via via più dettagliati;
- ampliando il sottoinsieme del sistema considerato.

#### Solo nelle slide (non spiegato a voce)

Nel top-down, gli **stub** simulano comportamenti o funzionalità mancanti dei moduli più profondi.

#### Nota del Prof

Nel top-down si parte dall'alto, cioè dalle funzionalità più visibili, e si scende progressivamente verso i dettagli interni. Gli stub vengono rimpiazzati quando i componenti reali diventano disponibili.

Il vantaggio del top-down è che consente di verificare presto il comportamento generale del sistema. Il limite è che può richiedere molti stub, soprattutto quando i moduli inferiori sono numerosi o complessi.

### 21.3.3 Bottom-up

La strategia **bottom-up** parte dalle unità elementari, cioè dai componenti più interni o lontani dai bordi del sistema.

In questo caso il raffinamento avviene considerando progressivamente interfacce più generali o parti più periferiche del sistema. I moduli superiori non ancora disponibili vengono simulati tramite **test driver**.

**Solo nelle slide (non spiegato a voce)**

Nel bottom-up, i **test driver** simulano il comportamento di moduli o componenti più esterni del sistema. Se alcune funzionalità mancanti devono comunque essere richiamate, possono essere emulate tramite stub.

Il vantaggio del bottom-up è che si costruisce progressivamente fiducia sui componenti di base. Il limite è che il comportamento complessivo del sistema emerge più tardi, perché le interfacce più alte vengono integrate solo nelle fasi successive.

### 21.3.4 Spiegazione dei diagrammi

**Solo nelle slide (non spiegato a voce)**

Il diagramma delle slide mostra una gerarchia di moduli: C1 è il modulo più alto, che interagisce con C2 e C3; questi a loro volta portano verso moduli più interni come C4 e C5.

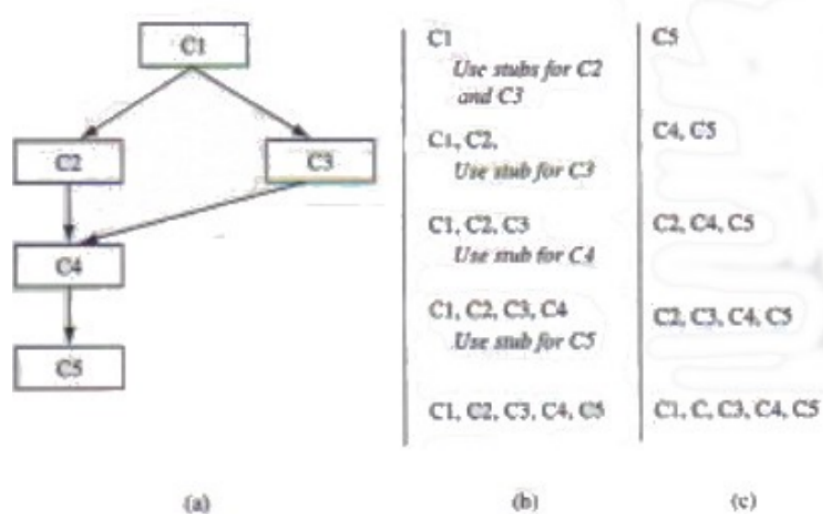


Figura 21.1: Principali strategie di integration testing. La figura mette a confronto approccio big bang, top-down e bottom-up, evidenziando il ruolo di stub e test driver nel raffinamento progressivo dell'integrazione.

Nel caso **top-down**, si parte da C1 e si simulano inizialmente C2 e C3 con stub. Successivamente si introducono C2 e C3 reali, continuando a simulare i moduli più profondi, fino ad arrivare al sistema completo.

Nel caso **bottom-up**, si parte dai moduli più bassi, ad esempio C5 o C4-C5, usando test driver per simulare i livelli superiori. Si risale poi gradualmente verso C1.

#### IMPORTANTE PER L'ESAME

Non esiste una strategia sempre migliore. La scelta dipende dalla struttura del sistema, dalla disponibilità dei componenti, dal costo di creare stub e driver, e dagli obiettivi del test.

## 21.4 Test di integrazione in sistemi Object-Oriented

Nei sistemi object-oriented, come quelli Java, i comportamenti emergono spesso dallo scambio di messaggi tra oggetti. Per questo sono utili strumenti che permettano di controllare e simulare tali interazioni.

Gli strumenti principali sono:

- **JUnit**, usato come test driver;
- **Mockito**, usato per stubbing e mocking.

#### Nota del Prof

Nei sistemi a oggetti può essere scomodo o costoso costruire manualmente tutte le dipendenze reali. Per esempio, invece di implementare un finto database da zero, si può usare un mock che simula solo il comportamento necessario al test.

#### Solo nelle slide (non spiegato a voce)

Gli strumenti di test supportano il testing incrementale, permettendo di isolare specifiche interazioni del SUT e di emulare componenti mancanti dell'ambiente.

Simulare collaboratori esterni consente di testare una specifica interazione senza dover istanziare l'intera rete di oggetti coinvolti.

## 21.5 Mockito

**Mockito** è un framework Java per la definizione di stub e mock a supporto dello sviluppo di test di unità e di integrazione.

Mockito permette di:

- dichiarare mock e stub;
- definire valori di ritorno per operazioni su specifiche istanze;
- ridefinire comportamenti in base agli input;
- verificare se certe interazioni sono avvenute;
- supportare pratiche di sviluppo guidate dai test, come TDD e BDD.

## 21.5.1 Costrutti principali

I principali costrutti di Mockito sono:

- `mock()`: crea un mock programmaticamente;
- `spy()`: crea una spy, cioè un oggetto reale osservabile e parzialmente controllabile;
- `@Mock`: dichiara un mock tramite annotazione;
- `@Spy`: dichiara una spy tramite annotazione;
- `@Captor`: cattura argomenti passati ai mock;
- `@InjectMocks`: inietta automaticamente mock dentro l'oggetto sotto test;
- `thenReturn`: definisce un valore di ritorno fisso;
- `given(...).willReturn(...)`: variante in stile BDD;
- `thenAnswer`: definisce un comportamento dinamico;
- `verify`: controlla che una certa interazione sia avvenuta;
- `then(...).should(...)`: variante BDD della verifica;
- `MockitoJUnitRunner` e `MockitoJUnit.rule()`: integrazione con JUnit.

### Nota del Prof

Un mock sostituisce il comportamento dell'oggetto configurandone le risposte. Una spy, invece, mantiene il comportamento reale dell'oggetto, ma permette di osservare e verificare le chiamate ai suoi metodi.

### Nota del Prof

`@InjectMocks` è utile quando si vuole testare un oggetto reale, ma sostituire automaticamente alcune sue dipendenze interne con mock controllati dal test.

### Nota del Prof

`thenAnswer` è utile quando una risposta fissa non basta. Permette di definire una logica dinamica che genera il risultato in base agli argomenti ricevuti.

## 21.5.2 Esempi Mockito

### Solo nelle slide (non spiegato a voce)

Le slide rimandano ai file `SimpleAgendaTest.java` e `AgendaTest.java` come esempi di uso di Mockito.

Nel caso di `SimpleAgendaTest`, il test lavora direttamente su un'interfaccia. Si crea un mock dell'agenda e si configura il comportamento atteso per determinate chiamate.

### Nota del Prof

Per esempio, si può configurare il mock in modo che, quando viene chiamato `getAppointment` con una certa data, restituisca una stringa prefissata. Con `thenAnswer`, invece, si può definire una risposta dinamica basata sull'input.

Nel caso di `AgendaTest`, si testa un'implementazione reale, come `MyAgenda`, ma si sostituisce una parte del suo stato interno, ad esempio una `Map` degli appuntamenti, tramite mock.

### Nota del Prof

In questo modo il test esercita le API reali di `MyAgenda`, ma controlla artificialmente una dipendenza interna, rendendo l'ambiente di test più prevedibile.

### IMPORTANTE PER L'ESAME

Mockito non serve a testare il mock in sé. Serve a controllare il comportamento del SUT quando interagisce con dipendenze simulate.

## 21.6 Maven nel testing

**Maven** è un sistema di build e gestione delle dipendenze sviluppato da Apache. È molto usato nei progetti Java ed è basato su un'architettura modulare a plug-in.

Maven gestisce:

- compilazione;
- dipendenze;
- test;
- packaging;
- installazione e deploy;
- plug-in collegati al ciclo di build.

### Nota del Prof

Maven gestisce automaticamente le dipendenze in modo transitivo. Lo sviluppatore dichiara le librerie nel `pom.xml`; Maven scarica da Maven Central sia le librerie indicate sia le loro sotto-dipendenze, mettendole in cache nella directory locale `.m2`.

Il file `pom.xml` descrive il progetto e contiene:

- coordinate del progetto;
- dipendenze;
- plug-in;
- configurazioni di build;

- informazioni sui moduli.

Le principali fasi del ciclo di build Maven sono:

- `validate`;
- `compile`;
- `test`;
- `package`;
- `integration-test`;
- `verify`;
- `install`;
- `deploy`;
- `clean`;
- `site`.

### IMPORTANTE PER L'ESAME

Le fasi Maven sono sequenziali. Un errore in una fase, per esempio un test fallito durante `test`, può interrompere la build e impedire l'esecuzione delle fasi successive.

## 21.7 Maven Surefire Plugin

Il **Maven Surefire Plugin** è pensato principalmente per eseguire i **test di unità**.

Viene attivato durante la fase:

`test`

del ciclo di build Maven.

Surefire scansiona il progetto Maven cercando test JUnit, tipicamente nella directory:

`${basedir}/src/test/`

I test vengono riconosciuti se contengono metodi pubblici annotati con `@Test` e se le classi rispettano determinati pattern di nomenclatura:

- `Test*`;
- `*Test`;
- `*Tests`;
- `*TestCase`.

I report vengono generalmente generati in:

`${basedir}/target/surefire-reports/TEST-*.xml`

Solo nelle slide (non spiegato a voce)

Le slide mostrano una configurazione del `pom.xml` in cui il plugin `maven-surefire-plugin` è dichiarato sotto `pluginManagement`. È inoltre mostrata la dipendenza da JUnit con `scope` impostato a `test`.

### IMPORTANTE PER L'ESAME

Se una classe di test non rispetta i pattern riconosciuti da Surefire, Maven potrebbe ignorarla. Questo può produrre una falsa sensazione di successo, perché la build passa senza aver realmente eseguito quel test.

## 21.8 Maven Failsafe Plugin

Il **Maven Failsafe Plugin** è pensato per eseguire i **test di integrazione**.

Viene attivato durante le fasi:

`integration-test` e `verify`

Failsafe è utile quando i test richiedono attività più complesse, come:

- build di più moduli Maven;
- configurazione di un ambiente dedicato;
- avvio di container applicativi, come Tomcat o Jetty;
- deploy e lancio di servizi o moduli;
- clean-up e teardown dell'ambiente dopo l'esecuzione.

Failsafe riconosce per default classi di test con pattern:

- `IT*`;
- `*IT`;
- `*ITCase`.

I report vengono generalmente generati in:

`${basedir}/target/failsafe-reports/failsafe-summary.xml`

Solo nelle slide (non spiegato a voce)

Le slide mostrano una configurazione del `pom.xml` in cui il plugin `maven-failsafe-plugin` associa i goal `integration-test` e `verify` all'interno della sezione `executions`.

## 21.9 Surefire vs Failsafe

La distinzione principale è:

- **Surefire**: esegue unit test nella fase `test`;
- **Failsafe**: esegue integration test nelle fasi `integration-test` e `verify`.

```
1. <project>
2.   [...]
3.   <build>
4.     <plugins>
5.       <plugin>
6.         <groupId>org.apache.maven.plugins</groupId>
7.         <artifactId>maven-failsafe-plugin</artifactId>
8.         <version>3.0.0-M4</version>
9.         <executions>
10.          <execution>
11.            <goals>
12.              <goal>integration-test</goal>
13.              <goal>verify</goal>
14.            </goals>
15.          </execution>
16.        </executions>
17.      </plugin>
18.    </plugins>
19.  </build>
20.  [...]
21. </project>
```

```
1. <dependencies>
2.   [...]
3.   <dependency>
4.     <groupId>junit</groupId>
5.     <artifactId>junit</artifactId>
6.     <version>4.11</version>
7.   </dependency>
8.   [...]
9. </dependencies>
```

Figura 21.2: Configurazione del Maven Failsafe Plugin per l'esecuzione dei test di integrazione durante le fasi `integration-test` e `verify` del ciclo di build Maven.

### Nota del Prof

Quando si esegue `mvn test`, Maven attiva i test gestiti da Surefire. Quando si esegue `mvn verify`, Maven arriva anche alle fasi pensate per i test di integrazione e può quindi attivare Failsafe.

Separare unit test e integration test è importante perché hanno costi e scopi diversi. I test di unità dovrebbero essere rapidi e isolati, mentre i test di integrazione possono richiedere ambienti più pesanti, moduli multipli, servizi esterni o container.

### IMPORTANTE PER L'ESAME

Chiamare un integration test con un nome compatibile con Surefire, ad esempio `PippoTest.java`, può farlo eseguire nella fase sbagliata. Per i test di integrazione conviene usare pattern come `*IT.java`.

## 21.10 Raccomandazioni per il progetto

Nel progetto è importante collegare correttamente progettazione dei test, implementazione, build e Continuous Integration.

### IMPORTANTE PER L'ESAME

Nel progetto è fortemente consigliato sperimentare la progettazione e realizzazione di test di unità associandoli al processo di Continuous Integration tramite Maven Surefire.

### IMPORTANTE PER L'ESAME

È inoltre consigliato progettare e realizzare test di integrazione associandoli alla Continuous Integration tramite Maven Failsafe.

### IMPORTANTE PER L'ESAME

Mockito dovrebbe essere usato per simulare componenti e dipendenze sia nei test di unità sia nei test di integrazione.

Errori comuni da evitare:

- usare nomi di test non conformi ai pattern riconosciuti da Maven;
- inserire integration test complessi in Surefire senza setup e teardown adeguati;
- scrivere test senza `assert` o senza `verify`;
- abusare di matcher troppo generici, come `any()`, indebolendo l'oracolo del test;
- confondere il test driver con il SUT;
- testare lo stub o il mock invece dell'unità reale sotto test;
- non agganciare i test automatici alla CI.

# Capitolo 22

## Approcci alla generazione dei test

### 22.1 Domande fondamentali del testing

La generazione dei test parte da tre domande fondamentali:

- quali e quanti input selezionare;
- quando è possibile smettere di testare;
- come stabilire se il risultato ottenuto è corretto.

Il dominio degli input è solitamente troppo vasto, o addirittura concettualmente infinito, per poter essere esplorato in modo esaustivo. Il problema centrale diventa quindi scegliere un sottoinsieme di valori **rappresentativi**, capaci di esporre il sistema a condizioni significative e potenzialmente critiche.

Per stabilire quando interrompere l'attività di testing è necessario definire criteri di copertura e soglie minime di accettazione. Tra i criteri possibili rientrano:

- copertura degli statement;
- copertura degli archi;
- copertura delle condizioni;
- copertura dei flussi;
- copertura delle funzionalità.

Un ulteriore problema fondamentale è l'**oracle problem**: per sapere se un test è passato o fallito, bisogna conoscere l'output atteso. Tale output può essere dedotto da specifiche, dati storici, sistemi equivalenti già in esecuzione o proprietà che devono valere.

#### Nota del Prof

Un caso di test non è soltanto un insieme di input. Un test completo deve specificare almeno **input**, **contesto** e **output atteso**. Se manca l'output atteso, non si sta progettando realmente un test, ma solo scegliendo dati da fornire al sistema.

## IMPORTANTE PER L'ESAME

È legittimo progettare casi di test in cui l'output atteso è una failure controllata, un errore o il lancio di un'eccezione. In questi casi il test passa se il sistema fallisce nel modo previsto.

## 22.2 Approcci alla generazione dei test

Le slide presentano diversi approcci per selezionare o generare casi di test:

- uso di dati da esecuzioni storiche;
- selezione randomica degli input;
- generazione automatica guidata da misure di adequacy;
- partizionamento del dominio in classi di equivalenza;
- uso di Large Language Models.

Ogni approccio ha vantaggi, limiti e contesti d'uso differenti.

### 22.2.1 Uso di dati storici

L'uso di dati storici consiste nel costruire test a partire da esecuzioni reali o passate del sistema.

I principali vantaggi sono:

- si testano modalità di utilizzo effettivamente sperimentate;
- il metodo è utile per attività di validazione;
- permette di focalizzarsi sulle parti del sistema maggiormente sollecitate dagli utenti;
- può dare indicazioni sulla reliability effettiva del sistema in esercizio.

Gli svantaggi principali sono:

- non risolve direttamente il problema della selezione o prioritizzazione dei test;
- può produrre un numero molto alto di casi;
- non è applicabile a nuove funzionalità o aspetti del sistema non ancora utilizzati.

### 22.2.2 Random testing

Il **random testing** consiste nel selezionare valori di input, o casi di test, in modo casuale.

I vantaggi principali sono:

- semplicità apparente di realizzazione;
- capacità di far emergere malfunzionamenti grossolani;
- utilità nelle fasi iniziali del testing.

Gli svantaggi principali sono:

- è una ricerca cieca;
- richiede un numero elevato di test per fornire livelli di confidenza significativi;
- realizzare un processo realmente randomico può essere più complesso del previsto;
- bisogna prestare attenzione a eventuali bias nascosti nella generazione casuale.

#### Nota del Prof

Il random testing può essere utile per scoprire bug grossolani, ma non sostituisce una strategia ragionata di selezione degli input. La casualità può sembrare neutrale, ma spesso contiene bias dovuti al modo in cui viene implementata.

### 22.2.3 Generazione guidata da adequacy

Le misure di **adequacy** stimano la bontà di un insieme di test. La generazione automatica può quindi essere guidata da obiettivi di copertura o da altre metriche di adeguatezza.

I vantaggi principali sono:

- alto livello di automatizzabilità;
- facile integrazione con processi di sviluppo e Continuous Integration;
- possibilità di generare test in modo sistematico rispetto a un obiettivo misurabile.

Gli svantaggi principali sono:

- se la generazione è guidata solo dal SUT, possono risultare molti test poco significativi rispetto alle condizioni reali di utilizzo;
- se la generazione è guidata da modelli del SUT, gli strumenti possono essere meno versatili nell'integrazione con i processi di sviluppo;
- sono presenti complessità legate a iper-parametri ed euristiche di generazione.

### 22.2.4 Classi di equivalenza

La generazione tramite **classi di equivalenza** consiste nel partizionare il dominio degli input, dei parametri e dello stato del SUT in gruppi che dovrebbero produrre comportamenti equivalenti.

L'idea è selezionare almeno un test rappresentativo per ogni partizione.

I vantaggi principali sono:

- l'approccio è indipendente dall'implementazione;
- si basa su requisiti, specifiche e conoscenza del dominio;
- permette di identificare aspetti salienti del comportamento atteso.

Gli svantaggi principali sono:

- la qualità dei test dipende dalla chiarezza di requisiti e specifiche;
- vengono considerati solo vincoli e limitazioni noti o esplicitamente deducibili;
- richiede familiarità con il SUT;
- richiede conoscenza dei contesti operativi e di deployment.

### 22.2.5 Uso di Large Language Models

Gli LLM possono essere interrogati in linguaggio naturale per generare casi di test o programmi di test.

I vantaggi principali sono:

- semplicità apparente di interazione;
- elevata capacità di ragionamento in tempi ridotti;
- possibilità di elaborare codice e linguaggio naturale nello stesso contesto.

Gli svantaggi principali sono:

- forte dipendenza dalla formulazione del prompt;
- rischio di eccessiva fiducia nelle risposte;
- possibili bias o allucinazioni;
- ragionamento black-box e probabilistico;
- risultati non deterministici.

#### IMPORTANTE PER L'ESAME

Gli LLM possono essere utili per supportare la generazione dei test, ma non devono essere trattati come oracoli affidabili. I test generati vanno sempre validati, riparati e giustificati.

## 22.3 Randoop

**Randoop** è un generatore automatico di unit test per programmi Java. Produce automaticamente programmi JUnit che implementano casi di test.

L'approccio seguito è detto **feedback-directed random test generation**. Randoop genera sequenze casuali di invocazioni di metodi e usa il feedback ottenuto dall'esecuzione per decidere se mantenere, scartare o minimizzare tali sequenze.

### 22.3.1 Test utili, ridondanti e illegali

Un **test utile** è una sequenza valida che può essere mantenuta come caso di test.

Un **test ridondante** è una sequenza che non aggiunge informazione significativa rispetto a sequenze più semplici. Randoop tende a scartare questi casi.

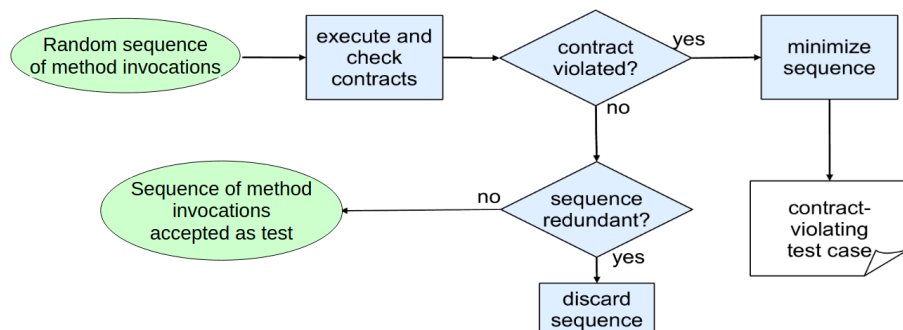
Un **test illegale** è una sequenza che viola i contratti o produce situazioni non ammissibili. Tali test non devono essere generati o devono essere scartati.

### 22.3.2 Singola iterazione di Randoop

La singola iterazione di Randoop può essere sintetizzata così:

sequenza randomica → esecuzione e check dei contratti → valutazione

Se viene violato un contratto, Randoop minimizza la sequenza e produce un test case che evidenzia la violazione. Se la sequenza è ridondante, viene scartata. Se invece è valida e non ridondante, viene accettata come test.



| Method           |                                                                        |
|------------------|------------------------------------------------------------------------|
| contract         | description                                                            |
| Exception (Java) | method throws no NullPointerException if no input parameter was null   |
|                  | method throws no AssertionError                                        |
| Exception (.NET) | method throws no NullReferenceException if no input parameter was null |
|                  | method throws no IndexOutOfRangeException                              |
|                  | method throws no AssertionError                                        |

Contratti di default in Randoop

| Object   |                                  |
|----------|----------------------------------|
| contract | description                      |
| equals   | o.equals(o) returns true         |
|          | o.equals(o) throws no exception  |
| hashCode | o.hashCode() throws no exception |
| toString | o.toString() throws no exception |

Estratto dalla presentazione di "Feedback-directed random test generation" by Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball ad ICSE 2007.

24

Figura 22.1: Flusso di funzionamento di Randoop: una sequenza randomica di invocazioni viene eseguita e controllata rispetto a contratti di default. Se un contratto viene violato, la sequenza viene minimizzata e trasformata in un test case che evidenzia la violazione; se non viola contratti ma è ridondante, viene scartata; altrimenti viene accettata come test.

#### Solo nelle slide (non spiegato a voce)

Le slide mostrano il flusso: sequenza randomica di invocazioni, esecuzione e controllo dei contratti, eventuale minimizzazione della sequenza, scarto delle sequenze ridondanti e accettazione delle sequenze utili.

### 22.3.3 Contratti di default

Randoop usa contratti di default per valutare il comportamento delle sequenze generate. Per esempio, può considerare anomalo il lancio di alcune eccezioni o la violazione di proprietà base degli oggetti.

#### Solo nelle slide (non spiegato a voce)

Tra i contratti riportati nelle slide compaiono controlli su eccezioni inattese e su metodi standard come `equals`, `hashCode` e `toString`.

### 22.3.4 Randoop: condizioni per la generazione dei test

Randoop genera test per un metodo o un costruttore  $M$  solo se alcune condizioni sono soddisfatte.

In particolare,  $M$  deve essere raggiungibile attraverso uno dei parametri di configurazione previsti da Randoop, come:

- `-testjar;`
- `-classlist;`
- `-testclass;`
- `-methodlist.`

Inoltre, il metodo non deve essere stato esplicitamente escluso dalla generazione tramite opzioni come:

- `-omit-methods;`
- `-omit-methods-file.`

Infine, la classe che contiene il metodo, il package e il tipo di ritorno devono essere inclusi nel classpath di Randoop.

#### Solo nelle slide (non spiegato a voce)

Le slide mostrano anche il riferimento alla documentazione ufficiale di Randoop e alle opzioni di esecuzione da riga di comando. L'idea operativa è fornire a Randoop il codice da analizzare, il classpath corretto e le classi o i metodi per cui generare test.

### IMPORTANTE PER L'ESAME

Randoop non genera test in modo magico per tutto il progetto: bisogna configurare correttamente classpath, classi, metodi inclusi ed eventuali esclusioni. Una configurazione incompleta può produrre pochi test, test non compilabili o nessun test utile.

## 22.3.5 Indicazioni operative

### Nota del Prof

Randoop può essere usato offline, senza integrarlo direttamente nel processo di build Maven. È possibile generare localmente molti test, selezionare solo quelli realmente utili e includere nel repository soltanto i test ritenuti significativi per la sperimentazione.

### IMPORTANTE PER L'ESAME

Non bisogna includere automaticamente tutti i test generati. I test prodotti da Randoop vanno analizzati, selezionati e giustificati.

## 22.4 Classi di equivalenza

Il partizionamento in classi di equivalenza consiste nel dividere il dominio di input in sottoinsiemi. Per ogni elemento di una partizione, il SUT dovrebbe evidenziare lo stesso comportamento, oppure dalle specifiche si può assumere che sia così.

La regola generale è includere nella test suite almeno un elemento per ogni partizione identificata.

La qualità della test suite dipende da:

- chiarezza di requisiti e specifiche;
- vincoli o limitazioni esplicitamente espressi;
- familiarità con il SUT;
- conoscenza dei contesti operativi e di deployment;
- conoscenza dell'implementazione, quando disponibile.

### Nota del Prof

Un parametro del test non coincide necessariamente con un parametro formale del metodo Java. Può includere anche lo stato interno della classe, la configurazione dell'ambiente, il contenuto della persistenza o lo stato di altre istanze collegate al SUT.

### 22.4.1 Linee guida generali

Le slide forniscono alcune linee guida per costruire classi di equivalenza.

## Range

Per valori numerici o intervalli, bisogna considerare:

- un valore nel range valido;
- un valore sotto il range;
- un valore sopra il range.

## Stringhe

Per le stringhe bisogna considerare almeno:

- stringhe valide;
- stringhe non valide;
- stringa vuota "";
- valore `null`.

## Enumerazioni

Per le enumerazioni, ogni valore può essere assegnato a una classe di equivalenza differente.

## Array

Per gli array bisogna considerare almeno:

- array legali;
- array vuoti;
- array che superano la lunghezza massima consentita.

## Tipi complessi

Per i tipi di dato complessi, le regole vanno applicate iterativamente ai campi interni della struttura. È importante considerare anche il valore `null`.

### IMPORTANTE PER L'ESAME

Le linee guida basate sul tipo sintattico sono utili, ma non sufficienti. Bisogna considerare anche la semantica del parametro. Una **String** che rappresenta una password non va trattata solo come stringa generica, ma come elemento del dominio applicativo.

### 22.4.2 Semantica del parametro

Se un parametro è sintatticamente una stringa ma semanticamente rappresenta una password, le classi di equivalenza possono includere:

- password vuota;
- password null;
- password valida nel dominio;
- password corretta;
- password errata;
- password non valida nel dominio.

#### Nota del Prof

La semantica del parametro può essere più importante del suo tipo sintattico. Limitarsi al tipo Java rischia di produrre partizioni troppo povere.

### 22.4.3 Validità del valore e correttezza nel SUT

È importante non confondere la **validità** di un valore con la **correttezza** rispetto alla configurazione del SUT.

Per esempio, una email può essere sintatticamente valida, ma non corrispondere ad alcun utente registrato nella configurazione del sistema.

#### IMPORTANTE PER L'ESAME

Un valore valido non è necessariamente corretto per uno specifico stato del SUT. La progettazione dei test deve considerare anche il contesto e lo stato del sistema.

## 22.5 Criteri per identificare i test

Le slide distinguono due criteri principali per combinare le classi di equivalenza:

- approccio unidimensionale;
- approccio multidimensionale.

### 22.5.1 Approccio unidimensionale

Nell'approccio unidimensionale, ogni parametro di input è considerato indipendentemente dagli altri. I test sono scelti o sintetizzati in modo che tutte le classi di equivalenza considerate siano coperte almeno una volta.

Questo approccio riduce il numero di test, ma può perdere combinazioni significative tra parametri.

### 22.5.2 Approccio multidimensionale

Nell'approccio multidimensionale, si considera il prodotto cartesiano delle classi di equivalenza dei vari parametri. I test coprono quindi la matrice di combinazione delle classi.

Questo approccio è più completo, ma può generare un numero molto alto di test.

#### IMPORTANTE PER L'ESAME

L'approccio unidimensionale è più economico, ma meno potente nel far emergere failure legate all'interazione tra parametri. L'approccio multidimensionale è più sistematico, ma può esplodere combinatoriamente.

### 22.5.3 Procedura sistematica

La procedura sistematica per il partizionamento prevede quattro passi:

1. identificare i domini di input;
2. identificare le classi di equivalenza per ogni parametro del test;
3. combinare le classi di equivalenza;
4. eliminare le combinazioni non ammissibili.

I domini possono essere identificati a partire da:

- key abstraction;
- feature;
- needs;
- requisiti;
- tipi di dato legati a scelte implementative;
- condizioni imposte in modo esplicito o implicito.

### 22.5.4 Uso della category partition

Il metodo di **category partition** può essere usato sia nel testing manuale sia nel testing automatico.

Nel **manual software testing**, la category partition aiuta a ragionare sul dominio del problema, sulle classi di input significative e sui casi da includere nella test suite. Questo è l'approccio richiesto nel corso: lo studente deve mostrare di saper identificare partizioni, combinazioni e boundary values in modo motivato.

Nel **automatic software testing**, invece, la category partition può essere usata come base per strumenti automatici, euristiche o soluzioni basate su AI capaci di generare casi di test.

### Solo nelle slide (non spiegato a voce)

Le slide specificano che l'uso automatico avanzato della category partition richiede la progettazione e lo sviluppo di strumenti di generazione automatica, eventualmente basati su euristiche o AI. Questo è indicato come lavoro più adatto a una tesi o a sviluppi avanzati.

### IMPORTANTE PER L'ESAME

Nel progetto non basta dire quali test sono stati scelti. Bisogna spiegare come sono state identificate le partizioni, quali combinazioni sono state considerate e perché alcune combinazioni sono state mantenute o eliminate.

## 22.6 Boundary-value analysis

La **boundary-value analysis** si basa sull'osservazione empirica che molti bug di programmazione si manifestano ai confini delle classi di equivalenza.

Errori tipici come usare `<` al posto di `<=`, oppure gestire male un valore minimo o massimo, emergono spesso in prossimità dei bordi del dominio.

La procedura consiste in:

1. partizionare il dominio di riferimento per ogni parametro;
2. identificare i confini di ogni partizione;
3. selezionare valori in modo che ogni confine compaia in almeno un test.

Nel caso multidimensionale, ogni confine può essere combinato con tutte le possibili combinazioni degli altri parametri. Nel caso unidimensionale, ogni confine deve apparire almeno in una tupla di input.

### IMPORTANTE PER L'ESAME

La boundary-value analysis è uno strumento centrale nella progettazione dei test, perché molti difetti si trovano nei valori limite tra classi di equivalenza.

### IMPORTANTE PER L'ESAME

Una selezione unidimensionale dei boundary values può ridurre molto il numero di test, ma può anche limitare la capacità di esporre failure. Alcune combinazioni escluse potrebbero essere significative.

## 22.7 Esempio: `LedgerHandle.asyncReadEntriesInternal`

Le slide considerano il metodo:

```
void asyncReadEntriesInternal(  
    long firstEntry,  
    long lastEntry,
```

```

    ReadCallback cb,
    Object ctx,
    boolean isRecoveryRead
)

```

dalla classe `LedgerHandle` di Apache BookKeeper.

### 22.7.1 Classi di equivalenza iniziali

Una prima identificazione delle classi di equivalenza può essere:

- `isRecoveryRead`: {false}, {true};
- `cb`: {null}, {valid\_instance}, {invalid\_instance};
- `ctx`: {null}, {valid\_instance}, {invalid\_instance};
- `firstEntry`: { $\leq 0$ }, { $> 0$ };
- `lastEntry`: { $\leq 0$ }, { $> 0$ };
- `LedgerHandle`: {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}.

### 22.7.2 Miglioramento semantico

Una classificazione solo positiva/negativa per `firstEntry` e `lastEntry` è troppo debole. Questi parametri rappresentano indici e devono essere considerati anche in relazione tra loro.

Una classificazione migliore per `lastEntry` è:

- `lastEntry < firstEntry`;
- `lastEntry = firstEntry`;
- `lastEntry > firstEntry`.

#### Nota del Prof

Il miglioramento deriva dall'analisi semantica del parametro. Non basta sapere che `lastEntry` è un numero; bisogna capire che rappresenta un indice correlato a `firstEntry`.

### 22.7.3 Ruolo dello stato del SUT

Nel caso di `LedgerHandle`, anche lo stato del ledger è rilevante. Non basta considerare i parametri formali del metodo. Bisogna tenere conto, per esempio, del fatto che il ledger sia vuoto, abbia entry sufficienti o non abbia entry sufficienti.

#### IMPORTANTE PER L'ESAME

Il SUT stesso può costituire parte del dominio di input. Lo stato interno dell'oggetto prima dell'invocazione è parte del contesto del test.

## 22.7.4 Attenzione agli `invalid_instance`

Le slide avvertono di non escludere casi `invalid_instance` per tipi complessi con giustificazioni deboli, come:

“Il costruttore della classe torna solo istanze corrette.”

Questa giustificazione non è sufficiente perché:

- il costruttore potrebbe cambiare in futuro introducendo bug;
- soluzioni polimorfe potrebbero legare istanze di sottoclassi con implementazioni diverse;
- un mock potrebbe simulare comportamenti non previsti dall’implementazione standard.

### IMPORTANTE PER L'ESAME

Non bisogna escludere input complessi non validi senza una motivazione solida. Le assunzioni sull’implementazione attuale possono diventare false in presenza di modifiche o polimorfismo.

## 22.7.5 Boundary values per `firstEntry` e `lastEntry`

Limitandosi a `firstEntry` e `lastEntry`, le slide indicano boundary values come:

- `firstEntry`:  $-1, 0, 1$ ;
- `lastEntry`:  $\text{firstEntry} - 1, \text{firstEntry}, \text{firstEntry} + 1$ .

Una selezione minimale unidimensionale potrebbe includere:

- `firstEntry = -1, lastEntry = -2`;
- `firstEntry = 0, lastEntry = 0`;
- `firstEntry = 1, lastEntry = 2`.

Tuttavia, una selezione più ricca può considerare combinazioni come:

- `firstEntry = 1, lastEntry = 0`;
- `firstEntry = 1, lastEntry = -1`;
- `firstEntry = -1, lastEntry = 0`;
- `firstEntry = 0, lastEntry = -1`.

### IMPORTANTE PER L'ESAME

Una selezione troppo minimale può perdere failure. Non bisogna escludere a priori combinazioni potenzialmente significative solo perché si vuole ridurre il numero di

test.

## 22.8 LLM per la generazione dei test

I **Large Language Models** sono grandi reti neurali addestrate su grandissime moli di dati. Uno dei loro punti di forza è l'interazione tramite linguaggio naturale.

Nel contesto della Software Engineering, gli LLM possono supportare la generazione automatica di artefatti software, tra cui programmi di test.

### 22.8.1 Vantaggi

I principali vantaggi sono:

- rapidità nella generazione;
- apparente semplicità di interazione;
- capacità di elaborare codice e linguaggio naturale;
- possibilità di generare bozze di test program.

### 22.8.2 Limiti

I principali limiti sono:

- forte dipendenza dal prompt;
- output probabilistici e non deterministici;
- rischio di allucinazioni;
- possibili bias;
- rischio di eccessiva fiducia nelle risposte;
- difficoltà a controllare il ragionamento interno del modello.

#### IMPORTANTE PER L'ESAME

Gli LLM non sostituiscono la progettazione dei test. Possono generare candidati, ma l'ingegnere deve controllare compilabilità, significatività, oracle e coerenza con le specifiche.

## 22.9 Prompting per test generation

Una richiesta a un LLM viene formulata tramite un **prompt**. È possibile strutturare il prompt specificando:

- ruolo del modello;

- contesto;
- codice del PUT, cioè Program Under Test;
- formato della risposta attesa;
- vincoli sul numero o tipo di test;
- livello di dettaglio desiderato.

### 22.9.1 Struttura dei prompt

Un prompt è una richiesta in linguaggio naturale inviata a un LLM. La struttura del prompt può essere definita in modo abbastanza libero e può riguardare aspetti:

- sintattici, cioè il formato della richiesta;
- semantici, cioè il significato del task richiesto;
- conversazionali, cioè il ruolo assegnato al modello;
- contestuali, cioè le informazioni di progetto o dominio fornite.

È possibile specificare anche la struttura della risposta attesa. Per esempio, si può chiedere che l'output sia un file JUnit completo, che inizi e finisca con marker specifici, oppure che includa solo codice senza spiegazioni.

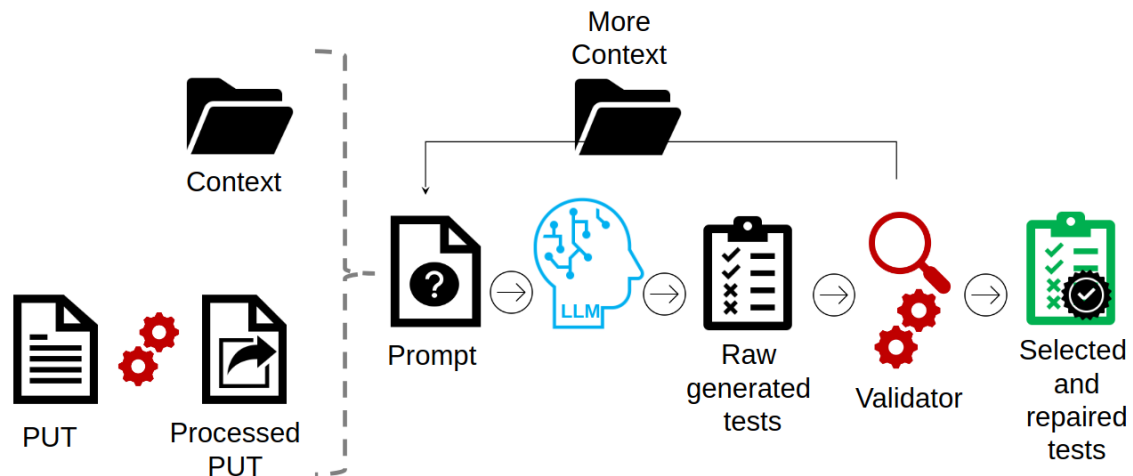
#### Solo nelle slide (non spiegato a voce)

Le slide mostrano prompt in cui il modello viene istruito a comportarsi come un *professional software tester*, a generare un file JUnit 4, a testare tutti i metodi della classe e a delimitare l'output con marker come `###Test START###` e `###Test END###`.

### 22.9.2 Pure-prompting schema

#### Solo nelle slide (non spiegato a voce)

Il pure-prompting schema include: PUT, processed PUT, context, prompt, raw generated tests, validator e selected/repaiored tests.



55

Image by Antonia Bertolino – Keynote at ICST 2025

Figura 22.2: Schema di generazione dei test tramite LLM: il Program Under Test viene eventualmente preprocessato e combinato con prompt e contesto aggiuntivo. L’LLM produce test grezzi, che vengono poi validati, selezionati ed eventualmente riparati prima di essere usati.

Il flusso può essere descritto così:

1. il PUT e il contesto vengono preparati;
2. viene costruito un prompt;
3. il prompt viene inviato all’LLM;
4. l’LLM produce test grezzi;
5. un validator controlla i test;
6. i test vengono selezionati, riparati o scartati.

### 22.9.3 Zero-shot prompting

Nel **zero-shot prompting**, il modello riceve direttamente la richiesta di generare test senza esempi espliciti.

Un esempio di struttura è chiedere al modello di agire come un tester professionale e generare un file JUnit per una classe specifica.

(a) Zero-shot Prompting

As a professional software tester who writes Java test methods, generate a complete Junit 4 test file `{{class_name}}Test.java` to comprehensively test all methods in the following class named `{{class_name}}`. Your output file must start with `###Test START##` and finish with `###Test END##`. Here is the source code:\n`{{source_code}}`

Figura 22.3: Esempio di prompt zero-shot per la generazione di test JUnit 4. Il prompt assegna al modello il ruolo di tester professionale, specifica la classe da testare, richiede di coprire tutti i metodi, impone marker di inizio e fine dell'output e include il codice sorgente del SUT tramite `{{source_code}}`.

#### 22.9.4 Variante zero-shot senza codice del SUT

Una variante interessante dello zero-shot prompting consiste nel rimuovere dal prompt il codice del SUT.

In questo caso si può osservare come cambia l'output dell'LLM quando non riceve direttamente l'implementazione da testare. Questa variante può aiutare a capire quanto il modello stia ragionando sulle informazioni fornite e quanto, invece, stia producendo test generici o basati su conoscenza pregressa.

(a) Zero-shot Prompting

As a professional software tester who writes Java test methods, generate a complete Junit 4 test file `{{class_name}}Test.java` to comprehensively test all methods in the following class named `{{class_name}}`. Your output file must start with `###Test START##` and finish with `###Test END##`.



Una variante di prompt che potrebbe rivelarsi interessante da studiare riguarda la rimozione del codice del SUT. Ad esempio:

- non esplicitando questa informazione, come verrebbe influenzato l'output?
- in base ai risultati ottenuti, potremmo intuire qualcosa sul LLM utilizzato?

Figura 22.4: Variante zero-shot senza codice del SUT: il prompt mantiene ruolo, obiettivo, classe target e vincoli di output, ma rimuove il codice sorgente. Questa configurazione permette di osservare quanto la qualità dei test dipenda dal contesto fornito al modello.

#### IMPORTANTE PER L'ESAME

Se si rimuove il codice del SUT, i test generati possono diventare più generici, meno compilabili o meno aderenti al comportamento reale. Questa variante è utile per discutere criticamente il ruolo del contesto nel prompting.

## 22.9.5 Few-shot prompting

Nel **few-shot prompting**, oltre alla richiesta principale, il prompt contiene uno o più esempi di test o di risposte attese.

(b) Few-shot Prompting

```
As a professional software tester who writes Java test methods, consider the following
examples: {fewshot_example}nNow, generate JUnit 4 test cases to comprehensively test
all methods in the following class named {class_name} Your output file must start with
{###Test START##} and finish with {###Test END##} Here is the source code: \n{source_code
}.
```

Figura 22.5: Esempio di prompt few-shot per la generazione di test JUnit 4. Rispetto allo zero-shot, il prompt include uno o più esempi tramite {fewshot\_example}, oltre al nome della classe, al codice sorgente del SUT e ai marker di inizio e fine dell'output.

Questi esempi servono a guidare il modello rispetto a:

- stile del codice;
- struttura della classe di test;
- uso delle annotazioni JUnit;
- forma delle assert;
- livello di dettaglio richiesto.

Rispetto allo zero-shot, il few-shot può produrre output più aderenti al formato desiderato, ma dipende fortemente dalla qualità degli esempi forniti.

### IMPORTANTE PER L'ESAME

Gli esempi inseriti nel prompt influenzano molto l'output. Esempi poveri, incompleti o con assert deboli possono guidare l'LLM verso test altrettanto deboli.

## 22.9.6 Chain-of-Thought

Nel contesto delle slide, il prompting con **Chain-of-Thought** aggiunge indicazioni operative passo-passo per guidare il modello nel processo di generazione.

Per esempio, si può chiedere di:

1. identificare i metodi pubblici;
2. individuare edge cases;
3. proporre scenari di test;

4. generare i test JUnit.

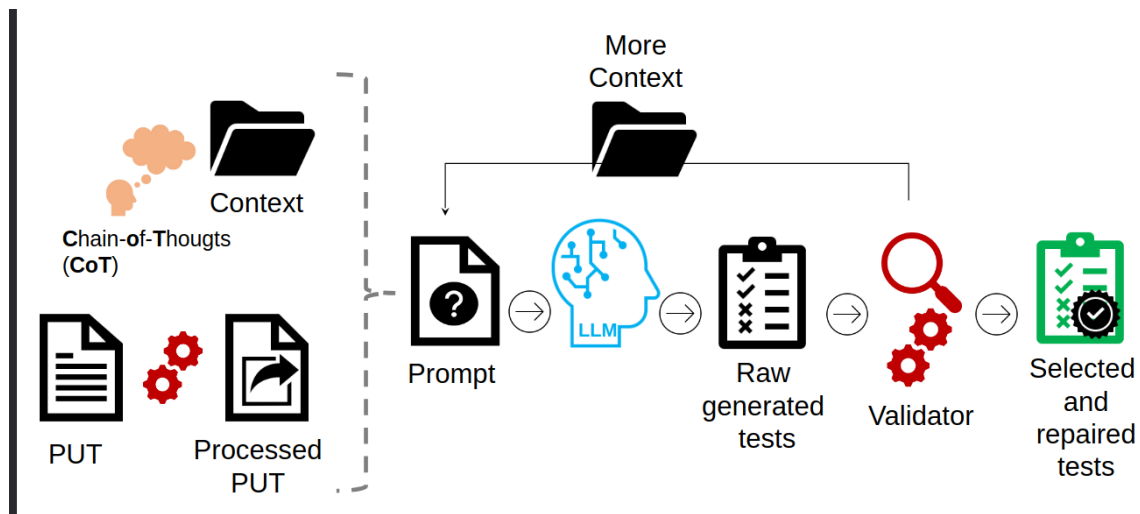


Figura 22.6: Schema di generazione dei test con Chain-of-Thought: oltre al PUT, al contesto e al prompt, il modello riceve indicazioni di ragionamento passo-passo. L'obiettivo è guidare l'LLM nell'analisi del codice, nell'identificazione degli edge cases e nella produzione di test più strutturati.

### 22.9.7 Guided Tree-of-Thoughts

Nel **Guided Tree-of-Thoughts prompting**, il modello viene guidato a simulare più ragionamenti o più prospettive alternative.

Un esempio tipico consiste nel chiedere al modello di immaginare più esperti di software testing che collaborano per costruire una test suite. Ogni esperto propone idee, controlla quelle degli altri, individua casi mancanti e scarta soluzioni errate.

(c) Guided Tree-of-Thoughts Prompting

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 4 test cases for the following Java class. To comprehensively test all methods in the following class named `{class_name}` they must following these steps:\n- Extract and list all the public methods including their signatures\n- For each methods, generate a basic JUnit 4 test case that checks the method's functionality\n- Given the source code of the class and the listed methods, identify potential edge cases and exception handling scenarios that should be tested\n- Generate JUnit 4 test cases that specifically test for the identified edge cases and exceptions\n- Merge all the individual test cases into a complete JUnit 4 test file `{class_name}Test.java` for the given Java class. All experts will propose one test cases for each method, share it with the group, and then proceed to the next step. If any expert realizes they're wrong at any point, they leave.\n\nThe Java class is: `{source_code}`\n\nAt the end they must propose one complete (including typical use cases, edge cases, and error scenarios) JUnit 4 test file `{class_name}Test.java` for the given Java class. The complete JUnit test file must start with `###Test START##` and finish with `###Test END##`

Figura 22.7: Esempio di prompt Guided Tree-of-Thoughts per la generazione di test JUnit 4. Il prompt simula più esperti di software testing che collaborano, estraendo metodi pubblici, identificando edge cases ed eccezioni, generando test individuali e fondendoli in una test suite finale.

La strategia può includere passaggi come:

1. identificare tutti i metodi pubblici;
2. verificare nomi, firme e funzionalità dei metodi;
3. individuare edge cases;
4. includere scenari di eccezione;
5. generare test JUnit;
6. fondere i singoli test in una test suite completa.

#### IMPORTANTE PER L'ESAME

Guided ToT non garantisce automaticamente test corretti, ma può aiutare a rendere più esplicito il processo di ragionamento richiesto all'LLM e a ridurre omissioni evidenti.

### 22.9.8 RAG

La **Retrieval-Augmented Generation** consiste nel fornire all'LLM informazioni aggiuntive recuperate da fonti esterne, come documentazione, specifiche, codice del progetto, API o esempi di utilizzo.

Nel contesto della generazione dei test, il RAG serve a dare al modello più contesto sul **Program Under Test** e sul progetto. In questo modo l'LLM non genera test basandosi solo sulla conoscenza generale appresa in addestramento, ma usa anche informazioni specifiche e aggiornate.

#### Nota del Prof

Per esempio, se si vuole generare un test per un metodo di Apache BookKeeper, il RAG può recuperare documentazione della classe, firme dei metodi, vincoli sui parametri ed esempi d'uso. Questo aiuta l'LLM a produrre test più aderenti al comportamento reale del sistema.

#### IMPORTANTE PER L'ESAME

Il RAG riduce il rischio che l'LLM inventi API, parametri o comportamenti non presenti nel progetto. Tuttavia, anche con RAG, i test generati devono essere controllati, compilati e validati.

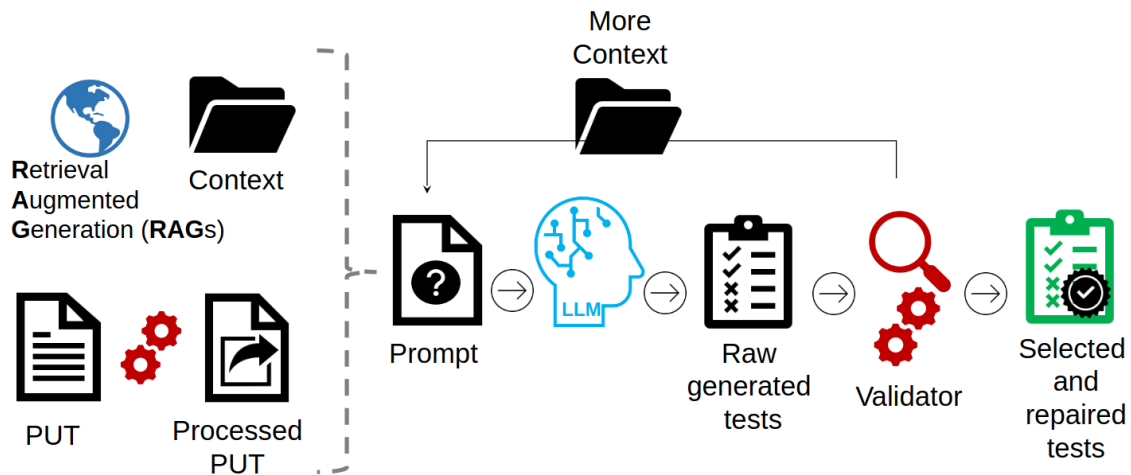


Figura 22.8: Schema di test generation con Retrieval-Augmented Generation: oltre al PUT, al prompt e al contesto iniziale, l'LLM riceve informazioni aggiuntive recuperate da fonti esterne, come documentazione, API o specifiche di progetto. Questo contesto supplementare aiuta a generare test più coerenti con il sistema reale.

## 22.10 Collegamento con progetto ed esame

Nel corso sono richiesti pochi test rispetto ai numeri industriali, ma le scelte devono essere motivate in modo rigoroso.

### Solo nelle slide (non spiegato a voce)

Le slide riportano esempi industriali e reali: Google esegue quotidianamente centinaia di migliaia di build e milioni di test; Apache BookKeeper ha centinaia di classi di test e migliaia di metodi annotati con `@Test`.

### IMPORTANTE PER L'ESAME

Non bisogna giustificare un basso numero di test dicendo che si vuole mantenere basso il costo o il numero di test. Le scelte devono essere giustificate con partizioni, boundary analysis, oracle e rimozione logica di casi ridondanti o non ammissibili.

Nel progetto bisogna:

- identificare partizioni del dominio;
- condurre boundary-value analysis;
- proporre test con output atteso;
- usare eventualmente LLM per generare test;
- variare i prompt;
- discutere la qualità dei test prodotti.

In particolare, può essere richiesto di proporre prompt:

- zero-shot;
- few-shot;
- Chain-of-Thought;
- Tree-of-Thoughts.

Gli errori da evitare sono:

- scrivere test senza oracle;
- usare prompt vaghi;
- fidarsi ciecamente dell'LLM;
- basare le partizioni solo sul tipo sintattico;
- escludere combinazioni solo per ridurre il numero di test;
- non giustificare la selezione dei casi.

# Capitolo 23

## Adeguatezza dei test e criteri di coverage

### 23.1 Adeguatezza dei test

Una volta iniziata l'attività di testing, uno dei problemi principali è stabilire **se e quando** sia possibile **smettere di testare** il **System Under Test (SUT)**.

Le domande fondamentali sono:

- i miei test sono sufficientemente buoni?
- il SUT è stato testato in modo abbastanza meticoloso?
- i test sono adeguati rispetto agli obiettivi prefissati?

#### Nota del Prof

Non potendo esplorare il dominio degli input in modo esaustivo, il testing è sempre un'attività *best-effort*. La qualità della test suite dipende da vincoli di tempo, budget, risorse computazionali e obiettivi del progetto.

Non esiste una tecnica di adeguatezza perfetta e generale. Le diverse tecniche dipendono dal tipo di processo, dagli artefatti disponibili e dalle strategie applicate. Di conseguenza, il testing richiede spesso di negoziare **trade-off** tra costo, profondità dell'analisi e livello di confidenza ottenuto.

#### IMPORTANTE PER L'ESAME

L'adeguatezza dei test non dimostra l'assenza di bug. Serve piuttosto a fornire evidenza del fatto che la test suite ha esplorato il SUT secondo criteri espliciti e misurabili.

### 23.2 Functional vs Structural Adequacy

I criteri di adeguatezza possono essere distinti in due grandi famiglie:

- **criteri funzionali**, basati su requisiti, specifiche o modelli del comportamento atteso;
- **criteri strutturali**, basati sull'effettiva implementazione del SUT.

### 23.2.1 Adeguatezza funzionale

L'adeguatezza funzionale è basata su modelli o specifiche del SUT non necessariamente eseguibili. L'attenzione è posta sui **comportamenti osservabili** del sistema, senza considerare direttamente il codice sorgente.

Questo approccio è vicino al testing **black-box**: si valuta se la test suite copre scenari significativi derivati da requisiti, specifiche, activity diagram o regole di business.

#### Solo nelle slide (non spiegato a voce)

Il limite principale dell'approccio funzionale è che non permette di scoprire errori dovuti a dettagli implementativi. Se il programmatore ha introdotto un caso limite non descritto dalle specifiche, un criterio puramente funzionale potrebbe non intercettarlo.

### 23.2.2 Adeguatezza strutturale

L'adeguatezza strutturale si basa invece sull'implementazione effettiva del SUT, come codice sorgente o **Control-Flow Graph**. L'obiettivo è esplorare la maggior parte possibile del codice realmente eseguito.

Questo approccio è vicino al testing **white-box**: si misura quanto codice, quanti rami o quante condizioni sono state effettivamente esercitate dalla test suite.

#### IMPORTANTE PER L'ESAME

La coverage strutturale aumenta la confidenza sul fatto che il codice sia stato esplorato, ma non garantisce che i test abbiano un oracle corretto né che siano state trovate failure.

#### Nota del Prof

Se un sistema dovrebbe offrire sia tè sia caffè, ma il programmatore implementa solo il caffè, una coverage strutturale del 100% sul codice esistente non farà emergere la funzionalità mancante del tè. I criteri strutturali non possono scoprire ciò che non è stato implementato.

## 23.3 Criteri funzionali

I criteri funzionali stabiliscono quando una test suite può essere considerata adeguata rispetto a requisiti o specifiche.

Nel caso di Apache BookKeeper, le slide riportano alcuni esempi di criteri funzionali:

- **CF1**: almeno un test di scrittura-lettura su una entry dove nessun bookie fallisce;
- **CF2**: almeno un test di scrittura-lettura che simuli il fallimento di  $Q_a - 2$  bookies;
- **CF3**: almeno un test di scrittura-lettura che simuli il fallimento di  $Q_a - 1$  bookies;
- **CF4**: almeno un test che scriva entry su un ledger e le recuperi verificando che siano nello stesso ordine;

- **CF5**: almeno un test con un writer e più reader, in cui i reader confrontano le entry recuperate.

#### Solo nelle slide (non spiegato a voce)

Altri esempi di criteri funzionali sono: **CF6**, almeno un test che si chiuda con successo; **CF7**, almeno un test che causi il fallimento del “CAS Write”; **CF8**, almeno un test che si chiuda con un errore.

#### Nota del Prof

Un criterio di adeguatezza funzionale stabilisce a priori quali scenari devono essere coperti. Una test suite è adeguata se copre gli scenari derivati dai requisiti o dalle specifiche, indipendentemente da come i test siano stati generati.

## 23.4 Criteri strutturali di control-flow

I criteri strutturali basati sul **control-flow** valutano quanto la test suite esplori il codice eseguibile del SUT.

Il programma può essere rappresentato tramite un **Control-Flow Graph** (CFG), composto da:

- nodi, che rappresentano statement o blocchi di statement;
- archi, che rappresentano i possibili passaggi di controllo;
- decisioni, che rappresentano punti di scelta come **if**, **while** o **switch**;
- condizioni atomiche, cioè sotto-espressioni booleane semplici.

#### IMPORTANTE PER L'ESAME

Se esistono statement, decisioni o condizioni sintatticamente irraggiungibili, questi elementi non dovrebbero penalizzare ingiustamente la metrica. Devono essere esclusi dal denominatore della misura di coverage.

### 23.4.1 Statement e Block Coverage

La **Statement Coverage** misura la percentuale di istruzioni eseguite almeno una volta dalla test suite.

La **Block Coverage** ragiona in modo simile, ma considera blocchi di istruzioni senza diramazioni interne.

Una test suite è pienamente adeguata rispetto a questo criterio quando tutti gli statement o blocchi raggiungibili sono stati eseguiti almeno una volta.

#### Nota del Prof

Anche una coverage molto alta può non bastare. Coprire il 99% delle righe può essere insufficiente se l'1% mancante contiene l'unico ramo che nasconde un bug critico.

### 23.4.2 Branch o Decision Coverage

La **Branch Coverage**, detta anche **Decision Coverage**, misura se ogni decisione è stata esplorata in tutti i suoi possibili esiti.

Per esempio, in un `if`, una test suite adeguata deve far valutare la decisione almeno una volta a `true` e almeno una volta a `false`.

Questo criterio è più forte della semplice statement coverage, perché obbliga a esplorare anche le alternative del flusso di controllo.

#### IMPORTANTE PER L'ESAME

La Branch Coverage considera il risultato complessivo della decisione, ma non spiega quali condizioni atomiche abbiano determinato quel risultato.

### 23.4.3 Condition Coverage

La **Condition Coverage** valuta le condizioni booleane atomiche contenute nelle decisioni complesse.

Una test suite è adeguata se ogni condizione atomica assume almeno una volta il valore `true` e almeno una volta il valore `false`.

Per esempio, nella decisione:

$$(i > 0) \wedge (j > 0)$$

le condizioni atomiche sono:

$$(i > 0) \quad (j > 0)$$

Entrambe devono essere valutate sia a `true` sia a `false`.

#### Nota del Prof

In linguaggi come Java o C++, la *lazy evaluation* può impedire la valutazione di alcune condizioni. Per esempio, in una congiunzione con `&&`, se la prima condizione è falsa, la seconda potrebbe non essere valutata.

#### IMPORTANTE PER L'ESAME

La Condition Coverage non implica necessariamente la Branch Coverage, e la Branch Coverage non implica necessariamente la Condition Coverage.

### 23.4.4 Branch-Condition Coverage

La **Branch-Condition Coverage** combina branch coverage e condition coverage.

L'obiettivo è coprire sia:

- gli esiti complessivi delle decisioni;
- i valori `true` e `false` delle condizioni atomiche.

Questo criterio è più forte dei due criteri presi singolarmente, ma può ancora non essere sufficiente per dimostrare che ogni condizione atomica abbia un effetto indipendente sulla decisione complessa.

### 23.4.5 Multiple Condition Coverage

La **Multiple Condition Coverage** richiede di coprire tutte le possibili combinazioni di valori delle condizioni atomiche.

Se una decisione contiene  $k$  condizioni atomiche, il numero di combinazioni possibili è:

$$2^k$$

Quindi, per una decisione con tre condizioni atomiche, bisognerebbe considerare otto combinazioni.

#### IMPORTANTE PER L'ESAME

La Multiple Condition Coverage è molto completa, ma soffre di esplosione combinatoria. Per questo motivo può essere impraticabile nei sistemi reali.

### 23.4.6 MC/DC Coverage

La **Modified Condition/Decision Coverage** (MC/DC) è un compromesso pratico tra completezza e costo.

L'obiettivo è evitare l'esplosione combinatoria della Multiple Condition Coverage, ma mantenere una forte garanzia sul ruolo delle condizioni atomiche.

Una test suite MC/DC adeguata deve garantire che:

1. ogni blocco o statement rilevante sia coperto;
2. ogni condizione atomica assuma almeno una volta **true** e almeno una volta **false**;
3. ogni decisione assuma tutti i suoi possibili esiti;
4. ogni condizione atomica mostri un **effetto indipendente** sul risultato della decisione complessa.

#### IMPORTANTE PER L'ESAME

L'effetto indipendente significa che bisogna trovare due test in cui cambia una sola condizione atomica, tutte le altre restano uguali, e il risultato della decisione complessa cambia.

MC/DC è particolarmente rilevante nei contesti critici, perché fornisce un livello di confidenza elevato senza richiedere necessariamente tutte le combinazioni possibili.

## 23.5 Relazioni tra i criteri

I criteri di coverage non sono tutti equivalenti e non misurano la stessa cosa.

Una coverage alta non garantisce l'assenza di bug. Essa indica solo che il codice è stato esplorato secondo un certo criterio. Per rivelare failure servono anche oracle corretti e test significativi.

### Solo nelle slide (non spiegato a voce)

Le slide sottolineano che non esiste una garanzia per cui alta coverage strutturale implichi esposizione di failure. La coverage strutturale aumenta solo la confidenza nell'esplorazione del SUT.

Un esempio classico riguarda la decisione:

$$(i > 0) \wedge (j > 0)$$

Con i test:

$$(i = -3, j = 2) \quad (i = 4, j = 2)$$

si può ottenere Branch Coverage, perché la decisione complessiva assume sia **false** sia **true**. Tuttavia, la condizione  $j > 0$  rimane sempre vera, quindi non si ottiene Condition Coverage completa.

Viceversa, con i test:

$$(i = -3, j = 2) \quad (i = 24, j = -1)$$

le condizioni atomiche possono assumere sia **true** sia **false**, ma la decisione complessiva può rimanere sempre falsa. In questo caso si può ottenere Condition Coverage senza ottenere Branch Coverage completa.

### IMPORTANTE PER L'ESAME

Coprire le condizioni atomiche non basta a coprire i rami, e coprire i rami non basta a dimostrare il ruolo delle singole condizioni atomiche.

## 23.6 MC/DC

La MC/DC nasce per gestire decisioni complesse senza dover eseguire tutte le combinazioni booleane possibili.

Nel caso di una decisione con molte condizioni atomiche, la Multiple Condition Coverage richiederebbe  $2^k$  combinazioni. MC/DC riduce il costo, richiedendo però di dimostrare che ciascuna condizione atomica possa influenzare indipendentemente l'esito della decisione.

### IMPORTANTE PER L'ESAME

MC/DC è più forte della semplice Branch Coverage e della semplice Condition Coverage, perché richiede di mostrare l'effetto indipendente di ogni condizione atomica sulla decisione complessa.

Per dimostrare l'effetto indipendente di una condizione atomica, bisogna individuare una coppia di test in cui:

- la condizione atomica considerata cambia valore;
- tutte le altre condizioni atomiche restano invariate;

- il valore finale della decisione cambia.

Questo criterio permette di capire se una condizione atomica è realmente capace di determinare l'esito della decisione, invece di essere mascherata da altre condizioni.

## 23.7 Esempio MC/DC da Mathur

Solo nelle slide (non spiegato a voce)

L'esempio MC/DC è tratto dal problema P7.14 del libro di Mathur.

Il programma considerato decide quale funzione di sparo invocare, tra **fire-1**, **fire-2** e **fire-3**, sulla base di coordinate, direzione corrente e direzione precedente. Il comportamento avviene all'interno di un ciclo.

### 23.7.1 Requisiti

I requisiti principali sono:

- **R1**: definisce le regole per scegliere quale funzione di sparo invocare;
- **R1.1**: specifica il caso in cui invocare **fire-1**;
- **R1.2**: specifica il caso in cui invocare **fire-2**;
- **R1.3**: rappresenta il caso di default, cioè l'invocazione di **fire-3**;
- **R2**: stabilisce che l'invocazione continua in loop finché **done** non diventa **true**.

### 23.7.2 Decisioni e condizioni

Le decisioni principali sono:

- **C<sub>1</sub>**: condizione del ciclo, cioè **!done**;
- **C<sub>2</sub>**: condizione complessa:

$$(x < y) \wedge (z \cdot z > y) \wedge (prev == \text{"East"})$$

- **C<sub>3</sub>**: condizione complessa dell'**else-if**:

$$((x < y) \wedge (z \cdot z \leq y)) \vee (current == \text{"South"})$$

### 23.7.3 Test suite T1

La prima test suite,  $T_1$ , contiene quattro test iniziali:

$$t_1, t_2, t_3, t_4$$

Questa suite raggiunge il 100% di statement coverage, block coverage e decision coverage.

Tuttavia,  $T_1$  non è sufficiente per ottenere condition coverage completa, perché l'espressione  $x < y$  risulta sempre vera nei test considerati.

### IMPORTANTE PER L'ESAME

Una suite può raggiungere statement, block e decision coverage senza essere adeguata rispetto alla condition coverage.

#### 23.7.4 Test suite T2

La seconda test suite,  $T_2$ , aggiunge un test  $t_5$  per forzare la condizione  $x < y$  a **false**.

In questo modo si migliora la copertura delle condizioni atomiche e si raggiunge la condition coverage.

Tuttavia,  $T_2$  non è ancora sufficiente per MC/DC, perché non dimostra l'effetto indipendente di tutte le condizioni atomiche. In particolare, non viene provato in modo indipendente l'effetto della condizione:

*current* == "South"

sulla decisione  $C_3$ .

#### 23.7.5 Test suite T3

La test suite  $T_3$  aggiunge ulteriori test:

$t_6, t_7, t_8, t_9$

Questi test permettono di formare coppie in cui varia una sola condizione atomica alla volta, mentre le altre restano invariate. Se il risultato della decisione cambia, allora è dimostrato l'effetto indipendente della condizione considerata.

Per questo motivo,  $T_3$  è considerata **MC/DC adequate**.

### IMPORTANTE PER L'ESAME

Una suite MC/DC adequate deve dimostrare che ogni condizione atomica può cambiare indipendentemente il risultato della decisione complessa.

#### 23.7.6 Importanza dell'ordine dei test

Nell'esempio di Mathur, l'ordine dei test è importante perché il programma contiene stato interno aggiornato durante l'esecuzione, come le variabili **prev** e **current**.

Due test possono avere gli stessi valori di input per  $x$ ,  $y$  e  $z$ , ma produrre effetti diversi se lo stato interno del programma è diverso.

### IMPORTANTE PER L'ESAME

In presenza di stato interno o cicli, i test non sono sempre indipendenti. L'ordine di esecuzione può influenzare il comportamento osservato.

## 23.8 Tool e collegamento con il progetto

**JaCoCo** è uno degli strumenti più usati in ambito Java per misurare la code coverage. Può essere integrato con Maven e con processi di Continuous Integration.

### Nota del Prof

Nel progetto è utile scrivere prima test manuali black-box e poi usare JaCoCo per osservare quali statement o branch non sono stati coperti. A quel punto si possono aggiungere test mirati in ottica white-box.

JaCoCo può produrre report di coverage, spesso consultabili in formato HTML nella directory:

`target/site/jacoco`

### Solo nelle slide (non spiegato a voce)

Le slide richiamano anche altri strumenti di coverage, come Emma, Cobertura e SonarCloud, oltre alla possibilità di usare JaCoCo in progetti Maven multi-modulo.

Nel report di progetto non basta riportare il valore numerico della coverage. È necessario spiegare:

- quale criterio è stato usato;
- quale valore è stato ottenuto inizialmente;
- quali test sono stati aggiunti per migliorare la coverage;
- quali limiti restano;
- quali trade-off sono stati accettati.

### IMPORTANTE PER L'ESAME

Non basta dire “ho raggiunto alta coverage”. Bisogna spiegare cosa misura quella coverage, quali parti del codice sono state esplorate, quali non sono state coperte e perché.

# Capitolo 24

## Mutation Testing

### 24.1 Limiti dei criteri di adequacy precedenti

I criteri di adeguatezza visti in precedenza permettono di stimare quanto una test suite esplori il **System Under Test** (SUT), ma non misurano direttamente la sua capacità di rilevare difetti.

La **functional adequacy** si basa su specifiche, requisiti o comportamenti osservabili. Il suo limite principale è che non riesce a scoprire errori dovuti a dettagli implementativi introdotti dal programmatore.

La **structural adequacy**, invece, si basa sul codice e misura l'esplorazione di statement, branch o condizioni. Tuttavia, anche una coverage molto alta non garantisce che i test siano realmente efficaci nel trovare difetti.

#### Solo nelle slide (non spiegato a voce)

I criteri funzionali e strutturali danno solo un'idea di quanta parte del SUT sia stata esplorata, ma non sono indici diretti dell'effettiva capacità di *fault detection* della test suite.

#### Nota del Prof

L'assunzione secondo cui esplorare tutto il codice faccia trovare i bug è solo una correlazione empirica. Se si raggiunge il 100% di coverage, ma le asserzioni non controllano il reale stato del sistema, i test possono comunque passare ignorando errori logici.

#### IMPORTANTE PER L'ESAME

Alta coverage non significa automaticamente test efficaci. La coverage misura l'esplorazione del codice, non la forza degli oracle né la capacità della test suite di rilevare difetti.

### 24.2 Idea del Mutation Testing

Il **Mutation Testing** è un approccio basato sull'inserimento deliberato di **mutazioni**, cioè alterazioni artificiali, all'interno del codice del SUT.

L'obiettivo non è misurare semplicemente quanto codice viene eseguito, ma stimare la bontà di una test suite osservando se essa riesce ad accorgersi di difetti artificialmente introdotti.

In questo senso, il Mutation Testing misura:

- la potenziale capacità della test suite di rilevare errori;
- la robustezza della test suite rispetto a modifiche ed evoluzioni future del codice;
- la capacità degli oracle di distinguere il comportamento originale da quello alterato.

#### IMPORTANTE PER L'ESAME

Il Mutation Testing valuta la qualità di una test suite esistente introducendo piccoli difetti artificiali nel SUT e verificando se i test riescono a rilevarli.

## 24.3 Concetti fondamentali

Nel Mutation Testing si usano alcuni concetti ricorrenti.

$P$  – **SUT originale**: è il programma di partenza, assunto per ipotesi come corretto.

$M$  – **Mutante**: è una variante del SUT in cui è stata introdotta una sola alterazione artificiale.

$T$  – **Test set**: è l'insieme dei test usati per valutare il programma originale e i mutanti.

$D$  – **Mutanti killed o distinguished**: sono i mutanti rivelati dai test, cioè quelli per cui almeno un test produce un comportamento diverso rispetto al programma originale.

$L$  – **Mutanti live**: sono i mutanti vivi, cioè quelli che nessun test è riuscito a distinguere dal programma originale.

$E$  – **Mutanti equivalenti**: sono mutanti sintatticamente diversi dal programma originale, ma semanticamente equivalenti, quindi indistinguibili da qualunque test.

#### Solo nelle slide (non spiegato a voce)

Il test set  $T$  può essere visto come insieme di tuple di input oppure come insieme di *test program* dotati di **assert**. Nel secondo caso, i test devono passare sul programma originale  $P$  e fallire sul mutante  $M$ , se la mutazione è osservabile.

#### Nota del Prof

Ogni mutante dovrebbe contenere una sola alterazione. Se un mutante contenesse più modifiche, un errore potrebbe mascherarne un altro e rendere difficile capire quale modifica sia stata effettivamente rilevata dai test.

## 24.4 Processo di Mutation Testing

Il processo di Mutation Testing può essere descritto come una sequenza di passi.

1. Si esegue il test set  $T$  sul programma originale  $P$ . I test devono passare, perché  $P$  viene assunto come comportamento di riferimento.
2. Si generano i mutanti  $M$ , ciascuno ottenuto introducendo una piccola alterazione artificiale nel SUT.
3. I mutanti generati vengono inseriti nell'insieme dei mutanti vivi  $L$ .
4. Per ogni mutante  $M$ , si eseguono i test del test set  $T$ .
5. Per ogni test  $t$ , si confronta il comportamento del programma originale con quello del mutante:

$$P(t) = M(t)$$

6. Se almeno un test produce un comportamento diverso:

$$P(t) \neq M(t)$$

allora il mutante viene considerato **killed** o **distinguished** e viene spostato nell'insieme  $D$ .

7. Se invece nessun test riesce a distinguere il mutante dal programma originale, il mutante resta vivo.
8. Al termine del processo, i mutanti vivi vengono analizzati per individuare eventuali mutanti equivalenti  $E$ .
9. Infine, si calcola il **Mutation Score**.

Solo nelle slide (non spiegato a voce)

Nel diagramma del processo, le linee continue rappresentano il flusso di controllo, mentre le linee tratteggiate rappresentano il trasferimento di dati o artefatti.

### IMPORTANTE PER L'ESAME

Un mutante viene ucciso quando almeno un test riesce a distinguere il comportamento del mutante dal comportamento del programma originale.

## 24.5 Mutation Score

Il **Mutation Score** misura la capacità della test suite di uccidere i mutanti generati. Una prima formulazione è:

$$MS = \frac{|D|}{|L| + |D|}$$

dove:

- $|D|$  è il numero di mutanti uccisi;
- $|L|$  è il numero di mutanti vivi non equivalenti.

Una formulazione alternativa, che tiene conto dei mutanti equivalenti, è:

$$MS = \frac{|D|}{|M| - |E|}$$

dove:

- $|M|$  è il numero totale di mutanti generati;
- $|E|$  è il numero di mutanti equivalenti;
- $|M| - |E|$  rappresenta il numero di mutanti effettivamente rilevabili.

L'interpretazione è la seguente:

- **Mutation Score = 1:** la test suite ha ucciso tutti i mutanti non equivalenti ed è considerata adeguata rispetto al criterio di mutation testing;
- **Mutation Score = 0:** nessun mutante è stato ucciso, quindi la test suite non è riuscita ad accorgersi di alcuna alterazione artificiale.

#### Nota del Prof

I mutanti equivalenti non possono essere rivelati da nessun test. Se venissero conteggiati nel denominatore, il Mutation Score risulterebbe artificialmente basso, penalizzando la test suite per un limite teorico del generatore di mutanti.

#### IMPORTANTE PER L'ESAME

I mutanti equivalenti devono essere esclusi dal calcolo del Mutation Score, perché nessun test potrebbe distinguerli dal programma originale.

## 24.6 Condizioni per uccidere un mutante

Perché un mutante venga ucciso, devono essere soddisfatte contemporaneamente tre condizioni.

- C1 – Raggiungibilità:** deve esistere un cammino di esecuzione che raggiunga lo `statement` mutato.
- C2 – Infezione dello stato:** dopo l'esecuzione dello `statement` mutato, lo stato interno del mutante deve differire dallo stato interno del programma originale.
- C3 – Propagazione dello stato:** la differenza di stato deve propagarsi fino a un punto osservabile, come l'output finale o una `assert`.

Se un test raggiunge il codice mutato ma la mutazione non altera lo stato, il mutante non viene ucciso. Se altera lo stato ma tale differenza viene sovrascritta prima dell'osservazione, il mutante continua comunque a sopravvivere.

### IMPORTANTE PER L'ESAME

Per uccidere un mutante non basta eseguire la riga mutata: la mutazione deve modificare lo stato e tale modifica deve propagarsi fino a un comportamento osservabile.

### IMPORTANTE PER L'ESAME

Se non esiste alcun test nel dominio degli input capace di soddisfare contemporaneamente C1, C2 e C3, allora il mutante è equivalente.

## 24.7 Principi importanti

### 24.7.1 Competent Programmer Hypothesis

La **Competent Programmer Hypothesis** afferma che un programmatore competente produce un programma non troppo distante dalla versione corretta.

L'idea alla base del Mutation Testing è che molti difetti reali siano piccoli errori locali, come operatori sbagliati, condizioni invertite o modifiche sintattiche minime. Per questo motivo, introdurre mutazioni artificiali piccole può aiutare a stimare la capacità della test suite di rilevare difetti realistici.

#### Nota del Prof

L'assunzione è che il programmatore abbia compreso l'algoritmo e che il programma scritto sia vicino a quello corretto, ma possa comunque contenere piccoli errori sintattici, logici o tipografici.

### 24.7.2 Indecidibilità dell'equivalenza

Stabilire in modo generale e automatico se un mutante è semanticamente equivalente al programma originale è un problema **indecidibile**.

Per questo motivo, nella pratica, l'identificazione dei mutanti equivalenti richiede spesso analisi manuale.

### IMPORTANTE PER L'ESAME

Il tester ha un ruolo importante nell'analisi dei mutanti sopravvissuti: deve capire se sono dovuti a test deboli, oracle incompleti, codice non raggiunto oppure mutanti equivalenti.

### 24.7.3 Strong mutation e weak mutation

Si distinguono due forme principali di mutation testing.

Nella **strong mutation**, un mutante è considerato ucciso solo se produce un comportamento osservabile esternamente diverso dal programma originale.

Nella **weak mutation**, invece, l'osservazione può fermarsi a stati intermedi interni, per esempio valori di variabili locali subito dopo lo statement mutato.

## IMPORTANTE PER L'ESAME

Un mutante può essere equivalente rispetto alla strong mutation, perché non cambia l'output osservabile, ma non equivalente rispetto alla weak mutation, perché produce uno stato interno diverso.

## 24.8 PIT / Pitest

**PIT**, o **Pitest**, è un framework Java usato per automatizzare il Mutation Testing.

PIT si integra con il ciclo di build e può essere usato insieme a Maven e Surefire. Il tool modifica il bytecode, genera mutanti, esegue i test e produce un report sui mutanti uccisi e sopravvissuti.

### Solo nelle slide (non spiegato a voce)

PIT si integra nel progetto Maven tramite il plugin `pitest-maven`. La configurazione può essere inserita nel `pom.xml`, specificando le classi da mutare e i test da eseguire.

Gli elementi principali della configurazione sono:

- `targetClasses`: indica le classi del SUT da mutare;
- `targetTests`: indica le classi di test da usare;
- `goal mutationCoverage`: avvia l'analisi di mutation testing;
- `directory target/pit-reports`: contiene il report HTML generato.

Il goal Maven può essere eseguito con un comando del tipo:

```
mvn org.pitest:pitest-maven:mutationCoverage
```

### Nota del Prof

Nel report HTML di PIT è possibile aprire una classe e osservare, riga per riga, quante mutazioni sono state generate, quante sono state uccise e quante sono sopravvissute. Questo permette di capire quali aspetti del codice hanno ingannato la test suite.

## 24.9 Collegamento con il progetto

Nel progetto, il Mutation Testing può essere usato per valutare la qualità reale dei test già scritti.

## IMPORTANTE PER L'ESAME

JaCoCo e PIT misurano aspetti diversi. JaCoCo misura l'esplorazione white-box del codice; PIT misura la capacità della test suite di rilevare difetti artificiali.

Una possibile strategia è usare JaCoCo per capire quali parti del codice non sono state ancora esplorate e PIT per capire se i test già presenti contengono oracle sufficientemente forti.

Nel report di progetto è utile indicare:

- il tool usato;
- il Mutation Score ottenuto;
- il numero di mutanti generati;
- il numero di mutanti uccisi;
- il numero di mutanti sopravvissuti;
- eventuali mutanti equivalenti o sospetti equivalenti;
- le cause principali dei mutanti sopravvissuti.

Se sopravvivono molti mutanti, bisogna analizzare il report HTML e distinguere tra diverse cause possibili:

- test senza `assert`;
- oracle deboli;
- codice eseguito ma non verificato;
- rami difficili da raggiungere;
- mutanti equivalenti;
- mutazioni in parti poco rilevanti del codice.

### **IMPORTANTE PER L'ESAME**

Non bisogna riportare solo il numero finale prodotto da PIT. Bisogna spiegare quali mutanti sono sopravvissuti e perché.

# Capitolo 25

## Coverage-Driven Test Generation

### 25.1 Adeguatezza dei test

Il concetto di **adeguatezza dei test** nasce dall'esigenza di capire se e quando sia possibile smettere di testare un **System Under Test** (SUT).

Le domande principali sono:

- i miei test sono sufficientemente buoni?
- il SUT è stato testato in modo abbastanza meticoloso?
- i test sono adeguati rispetto agli obiettivi prefissati?

Le tecniche di adequacy servono quindi a stimare, nel modo più oggettivo possibile, la qualità dell'attività di testing. In pratica, permettono di definire criteri di accettazione degli artefatti prodotti, cioè soglie o condizioni minime per dire che una test suite è sufficientemente adeguata.

#### Nota del Prof

L'adeguatezza viene spesso usata alla fine della catena di testing, come controllo a posteriori per dimostrare che il lavoro è stato fatto bene. In questa lezione, però, il problema viene ribaltato: l'adeguatezza può essere usata anche come **obiettivo guidato** per generare automaticamente i test fin dall'inizio.

#### IMPORTANTE PER L'ESAME

L'adequacy non dimostra l'assenza di bug. Serve a misurare quanto una test suite soddisfi certi criteri espliciti di esplorazione o verifica del SUT.

### 25.2 Functional vs Structural Adequacy

Le tecniche di adequacy possono essere divise in due grandi famiglie: **functional adequacy** e **structural adequacy**.

La **functional adequacy** è un criterio di tipo black-box. Si basa su requisiti, specifiche o modelli comportamentali del sistema. L'attenzione è rivolta ai comportamenti osservabili del SUT, senza considerare direttamente il codice sorgente.

Solo nelle slide (non spiegato a voce)

Il limite principale della functional adequacy è che non riesce a intercettare errori dovuti a dettagli implementativi introdotti dallo sviluppatore.

La **structural adequacy**, invece, è un criterio di tipo white-box. Si basa sull'implementazione eseguibile del SUT e cerca di attraversare la maggior parte possibile del codice reale.

Il suo limite è duplice: da un lato non può scoprire funzionalità completamente mancanti, perché se una funzionalità non è implementata non esiste codice da coprire; dall'altro, non esiste una garanzia matematica secondo cui un'alta coverage implichi necessariamente l'esposizione di una failure.

Nota del Prof

Con la coverage strutturale misuriamo la nostra confidenza nell'esplorazione del sistema, ma non misuriamo direttamente il reale modo d'uso funzionale del SUT né la capacità effettiva di trovare difetti.

IMPORTANTE PER L'ESAME

Alta coverage non significa automaticamente buona test suite. Una riga può essere eseguita senza essere realmente verificata da un'oracolo significativo.

## 25.3 Generazione automatica white-box

La **generazione automatica white-box** parte dall'idea di usare il codice del SUT come input per produrre automaticamente casi di test.

Invece di progettare manualmente test con l'obiettivo di coprire il codice, si fornisce a un algoritmo un criterio di coverage da massimizzare. Lo strumento analizza il codice e genera automaticamente test, per esempio classi JUnit.

Solo nelle slide (non spiegato a voce)

Un esempio mostrato nelle slide riguarda una funzione come `areBothParamsPositive(int i, int j)`, contenente rami logici basati su condizioni come  $i > 0$  e  $j > 0$ . Uno strumento coverage-driven può generare automaticamente test JUnit per coprire i branch della funzione.

Tuttavia, bisogna distinguere tra **generare input** e **generare veri test**. Generare input significa trovare valori che stimolano il SUT e attraversano certe parti del codice. Generare un vero test significa anche produrre un **oracle**, cioè un'asserzione che stabilisca quale comportamento sia corretto.

Nota del Prof

Molti strumenti automatici sono bravi a generare stimoli, cioè valori di input e sequenze di chiamate. Il problema più difficile è generare anche le **assert**, perché senza oracle il test esegue il codice ma non verifica davvero la correttezza del

risultato.

## 25.4 Approcci evolutivi

Gli approcci evolutivi alla generazione dei test rientrano nella **Search-Based Software Engineering**. L'idea è trattare la generazione dei test come un problema di ricerca: si parte da una popolazione di soluzioni candidate e la si evolve progressivamente verso test migliori.

Il processo generale include:

- una **popolazione iniziale** di test candidati, spesso generati in modo casuale;
- una **fitness function**, che misura la bontà di un test rispetto all'obiettivo, per esempio code coverage o fault coverage;
- operazioni di **crossover**, che combinano due test candidati per produrre nuovi test;
- operazioni di **mutazione dei test**, che alterano localmente un test;
- una fase di **selezione**, in cui vengono mantenuti i candidati migliori;
- un criterio di arresto, come il raggiungimento dell'ottimalità o l'esaurimento del budget computazionale.

### Nota del Prof

La mutazione dei test serve anche a evitare che l'algoritmo si blocchi in ottimi locali. Alterando casualmente una soluzione, l'algoritmo può esplorare zone dello spazio di ricerca che altrimenti non raggiungerebbe.

Il vantaggio principale di questi approcci è la loro genericità: possono essere automatizzati e applicati a molti SUT diversi. I limiti riguardano invece la difficoltà di modellare il software come problema evolutivo, la scelta della fitness function, la taratura degli iper-parametri e, soprattutto, la generazione degli oracle.

### IMPORTANTE PER L'ESAME

Negli approcci evolutivi, ottimizzare la coverage non basta. Una test suite generata automaticamente deve anche contenere asserzioni utili e non fragili.

## 25.5 Definizione automatica degli oracle

La definizione automatica degli **oracle** è uno dei problemi centrali della generazione white-box.

Trovare una sequenza di chiamate che raggiunge una certa riga di codice è utile solo in parte. Se alla fine il test non controlla l'effetto prodotto, allora non è realmente in grado di rivelare una failure.

Le asserzioni possono essere generate osservando:

- valori di ritorno dei metodi;

- stato pubblico dell'istanza;
- API pubbliche del SUT;
- valori osservabili dopo l'esecuzione del test.

Tuttavia, questa generazione automatica può produrre problemi. Alcune asserzioni possono essere irrilevanti, superflue o troppo legate a dettagli interni dell'implementazione. In questi casi, il test diventa fragile: può fallire dopo un refactoring corretto, anche se il comportamento funzionale del sistema non è cambiato.

#### Solo nelle slide (non spiegato a voce)

Le slide evidenziano il rischio di generare troppe asserzioni o asserzioni irrilevanti, che aumentano il rumore invece di migliorare davvero la qualità della test suite.

#### Nota del Prof

Lo sviluppatore deve comunque controllare le asserzioni generate. Un'asserzione prodotta automaticamente non è automaticamente significativa: deve essere collegata a un requisito, a una proprietà attesa o a un comportamento realmente rilevante.

## 25.6 Mutazioni e asserzioni

Per generare asserzioni più utili, alcuni approcci coverage-driven sfruttano idee derivate dal **Mutation Testing**.

#### IMPORTANTE PER L'ESAME

Bisogna distinguere chiaramente tra **mutazione dei test** e **mutazione del SUT**. La mutazione dei test riguarda modifiche casuali alla popolazione di test candidati. La mutazione del SUT riguarda invece alterazioni artificiali del codice sorgente per simulare fault.

L'idea è assumere che il SUT originale sia corretto, introdurre artificialmente una mutazione nel SUT e poi eseguire la stessa sequenza di test sia sul programma originale sia sul mutante.

Se il comportamento osservabile diverge, il tool può usare quella divergenza per generare un'asserzione. In altre parole, l'asserzione viene costruita in modo da accettare il comportamento del SUT originale e rifiutare quello del mutante.

Il processo può essere sintetizzato così:

1. si genera una sequenza di test;
2. si esegue la sequenza sul SUT originale;
3. si esegue la stessa sequenza su un mutante del SUT;
4. si confrontano ritorni, stato e valori osservabili;
5. si genera un'asserzione che cattura la differenza significativa.

### Nota del Prof

Un mutante viene rilevato solo se esiste un oracle capace di osservare la divergenza. Se il test attraversa il codice mutato ma non controlla il valore modificato, il mutante può sopravvivere.

## 25.7 Test come sequenze di istruzioni

Negli approcci evolutivi, un test viene modellato come una **sequenza di istruzioni**. Questa modellazione permette agli algoritmi di generare, combinare e modificare test in modo automatico.

### Solo nelle slide (non spiegato a voce)

Le slide distinguono tra **constructor statements**, **method statements**, **field statements** e **primitive statements**.

I **constructor statements** creano nuove istanze, per esempio tramite **new**. I **method statements** invocano metodi su oggetti già creati. I **field statements** accedono a campi pubblici, mentre i **primitive statements** definiscono valori primitivi, come interi, booleani o stringhe.

La generazione deve rispettare alcune regole. Ogni istruzione produce una nuova variabile riutilizzabile. I parametri passati a un metodo devono riferirsi a variabili già dichiarate, altrimenti il codice generato sarebbe sintatticamente non valido.

Inoltre, possono essere coinvolte anche classi dipendenti dal SUT. Questo è necessario perché molte API richiedono oggetti complessi come parametri; quindi, prima di invocare il metodo da testare, bisogna costruire uno stato valido e coerente.

### IMPORTANTE PER L'ESAME

Generare test automatici non significa solo scegliere valori primitivi. Spesso bisogna costruire intere sequenze di oggetti e chiamate per portare il SUT in uno stato utile al test.

## 25.8 EvoSuite

**EvoSuite** è uno strumento per la generazione automatica di test suite JUnit. Nasce da ricerche accademiche e implementa un approccio coverage-based alla generazione dei test.

### Solo nelle slide (non spiegato a voce)

EvoSuite lavora a livello di **bytecode Java**, cioè sui file **.class**. Per questo non richiede necessariamente il codice sorgente del SUT o delle librerie.

Il suo obiettivo è generare automaticamente una test suite cercando di:

- massimizzare la code coverage;
- generare test JUnit eseguibili;

- produrre asserzioni;
- minimizzare il numero di asserzioni non necessarie.

EvoSuite può essere usato in diverse modalità operative: stand-alone da riga di comando, tramite plugin per Eclipse, tramite plugin per IntelliJ o tramite plugin Maven.

#### IMPORTANTE PER L'ESAME

EvoSuite mostra bene il compromesso della generazione automatica: può produrre molti test e aumentare la coverage, ma le asserzioni generate devono essere controllate criticamente.

## 25.9 Collegamento con il progetto

Nel progetto è utile distinguere tra test scritti manualmente, test generati con strumenti randomici come Randoop, test prodotti da LLM e test generati da strumenti coverage-driven come EvoSuite.

I test manuali permettono di ragionare sulle specifiche e sulle classi di equivalenza. Randoop genera sequenze random guidate dal feedback. Gli LLM possono produrre test a partire da codice e prompt, ma richiedono validazione. EvoSuite e gli strumenti coverage-driven, invece, usano la coverage come obiettivo esplicito di generazione.

#### IMPORTANTE PER L'ESAME

Il peggior errore è fidarsi solo della coverage. Una test suite automatica può coprire molto codice ma contenere oracle deboli, asserzioni fragili o test poco significativi.

Nel report di progetto bisogna documentare:

- il tool usato;
- la configurazione adottata;
- il criterio di coverage considerato;
- i test generati;
- il tipo e la qualità delle asserzioni;
- i limiti osservati;
- eventuali test scartati o modificati.

#### Nota del Prof

Includere test automatici che fanno molte chiamate ma non contengono nessuna `assert` significativa significa non aver compreso lo scopo del testing. Un test deve stimolare il SUT, ma deve anche verificare il risultato.

# Capitolo 26

## Evosuite Test Generation

### 26.1 Adeguatezza dei test

Il concetto di **adeguatezza dei test** nasce dall'esigenza di capire se e quando sia possibile smettere di testare un **System Under Test** (SUT).

Le domande principali sono:

- i miei test sono sufficientemente buoni?
- il SUT è stato testato in modo abbastanza meticoloso?
- i test sono adeguati rispetto agli obiettivi prefissati?

Le tecniche di adequacy servono quindi a stimare, nel modo più oggettivo possibile, la qualità dell'attività di testing. In pratica, permettono di definire criteri di accettazione degli artefatti prodotti, cioè soglie o condizioni minime per dire che una test suite è sufficientemente adeguata.

#### Nota del Prof

L'adeguatezza viene spesso usata alla fine della catena di testing, come controllo a posteriori per dimostrare che il lavoro è stato fatto bene. In questa lezione, però, il problema viene ribaltato: l'adeguatezza può essere usata anche come **obiettivo guidato** per generare automaticamente i test fin dall'inizio.

#### IMPORTANTE PER L'ESAME

L'adequacy non dimostra l'assenza di bug. Serve a misurare quanto una test suite soddisfi certi criteri espliciti di esplorazione o verifica del SUT.

### 26.2 Functional vs Structural Adequacy

Le tecniche di adequacy possono essere divise in due grandi famiglie: **functional adequacy** e **structural adequacy**.

La **functional adequacy** è un criterio di tipo black-box. Si basa su requisiti, specifiche o modelli comportamentali del sistema. L'attenzione è rivolta ai comportamenti osservabili del SUT, senza considerare direttamente il codice sorgente.

Solo nelle slide (non spiegato a voce)

Il limite principale della functional adequacy è che non riesce a intercettare errori dovuti a dettagli implementativi introdotti dallo sviluppatore.

La **structural adequacy**, invece, è un criterio di tipo white-box. Si basa sull'implementazione eseguibile del SUT e cerca di attraversare la maggior parte possibile del codice reale.

Il suo limite è duplice: da un lato non può scoprire funzionalità completamente mancanti, perché se una funzionalità non è implementata non esiste codice da coprire; dall'altro, non esiste una garanzia matematica secondo cui un'alta coverage implichi necessariamente l'esposizione di una failure.

Nota del Prof

Con la coverage strutturale misuriamo la nostra confidenza nell'esplorazione del sistema, ma non misuriamo direttamente il reale modo d'uso funzionale del SUT né la capacità effettiva di trovare difetti.

IMPORTANTE PER L'ESAME

Alta coverage non significa automaticamente buona test suite. Una riga può essere eseguita senza essere realmente verificata da un oracolo significativo.

## 26.3 Generazione automatica white-box

La **generazione automatica white-box** parte dall'idea di usare il codice del SUT come input per produrre automaticamente casi di test.

Invece di progettare manualmente test con l'obiettivo di coprire il codice, si fornisce a un algoritmo un criterio di coverage da massimizzare. Lo strumento analizza il codice e genera automaticamente test, per esempio classi JUnit.

Solo nelle slide (non spiegato a voce)

Un esempio mostrato nelle slide riguarda una funzione come `areBothParamsPositive(int i, int j)`, contenente rami logici basati su condizioni come  $i > 0$  e  $j > 0$ . Uno strumento coverage-driven può generare automaticamente test JUnit per coprire i branch della funzione.

Tuttavia, bisogna distinguere tra **generare input** e **generare veri test**. Generare input significa trovare valori che stimolano il SUT e attraversano certe parti del codice. Generare un vero test significa anche produrre un **oracle**, cioè un'asserzione che stabilisca quale comportamento sia corretto.

Nota del Prof

Molti strumenti automatici sono bravi a generare stimoli, cioè valori di input e sequenze di chiamate. Il problema più difficile è generare anche le **assert**, perché senza oracle il test esegue il codice ma non verifica davvero la correttezza del

risultato.

## 26.4 Approcci evolutivi

Gli approcci evolutivi alla generazione dei test rientrano nella **Search-Based Software Engineering** (SBSE). L'idea è trattare la generazione dei test come un problema di ricerca e ottimizzazione combinatoria: si parte da una popolazione di soluzioni candidate e la si evolve progressivamente verso test migliori tramite meta-euristiche.

Il processo generale include:

- una **popolazione iniziale** di test candidati, modellati matematicamente come vettori di istruzioni e generati in modo casuale o basati su pattern esistenti;
- una **fitness function**, che misura quantitativamente la bontà di un test rispetto all'obiettivo prefissato (es. massimizzare la code o fault coverage);
- operazioni di **crossover**, che combinano probabilisticamente due genitori tagliando e mischiando le sequenze di istruzioni per produrre nuovi elementi;
- operazioni di **mutazione dei test**, che alterano localmente un test;
- una fase di **selezione**, in cui vengono mantenuti i candidati migliori regolati dalla fitness;
- un criterio di arresto, come il raggiungimento dell'ottimalità o l'esaurimento del budget computazionale.

### Nota del Prof

La mutazione dei test serve anche a evitare che l'algoritmo si blocchi in ottimi locali. Alterando casualmente una soluzione, l'algoritmo può esplorare zone dello spazio di ricerca che altrimenti non raggiungerebbe.

Il vantaggio principale di questi approcci è la loro genericità e la facilità di automazione su molti SUT diversi. I limiti riguardano invece la difficoltà di modellare la logica del software, la scelta della fitness function ideale, la taratura degli iper-parametri e la sintesi degli oracle.

### IMPORTANTE PER L'ESAME

Negli approcci evolutivi, ottimizzare la coverage non basta. Una test suite generata automaticamente deve anche contenere asserzioni utili e non fragili.

## 26.5 Definizione automatica degli oracle

La definizione automatica degli **oracle** è uno dei problemi centrali della generazione white-box.

Trovare una sequenza di chiamate che raggiunge una certa riga di codice è utile solo in parte. Se alla fine il test non controlla l'effetto prodotto, allora non è realmente in grado di rivelare una failure.

Le asserzioni possono essere generate osservando:

- valori di ritorno dei metodi;
- stato pubblico dell'istanza;
- API pubbliche del SUT e delle sue classi dipendenti;
- valori osservabili dopo l'esecuzione del test.

Tuttavia, questa generazione automatica a tappeto pone problemi. Molte asserzioni possono essere irrilevanti, superflue o troppo legate a dettagli interni dell'implementazione. In questi casi, il test diventa fragile: può fallire dopo un refactoring corretto, anche se il comportamento funzionale del sistema non è cambiato.

#### Solo nelle slide (non spiegato a voce)

Le slide evidenziano il rischio di generare troppe asserzioni o asserzioni irrilevanti, che aumentano il rumore e lo sforzo di manutenzione invece di migliorare davvero la qualità della test suite.

#### Nota del Prof

Lo sviluppatore deve comunque controllare formalmente le asserzioni generate. Un'asserzione prodotta automaticamente non è intrinsecamente significativa: deve essere collegata a un requisito, a una proprietà attesa o a un comportamento realmente rilevante del sistema.

## 26.6 Mutazioni e asserzioni

Per generare asserzioni stabili e mirate, approcci avanzati come il framework accademico  $\mu$ TEST sfruttano i concetti del **Mutation Testing** (l'introduzione artificiale di singoli difetti noti nel SUT per simulare fault).

#### IMPORTANTE PER L'ESAME

Bisogna distinguere chiaramente tra **mutazione dei test** e **mutazione del SUT**. La mutazione dei test riguarda modifiche casuali alla popolazione di test candidati all'interno dell'algoritmo evolutivo. La mutazione del SUT riguarda invece alterazioni artificiali del codice sorgente per simulare fault reali.

L'idea consiste nell'assumere che il SUT originale sia corretto, introdurre una mutazione nel SUT e poi eseguire la stessa sequenza di test sia sul programma originale sia sul mutante. Se il comportamento osservabile differisce, lo strumento sfrutta questa esatta divergenza per sintetizzare un'asserzione differenziale, costruita per accettare il comportamento del SUT originale e rifiutare quello del mutante.

Il paper di Fraser e Zeller integra questo bilanciamento direttamente nella fitness function complessiva da massimizzare:

$$fitness(t) = \frac{1}{D_f(t) + D_m(t)} + I_m(t) \quad (26.1)$$

Dove  $D_f(t)$  rappresenta la distanza dalla chiamata del metodo mutato,  $D_m(t)$  è la distanza dalla riga esatta mutata (branch/necessity distance), e  $I_m(t)$  rappresenta l'impatto della mutazione (sufficiency condition) espresso come:

$$I_m(t) = \frac{c \times |C| + r \times |A|}{1 + |t|} \quad (26.2)$$

Qui,  $C$  è l'insieme di statement con coverage modificata,  $A$  l'insieme delle differenze osservabili, e  $|t|$  la lunghezza del test.

Il processo può essere sintetizzato così:

1. si genera una sequenza di test ottimizzata per raggiungere il codice difettoso;
2. si esegue la sequenza sul SUT originale registrandone la traccia;
3. si esegue la stessa sequenza su un mutante del SUT;
4. si confrontano ritorni, stato e valori osservabili;
5. si genera un'asserzione ottimizzata tramite euristica greedy (*Minimum Set Covering Problem*) per scartare i controlli ridondanti.

#### Nota del Prof

Un mutante viene rilevato solo se esiste un oracle capace di osservare la divergenza. Se il test attraversa il codice mutato ma non controlla il valore modificato, il mutante può sopravvivere, evidenziando una debolezza intrinseca della test suite.

## 26.7 Test come sequenze di istruzioni

Negli approcci evolutivi, un test viene modellato come una **sequenza di istruzioni** (*statements*) fortemente interdipendenti. Questa astrazione permette agli algoritmi di generare, combinare e modificare test in modo valido.

#### Solo nelle slide (non spiegato a voce)

Le slide e il paper distinguono tra quattro tipologie di istruzioni utilizzabili:

- **Constructor statements:** creano nuove istanze delle classi (`new Object()`).
- **Method statements:** invocano metodi (statici o di istanza) su oggetti esistenti.
- **Field statements:** accedono direttamente a campi pubblici.
- **Primitive statements:** definiscono costanti e tipi primitivi (interi, booleani, stringhe).

La generazione deve rispettare regole sintattiche stringenti per produrre codice compilabile: ogni istruzione definisce una nuova variabile riutilizzabile (tranne i metodi `void`) e i parametri passati a un metodo o costruttore possono fare riferimento esclusivamente a variabili già dichiarate in precedenza nello stesso cromosoma.

Qualora un metodo richieda parametri complessi, l'algoritmo ricorsivo (**GenObject** nel paper) effettua una ricerca a ritroso inserendo le necessarie chiamate a classi dipendenti dal SUT pur di istanziare oggetti in uno stato interno coerente e valido per il test.

### IMPORTANTE PER L'ESAME

Generare test automatici orientati agli oggetti non significa solo scegliere valori primitivi. Spesso bisogna costruire intere sequenze complesse di oggetti e chiamate per portare il SUT in uno stato utile al test.

## 26.8 EvoSuite

**EvoSuite** rappresenta lo stato dell'arte industriale per la generazione automatica di intere test suite JUnit per sistemi Java, basandosi su un approccio completamente coverage-driven.

### Solo nelle slide (non spiegato a voce)

EvoSuite lavora e analizza direttamente a livello di **bytecode Java** (sui file `.class`). Grazie a questo approccio basato sulla manipolazione del bytecode a runtime tramite reflection, non richiede necessariamente la disponibilità dei file sorgente `.java` o delle librerie dipendenti.

L'algoritmo di EvoSuite ottimizza l'intera suite di test per raggiungere molteplici obiettivi:

- massimizzare la code coverage (branch, statement, mutation coverage);
- generare test JUnit indipendenti ed eseguibili in modo deterministico;
- produrre asserzioni e oracoli mirati;
- minimizzare la lunghezza dei test e rimuovere le asserzioni superflue tramite tecniche di riduzione (*assertion minimization*).

Nel celebre paper di valutazione su librerie reali ampiamente testate, gli esperimenti hanno dimostrato che la generazione guidata dai mutanti supera sistematicamente le suite scritte a mano in termini di *mutation score*:

- In **Joda-Time**, il tasso di mutanti rilevati è salito dal 74.26
- In **Commons-Math**, il punteggio è cresciuto significativamente dal 41.25

EvoSuite può essere integrato tramite riga di comando (stand-alone), plugin Maven o estensioni per IDE quali Eclipse e IntelliJ IDEA.

### IMPORTANTE PER L'ESAME

EvoSuite mostra bene il compromesso della generazione automatica: può produrre molti test e aumentare rapidamente la coverage su rami complessi, ma le asserzioni generate devono essere controllate criticamente per verificarne la reale semantica.

## 26.9 Collegamento con il progetto

Nel contesto delle attività pratiche del progetto d'esame (es. analisi su Apache Syncope), è fondamentale saper isolare, confrontare e documentare criticamente le diverse metodologie di produzione dei test:

1. **Test Manuali:** permettono di ragionare formalmente sui requisiti, sulle specifiche e sulle classi di equivalenza;
2. **Test Randomici (es. Randoop):** generano sequenze di codice guidate dal feedback di esecuzione a runtime per individuare crash e violazioni;
3. **Test da LLM:** producono casi di test a partire da prompt linguistici e pattern, richiedendo attenta validazione manuale;
4. **Test Coverage-Driven (es. EvoSuite):** usano la metrica di copertura del codice come una funzione obiettivo matematica da massimizzare deterministicamente.

### IMPORTANTE PER L'ESAME

Il peggior errore metodologico in un progetto è fidarsi ciecamente della sola metrica della coverage. Una test suite automatica può coprire molto codice ma contenere oracle deboli, asserzioni fragili o test poco significativi che non riveleranno mai un vero difetto.

Nel report di progetto finale è obbligatorio documentare in modo strutturato:

- il tool specifico e la versione adottata;
- la configurazione adottata (iper-parametri, budget temporale di ricerca);
- il criterio di coverage considerato come target;
- la struttura dei test e delle classi JUnit generate;
- il tipo e la qualità delle asserzioni sintetizzate;
- i limiti pratici osservati (es. gestione del non-determinismo, classi astratte o blocchi statici);
- l'elenco motivato di eventuali test scartati o modificati manualmente.

### Nota del Prof

Includere acriticamente nel report di progetto test automatici che fanno chiamate a vuoto senza contenere alcuna istruzione di `assert` significativa significa non aver compreso lo scopo ultimo del testing. Un test deve stimolare il SUT, ma acquisisce valore ingegneristico solo se ne verifica rigorosamente il risultato.